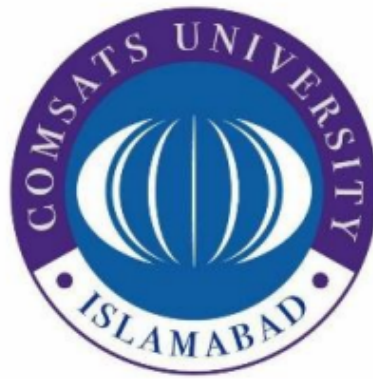


**COMSATS University Islamabad, Vehari Campus,
Pakistan**

Department of Computer Science

**EasyRide – An Intelligent Ride-Hailing Platform with
Real-Time Tracking, AI Assistance, and Dynamic Fare
Negotiation**



A Final Year Project Report Submitted in Partial Fulfillment of the
Requirements
for the Degree of

Bachelor of Science in Computer Science

Submitted By

M. Numan (CIIT/FA22-BCS-036/VHR)

M. Tayyab (CIIT/FA22-BCS-018/VHR)

Supervisor

Dr. Dewan Qasim

Session: Spring 2022-2026

Dedication

We dedicate this work to our parents, whose unwavering support, sacrifices, and prayers have been the foundation of our success. Their encouragement, patience, and belief in our abilities have guided us throughout this journey.

We also dedicate this to our teachers and mentors who inspired us to pursue knowledge with dedication and integrity.

Acknowledgement

All praise and gratitude are due for the successful completion of this project. The authors would like to express their sincere appreciation to their supervisor for continuous guidance, valuable suggestions, and constructive feedback throughout the development of this thesis. The supervision and mentorship provided significant support in completing this work successfully.

Special thanks are extended to the faculty members of the Department of Computer Science for their academic support and encouragement during the degree program.

The authors are also deeply grateful to their parents and families for their unwavering support, motivation, and prayers, which have been a constant source of strength.

Finally, appreciation is extended to everyone who directly or indirectly contributed to the successful completion of this project.

Certificate of Approval

This is to certify that the Final Year Project titled **“EasyRide – An Intelligent Ride-Hailing Platform with Real-Time Tracking, AI Assistance, and Dynamic Fare Negotiation”** has been developed by **M. Numan (CIIT/FA22-BCS-036/VHR)** and **M. Tayyab (CIIT/FA22-BCS-018/VHR)** under the supervision of **Dr. Dewan Qasim** and that in his opinion, the project is adequate in scope and quality for the award of the degree of Bachelor of Science in Computer Science.

Supervisor

External Examiner

Head of Department
Department of Computer Science

Plagiarism and AI Content Certificate

Date: _____

It is hereby certified that the similarity index (as generated by the Turnitin **Final Year Project Report** submitted by:

- M. Numan (CIIT/FA22-BCS-036/VHR)
- M. Tayyab (CIIT/FA22-BCS-018/VHR)

titled:

“EasyRide – An Intelligent Ride-Hailing Platform with Real-Time Tracking, AI Assistance, and Dynamic Fare Negotiation”

is [XX%] which is within the permissible limits as defined by the Higher Education Commission (HEC) rules and regulations. The similarity index report is attached herewith.

Furthermore, the AI-generated content detection index (if applicable as per institutional policy) is reported as [XX%]. The report has been reviewed, and the content is found to comply with university guidelines regarding ethical use of Artificial Intelligence tools.

The report is recommended for submission.

Supervisor Name: _____

Designation: _____

Signature: _____

Department: Department of Computer Science

University: COMSATS University Islamabad, Vehari Campus

Executive Summary

The transportation sector in Pakistan faces significant challenges related to ride-hailing services. Existing platforms such as Uber and Careem, while widely used in major cities, often fall short in areas like transparent fare negotiation, real-time safety mechanisms, and responsive customer support. Many local commuters struggle with unpredictable pricing, limited communication channels during rides, and a lack of intelligent assistance when issues arise. These gaps motivated the development of EasyRide, a locally-focused ride-hailing platform designed to address these shortcomings through modern web technologies and intelligent features.

EasyRide is a full-stack web application that connects passengers with drivers through a feature-rich, real-time platform. The system supports three distinct user roles: passengers who book rides, drivers who accept and fulfill ride requests, and administrators who oversee platform operations. Each role has a dedicated interface tailored to its specific needs.

The platform introduces several distinguishing features that set it apart from existing solutions. A dynamic fare negotiation system allows drivers to propose counter-offers on ride fares, which passengers can accept or decline in real time through WebSocket-powered communication. The system integrates an AI-powered assistant built on OpenAI's GPT-3.5 model, providing passengers and drivers with instant help regarding ride queries, payment issues, and general platform guidance. Real-time GPS tracking enables passengers to monitor their ride progress through Google Maps integration, while drivers continuously broadcast their location for accurate proximity-based matching.

Safety has been a core design consideration. An SOS emergency alert system allows both passengers and drivers to trigger distress signals during a ride, which are immediately broadcast to the admin dashboard along with GPS coordinates. In-ride chat functionality enables direct communication between the passenger and driver without sharing personal phone numbers. Additionally, a comprehensive support ticket system allows users to report issues categorized by type, which administrators can review and resolve through the admin panel.

The backend is built using Node.js with Express.js, connected to a MongoDB database through Mongoose ODM. Real-time features are powered by Socket.IO for bidirectional WebSocket communication. The frontend is developed using React with Vite as the build tool and Tailwind

CSS for styling. External services include OpenRouteService and Nominatim for geocoding and routing, Cloudinary for image storage, Gmail SMTP through Nodemailer for email-based OTP verification, and the OpenAI API for the AI assistant.

The admin panel provides a centralized dashboard for monitoring platform statistics, managing driver verification workflows, configuring fare settings per vehicle type, processing driver withdrawal requests, handling support tickets, and responding to SOS alerts. Broadcast messaging capability allows administrators to send notifications to all users or all drivers simultaneously.

This project demonstrates the practical integration of several computer science concepts including software engineering, web development, database design, real-time communication, and artificial intelligence. EasyRide serves as a meaningful step toward creating a practical, locally relevant ride-hailing solution that addresses real gaps in the current transportation technology landscape in Pakistan.

Abbreviations

Abbreviation	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
BO	Business Objective
BSCS	Bachelor of Science in Computer Science
CDN	Content Delivery Network
CORS	Cross-Origin Resource Sharing
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
DB	Database
DFD	Data Flow Diagram
ERD	Entity Relationship Diagram
FR	Functional Requirement
GPS	Global Positioning System
GPT	Generative Pre-trained Transformer
GSAP	GreenSock Animation Platform
HTML	HyperText Markup Language
HTTPS	HyperText Transfer Protocol Secure
JSON	JavaScript Object Notation
JWT	JSON Web Token
MERN	MongoDB, Express.js, React, Node.js
NoSQL	Not Only SQL
ODM	Object Document Mapping
ORS	OpenRouteService
OSM	OpenStreetMap
OSRM	Open Source Routing Machine
OTP	One-Time Password
PKR	Pakistani Rupee
RBAC	Role-Based Access Control
REST	Representational State Transfer
SDD	Software Design Document
SMTP	Simple Mail Transfer Protocol
SOS	Save Our Souls (Emergency Alert)
SRS	Software Requirements Specification

Abbreviation	Full Form
STD	State Transition Diagram
TTL	Time To Live
UAT	User Acceptance Testing
UC	Use Case
UI	User Interface
UML	Unified Modeling Language
UX	User Experience

Table of Contents

Executive Summary	v
List of Tables	xiv
List of Figures	xvi
1 Introduction	1
1.1 Introduction	1
1.2 Vision Statement	1
1.3 Related System Analysis	2
1.4 Project Deliverables	3
1.5 System Limitations / Constraints	4
1.6 Tools and Technologies	5
1.7 Relevance to Course Modules	6
1.7.1 Programming Fundamentals	6
1.7.2 Data Structures and Algorithms	6
1.7.3 Database Management Systems	6
1.7.4 Software Engineering	6
1.7.5 Other Relevant Courses	7
1.8 Chapter Summary	7
2 Problem Definition	8
2.1 Problem Statement	8
2.2 Proposed Solution Overview	8
2.3 Objectives of the Proposed System	8
2.4 Scope of the System	9
2.5 System Architecture and Modules	9
2.5.1 Module 1: User Management and Authentication	9
2.5.2 Module 2: Ride Booking and Fare Negotiation	10
2.5.3 Module 3: Real-Time Tracking and Ride Execution	11
2.5.4 Module 4: AI Chatbot and Safety Features	11
2.5.5 Module 5: Admin Panel and Platform Management	12
2.6 Assumptions and Dependencies	12
2.7 Chapter Summary	13
3 Requirement Analysis	14
3.1 Introduction to Requirement Analysis	14
3.2 Requirement Elicitation Techniques	14
3.3 User Classes and Characteristics	14
3.4 System Overview	15

3.5	Use Case Model	15
3.5.1	Use Case Diagram	16
3.5.2	Use Case Descriptions	17
3.5.2.1	Use Case 1: Book a Ride with Fare Negotiation	17
3.5.2.2	Use Case 2: AI Chatbot Assistance	18
3.5.2.3	Use Case 3: Driver Verification	19
3.5.2.4	Use Case 4: Register Account	20
3.5.2.5	Use Case 5: Login / Logout	21
3.5.2.6	Use Case 6: Verify OTP	22
3.5.2.7	Use Case 7: Track Ride	23
3.5.2.8	Use Case 8: In-Ride Chat	24
3.5.2.9	Use Case 9: Rate and Review	25
3.5.2.10	Use Case 10: Trigger SOS	26
3.5.2.11	Use Case 11: Support Ticket	27
3.5.2.12	Use Case 12: Configure Fares	28
3.5.2.13	Use Case 13: Manage Users	29
3.5.2.14	Use Case 14: Process Withdrawals	30
3.5.3	Event-Response Table	31
3.6	Functional Requirements	31
3.6.1	Module 1: User Management and Authentication	32
3.6.1.1	FR-1.1 User Registration and Authentication	32
3.6.2	Module 2: Ride Booking and Fare Negotiation	32
3.6.2.1	FR-2.1 Ride Booking with Real-Time Fare Negotiation	32
3.6.3	Module 3: Ride Execution and Tracking	33
3.6.3.1	FR-3.1 Real-Time Tracking, OTP Verification, and Ride Completion	33
3.6.4	Module 4: AI Chatbot	33
3.6.4.1	FR-4.1 AI Chatbot Assistance	33
3.6.5	Module 5: Safety and Communication Features	34
3.6.5.1	FR-5.1 SOS Alerts, In-Ride Chat, Support Tickets, and Notifications	34
3.6.6	Module 6: Admin Panel	35
3.6.6.1	FR-6.1 Admin Management Operations	35
3.7	Non-Functional Requirements	35
3.7.1	Performance Requirements	35
3.7.2	Security Requirements	36
3.7.3	Reliability Requirements	36
3.7.4	Usability Requirements	36
3.7.5	Scalability Requirements	37
3.7.6	Maintainability Requirements	37
3.8	External Interface Requirements	37
3.8.1	User Interface Requirements	37
3.8.2	Hardware Interface Requirements	38
3.8.3	Software Interface Requirements	38
3.8.4	Communication Interface Requirements	38
3.9	Assumptions and Dependencies	38
3.10	Requirement Traceability Matrix	39

3.11	Chapter Summary	39
4	Software Design	41
4.1	Introduction to Software Design	41
4.1.1	Purpose of this Document	41
4.1.2	Intended Audience	41
4.1.3	Document Overview	41
4.2	Design Methodology and Software Process Model	42
4.2.1	Design Methodology	42
4.2.2	Software Process Model	42
4.3	System Overview	43
4.3.1	Presentation Layer (Frontend – React 18)	44
4.3.2	Application Layer (Backend API – Node.js and Express.js 5.1)	44
4.3.3	Real-Time Communication Layer (Socket.IO 4.8)	44
4.3.4	AI Service Layer (OpenAI GPT-3.5 Turbo)	44
4.3.5	Data Layer (MongoDB)	44
4.4	Architectural Design	44
4.5	Design Models	45
4.5.1	Activity Diagrams	45
4.5.1.1	Module 1: User Registration and Authentication	46
4.5.1.2	Module 2: Ride Booking and Fare Negotiation	47
4.5.1.3	Module 3: Admin Operations	48
4.5.2	Class Diagram	49
4.5.3	Sequence Diagrams	49
4.5.3.1	Authentication Module (Registration and Login)	50
4.5.3.2	Ride and AI Modules	51
4.5.3.3	Fare Negotiation and Driver Acceptance	52
4.5.3.4	Real-Time Chat	53
4.5.4	State Transition Diagrams	54
4.6	Data Flow Diagrams	58
4.6.1	Level 0 Data Flow Diagram	58
4.6.2	Level 1 Data Flow Diagram	59
4.7	Data Design	60
4.7.1	Entity Relationship Diagram (ERD)	60
4.7.2	Database Architecture	60
4.8	Database Schema Diagram	61
4.8.1	Data Dictionary	62
4.8.1.1	Entity: User	62
4.8.1.2	Entity: Ride	62
4.8.1.3	Entity: Driver	63
4.8.1.4	Entity: Admin	63
4.8.1.5	Entity: Message	64
4.8.1.6	Entity: SOS	64
4.8.1.7	Entity: OTP	64
4.8.1.8	Entity: Support Ticket	65
4.8.1.9	Entity: Notification	65

4.8.1.10	Entity: Withdrawal	65
4.8.1.11	Entity: Fare Setting	66
4.8.1.12	Entity: Blacklist Token	66
4.9	Database Tables / Collections	66
4.9.1	User Collection / Table	66
4.9.2	Core Data Collection	67
4.9.3	Relationships Between Tables	68
4.9.4	Security and Data Integrity	68
4.9.5	Scalability Considerations	68
4.10	Chapter Summary	69
5	Implementation	70
5.1	Development Environment	70
5.2	Core Module Implementation	71
5.2.1	Module 1: User and Driver Management	71
5.2.2	Module 2: Ride Booking and Fare Negotiation	71
5.2.3	Module 3: Ride Execution and Tracking	72
5.2.4	Module 4: AI Chatbot	73
5.2.5	Module 5: Safety and Communication Features	73
5.2.6	Module 6: Admin Panel	74
5.3	Algorithm Implementation	74
5.3.1	Algorithm 1: User Login Authentication	74
5.3.2	Algorithm 2: Fare Estimation	75
5.3.3	Algorithm 3: Nearby Driver Matching	76
5.3.4	Algorithm 4: OTP Verification for Ride Start	77
5.3.5	Algorithm 5: AI Chatbot Response	78
5.4	External APIs / SDKs	79
5.5	User Interface Implementation	80
5.5.1	UI Design Approach	80
5.5.2	Screen-wise Implementation	80
5.5.2.1	Screen 1: Login Page	80
5.5.2.2	Screen 2: Passenger Registration Page	81
5.5.2.3	Screen 3: Driver Registration Page	82
5.5.2.4	Screen 4: Passenger Dashboard	84
5.5.2.5	Screen 5: Driver Dashboard	85
5.5.2.6	Screen 6: Admin Dashboard	86
5.5.2.7	Screen 7: Ride Booking Page	86
5.5.2.8	Screen 8: AI Chatbot Page	88
5.5.2.9	Screen 9: Landing Page	88
5.5.2.10	Screen 10: Fare Negotiation	89
5.5.2.11	Screen 11: Ride Execution and OTP Verification	91
5.5.2.12	Screen 12: Ride History	93
5.5.2.13	Screen 13: User and Driver Profile	93
5.5.2.14	Screen 14: Navigation Bar	94
5.5.2.15	Screen 15: Additional Pages	94
5.5.3	Navigation Flow	96

5.5.4	Responsiveness and Performance	97
5.6	Deployment	97
5.7	Chapter Summary	98
6	Testing and Evaluation	99
6.1	Testing Strategy	99
6.2	Unit Testing	99
6.2.1	UT-01: Passenger Registration	100
6.2.2	UT-02: Driver Registration	100
6.2.3	UT-03: User Login	100
6.2.4	UT-04: Ride Booking	101
6.2.5	UT-05: Fare Estimation	101
6.2.6	UT-06: AI Chatbot	101
6.2.7	UT-07: OTP Verification	102
6.2.8	UT-08: SOS Emergency Alert	102
6.2.9	UT-09: Admin Driver Verification	102
6.3	Integration Testing	102
6.4	System Testing	103
6.4.1	ST-01: Passenger Complete Journey	103
6.4.2	ST-02: Driver Complete Journey	104
6.4.3	ST-03: Admin Complete Journey	104
6.4.4	ST-04: Edge Cases and Error Handling	105
6.5	Performance Testing	105
6.6	Security Testing	105
6.7	User Acceptance Testing	106
6.8	Testing Summary	107
7	Conclusion and Future Work	109
7.1	Conclusion	109
7.2	Future Work	110
7.3	Limitations	111
	Appendices	112
	Appendix A – Turnitin Similarity Report	112
	Appendix B – AI Writing Detection Report	113
	Appendix C – Source Code	114

List of Tables

1.1	Comparative Analysis of Existing Systems	3
1.2	Tools and Technologies Used in the Project	5
3.1	User Classes and Characteristics	15
3.2	UC-001: Book a Ride with Fare Negotiation	17
3.3	UC-002: AI Chatbot Assistance	18
3.4	UC-003: Driver Verification	19
3.5	UC-004: Register Account	20
3.6	UC-005: Login / Logout	21
3.7	UC-006: Verify OTP	22
3.8	UC-007: Track Ride	23
3.9	UC-008: In-Ride Chat	24
3.10	UC-009: Rate and Review	25
3.11	UC-010: Trigger SOS	26
3.12	UC-011: Support Ticket	27
3.13	UC-012: Configure Fares	28
3.14	UC-013: Manage Users	29
3.15	UC-014: Process Withdrawals	30
3.16	Event-Response Table	31
3.17	Functional Requirement – User Registration and Authentication	32
3.18	Functional Requirement – Ride Booking and Fare Negotiation	32
3.19	Functional Requirement – Ride Execution and Tracking	33
3.20	Functional Requirement – AI Chatbot Assistance	33
3.21	Functional Requirement – Safety and Communication Features	34
3.22	Functional Requirement – Admin Panel Operations	35
3.23	Requirement Traceability Matrix	39
4.1	Data Dictionary – User Collection	62
4.2	Data Dictionary – Ride Collection	62
4.3	Data Dictionary – Driver Collection	63
4.4	Data Dictionary – Admin Collection	63
4.5	Data Dictionary – Message Collection	64
4.6	Data Dictionary – SOS Collection	64
4.7	Data Dictionary – OTP Collection	64
4.8	Data Dictionary – Support Ticket Collection	65
4.9	Data Dictionary – Notification Collection	65
4.10	Data Dictionary – Withdrawal Collection	65
4.11	Data Dictionary – Fare Setting Collection	66

4.12	Data Dictionary – Blacklist Token Collection	66
4.13	User Table Structure	67
5.1	Development Environment	70
5.2	External APIs and Services Used	79
5.3	Deployment Details	98
6.1	Unit Test Cases – Passenger Registration	100
6.2	Unit Test Cases – Driver Registration	100
6.3	Unit Test Cases – User Login	100
6.4	Unit Test Cases – Ride Booking	101
6.5	Unit Test Cases – Fare Estimation	101
6.6	Unit Test Cases – AI Chatbot	101
6.7	Unit Test Cases – OTP Verification	102
6.8	Unit Test Cases – SOS Emergency Alert	102
6.9	Unit Test Cases – Admin Driver Verification	102
6.10	Integration Test Cases	103
6.11	Performance Metrics	105
6.12	Security Test Cases	106
6.13	User Acceptance Testing Feedback Summary	107
6.14	Testing Summary	107

List of Figures

3.1	Use Case Diagram of EasyRide System	16
4.1	Context Diagram of EasyRide System	43
4.2	System Architecture Diagram of EasyRide	45
4.3	Activity Diagram – User Registration and Authentication	46
4.4	Activity Diagram – Ride Booking and Fare Negotiation	47
4.5	Activity Diagram – Admin Operations	48
4.6	Complete Class Diagram of EasyRide System	49
4.7	Sequence Diagram – User Registration	50
4.8	Sequence Diagram – User Login	50
4.9	Sequence Diagram – AI Chatbot Interaction	51
4.10	Sequence Diagram – Ride Booking	51
4.11	Sequence Diagram – Fare Negotiation	52
4.12	Sequence Diagram – Driver Ride Acceptance	53
4.13	Sequence Diagram – Real-Time In-Ride Chat	53
4.14	STD – User Authentication Process	54
4.15	STD – Ride Booking Lifecycle	55
4.16	STD – Driver Verification Process	56
4.17	STD – User Profile Management	56
4.18	STD – Fare Negotiation Process	57
4.19	STD – Chat Conversation Lifecycle	57
4.20	Data Flow Diagram Level 0	58
4.21	Data Flow Diagram Level 1	59
4.22	Entity Relationship Diagram (ERD) of EasyRide	60
4.23	Database Schema Diagram of EasyRide	61
5.1	Login Page	81
5.2	Passenger Registration Page	82
5.3	Driver Registration Page – Personal Details	83
5.4	Driver Registration Page – Vehicle Details	83
5.5	Driver Registration Page – Document Upload	84
5.6	Passenger Dashboard	85
5.7	Driver Dashboard	86
5.8	Ride Booking – Location Selection	87
5.9	Ride Booking – Vehicle Selection and Fare Estimate	88
5.10	EasyRide Landing Page	89
5.11	Passenger Waiting for Driver Response	89
5.12	Passenger Receives Driver Offer	90

5.13 Driver Receives New Ride Request	90
5.14 Driver Fare Negotiation Screen	91
5.15 Ride OTP Verification Screen	91
5.16 Ride In-Progress with GPS Tracking	92
5.17 Ride Vehicle Details	92
5.18 Ride History Page	93
5.19 User/Driver Profile Page	94
5.20 User/Driver Navigation Bar	94
5.21 Contact Us Page	95
5.22 Terms and Conditions Page	95
5.23 Password Change Page	96
5.24 System Navigation Diagram of EasyRide	96

Chapter 1

Introduction

1.1 Introduction

Transportation remains a persistent challenge for millions of commuters in Pakistan. In most cities, passengers depend on informal transport modes such as rickshaws, local vans, and un-metered taxis. The process of finding a ride is unpredictable; passengers must stand by roadsides negotiating fares with drivers who may quote inflated prices, especially during peak hours or in unfamiliar areas. There is no standardized pricing mechanism, no way to verify a driver's credentials beforehand, and no safety mechanism if something goes wrong during the ride.

While international ride-hailing platforms such as Uber and Careem have made significant improvements in urban mobility, they still have notable limitations in the Pakistani context. Both platforms use fixed algorithmic pricing that leaves no room for negotiation, a practice deeply rooted in local commuting culture. Additionally, their customer support relies on automated chatbots that rarely address rider concerns effectively, and their presence in smaller cities and semi-urban areas remains limited.

In response to these challenges, EasyRide has been developed as a full-stack web-based ride-hailing platform built using the MERN stack (MongoDB, Express.js, React, and Node.js). The platform introduces a real-time fare negotiation system powered by Socket.IO, allowing passengers and drivers to agree on a mutually acceptable fare before the ride begins. An AI-powered chatbot built on OpenAI's GPT-3.5 provides instant support for common queries, route suggestions, and ride-related assistance. The platform also features real-time GPS tracking, in-ride messaging, SOS emergency alerts, OTP-based ride verification, and a comprehensive admin panel for platform management.

EasyRide is designed with three types of users: Passengers who book rides, Drivers who accept and complete rides, and Administrators who manage the system including driver verification, fare settings, and support tickets. The platform uses modern web technologies including React 18 with Tailwind CSS and GSAP animations, Node.js with Express.js 5.1, Socket.IO 4.8, Mongoose ODM, Cloudinary for image uploads, and OpenRouteService with Nominatim for geocoding and routing.

This chapter provides the background context of the project, outlines the vision behind EasyRide, analyzes existing related systems, and describes the tools and technologies used. It also discusses project deliverables, system constraints, and the relevance of the project to academic coursework.

1.2 Vision Statement

The overall vision behind EasyRide is to provide a transparent, safe, and user-friendly ride-hailing experience that addresses the specific needs of Pakistani commuters. The platform is meant to bridge the gap between traditional fare negotiation culture and modern technology-driven ride management.

In the long run, EasyRide aims to reduce the frustration of unpredictable pricing and unsafe travel by introducing a system where every fare is negotiated and agreed upon in real time, every driver is verified before being allowed on the platform, and every ride is trackable from start to finish. By providing an AI chatbot that can handle common queries instantly, the platform offers first-line support that is available around the clock.

What makes EasyRide unique is the combination of real-time fare negotiation with Socket.IO, AI-powered assistance focused on ride-related queries, and comprehensive safety features including SOS alerts and OTP verification, all integrated into a single cohesive platform. Rather than being just another taxi booking app, it brings together dynamic pricing, intelligent support, real-time communication, and platform management into a complete ride-hailing ecosystem.

1.3 Related System Analysis

Before developing EasyRide, several existing ride-hailing and transportation platforms were studied to understand what solutions are currently available and where they fall short. This analysis helped identify the specific gaps that EasyRide aims to fill.

Uber is the largest global ride-hailing platform, available in several Pakistani cities including Lahore, Islamabad, and Karachi. Uber provides a polished user experience with GPS tracking, in-app payments, and driver ratings. However, Uber uses a fixed surge-pricing algorithm that does not allow passengers to negotiate fares. In the Pakistani context, this rigidity often leads to perceived overcharging during peak hours. Additionally, Uber's support system relies on pre-scripted automated responses that rarely resolve specific complaints effectively.

Careem (now owned by Uber) is a Middle East and Pakistan focused ride-hailing service. Careem introduced features like cash payments and support for local phone numbers, making it more accessible than Uber in some regions. However, like Uber, Careem does not offer any form of fare negotiation. Its driver verification process, while present, lacks transparency from the rider's perspective. Careem's AI or chatbot support is limited, and real-time in-ride communication between driver and passenger is minimal.

InDriver is a ride-hailing platform that differs from Uber and Careem by allowing passengers to set their own fare, which drivers can then accept, decline, or counter-offer. This model is closer to what EasyRide offers. However, InDriver's negotiation happens through a static bid system rather than a real-time interactive process. The platform also lacks AI-powered assistance, comprehensive safety features like SOS alerts, and a detailed admin management panel.

Bykea is a Pakistan-based platform focused on motorcycle ride-hailing and parcel delivery. Bykea is popular for short-distance, affordable rides in congested urban areas. However, it is limited to motorcycle transport, does not support fare negotiation, has no AI chatbot, and provides minimal safety features beyond basic ride tracking.

In contrast, EasyRide combines real-time fare negotiation via WebSockets, an AI chatbot for instant support, GPS tracking, SOS emergency alerts, OTP-based ride verification, in-ride messaging, driver verification with document uploads, and a full admin panel, all integrated into a single platform.

Table 1.1: Comparative Analysis of Existing Systems

Application Name	Identified Weaknesses	Proposed Improvements
Uber	- Fixed surge pricing with no negotiation - Automated support with limited resolution	- Real-time fare negotiation via Socket.IO - AI chatbot (GPT-3.5) for instant support
Careem	- No fare negotiation capability - Limited in-ride communication features	- Dynamic negotiation before ride starts - In-ride chat and real-time notifications
InDriver	- Static bid system, not real-time - No AI chatbot or SOS features	- Live Socket.IO based offer/counter-offer - SOS alerts, OTP verification, AI assistant
Bykea	- Limited to motorcycle transport only - Minimal safety features and no negotiation	- Multi-vehicle support (car, bike, rickshaw) - Full safety suite with GPS, SOS, and OTP

1.4 Project Deliverables

The following deliverables have been produced as part of the EasyRide project:

1. **Software Requirements Specification (SRS)** – A detailed document outlining all functional and non-functional requirements of the system, including use case descriptions, user classes, and system constraints.
2. **Software Design Document (SDD)** – A comprehensive design blueprint covering system architecture, UML models (class diagrams, sequence diagrams, state transition diagrams), data flow diagrams, database schema, and algorithm descriptions.
3. **EasyRide Web Application** – The fully implemented web platform consisting of:
 - React 18 frontend with Tailwind CSS responsive design and GSAP animations
 - Node.js/Express.js 5.1 backend RESTful API
 - Real-time fare negotiation engine using Socket.IO 4.8
 - AI-powered chatbot service using OpenAI GPT-3.5 Turbo
 - MongoDB database with Mongoose ODM
 - Real-time GPS tracking with OpenRouteService and Nominatim

- In-ride chat and notification system
- 4. **AI Chatbot Module** – A GPT-3.5 Turbo powered chatbot service integrated into the platform that provides instant ride-related assistance, route suggestions, fare queries, and general platform support to passengers and drivers.
- 5. **Admin Panel** – A comprehensive management interface for verifying driver accounts, managing users, configuring fare settings, handling support tickets, monitoring active rides, managing withdrawals, and viewing platform analytics.
- 6. **Testing Documentation** – Unit test cases, integration test results, and system-level testing reports validating the functionality and reliability of the platform.
- 7. **Source Code Repository** – Complete project source code organized into Frontend, Server, and Admin directories with setup instructions and documentation.

1.5 System Limitations / Constraints

While EasyRide covers a broad range of features, it operates within certain limitations and constraints:

- **No Mobile Application** – The system is currently a web-only platform. There is no native mobile app for Android or iOS in this release, although the interface is responsive and mobile-friendly.
- **Manual Driver Verification** – Driver accounts are verified manually by the Admin through uploaded license and vehicle documents. There is no automated verification system integrated with any government transport authority database.
- **Internet Dependency** – All features require an active internet connection. There is no offline functionality available. GPS tracking and real-time negotiation are particularly dependent on stable connectivity.
- **Payment Simulation** – The current version does not integrate with a live payment gateway. The wallet and commission system operates on simulated balances for demonstration purposes.
- **English Only** – The platform currently operates in English. Urdu language support is not included in this version.
- **Limited AI Training Scope** – The AI chatbot uses OpenAI's general-purpose GPT-3.5 model with system prompts tailored for ride-hailing. It is not fine-tuned on a domain-specific dataset of Pakistani transport regulations.
- **Geolocation Accuracy** – The platform relies on OpenStreetMap-based services (Nominatim, OSRM, OpenRouteService) for geocoding and routing, which may have limited accuracy in remote or newly developed areas.

1.6 Tools and Technologies

The technology stack for EasyRide was selected based on the requirements for a modern, scalable, and real-time web application. Each tool was chosen for its suitability to the specific component it serves.

Table 1.2: Tools and Technologies Used in the Project

Tool / Technology	Version	Purpose
React	18	Frontend UI development as a Single Page Application
Vite	6.x	Build tool and development server for the React frontend
Tailwind CSS	3.x	Utility-first CSS framework for responsive UI styling
GSAP	3.x	Animation library for smooth UI transitions and effects
Node.js	20+	Backend server runtime environment
Express.js	5.1	REST API framework for backend route handling
Socket.IO	4.8	WebSocket library for real-time fare negotiation, chat, and notifications
OpenAI GPT-3.5 Turbo	API	Large language model powering the AI chatbot
MongoDB	7.x	NoSQL database for storing all application data
Mongoose	8.x	ODM library for MongoDB schema management in Node.js
JWT (jsonwebtoken)	9.x	Token-based user session authentication
bcrypt	5.x	Secure password hashing library
Cloudinary	API	Cloud-based image and file upload service for driver documents and profiles
Nodemailer	6.x	Email service for OTP delivery and notifications
OpenRouteService	API	Routing, distance, and duration calculations
Nominatim	API	Geocoding and reverse geocoding service
OSRM	API	Open-source routing for real-time navigation
Google Maps API	API	Map rendering and location visualization
Git / GitHub	Latest	Version control and source code repository
VS Code	Latest	Primary IDE for development

1.7 Relevance to Course Modules

This project draws upon knowledge gained from multiple courses throughout the BSCS degree program. The following subsections describe how specific academic subjects contributed to the development of EasyRide.

1.7.1 Programming Fundamentals

The foundational programming concepts learned in this course were applied throughout the project. Object-oriented principles guided the structuring of both the backend API and the React frontend components. Modular design patterns were used extensively to keep the codebase organized, with separate route handlers, controllers, services, and middleware layers. Concepts like loops, conditionals, and data manipulation were used in implementing fare calculation algorithms, input validation, and business logic for ride management.

1.7.2 Data Structures and Algorithms

Knowledge of data structures played a role in designing the database schema and managing data efficiently. Array and object manipulation were central to handling nearby driver searches, applying geospatial filters, and sorting by distance. The fare negotiation algorithm relies on maintaining offer/counter-offer state through Socket.IO events. Pagination logic uses skip and limit techniques to handle large ride history result sets without overloading the frontend. The geospatial driver matching system uses coordinate-based distance calculations to find the nearest available drivers.

1.7.3 Database Management Systems

Database concepts were heavily applied in designing the 12 MongoDB collections for users, drivers, rides, messages, notifications, OTPs, fare settings, SOS records, support tickets, withdrawals, admin accounts, and blacklisted tokens. Although MongoDB is a NoSQL database, the principles of normalization and referential integrity were considered when linking related collections through ObjectId references. Indexing strategies, including 2dsphere geospatial indexes on driver locations, were applied to optimize search performance. TTL indexes were used on OTP documents for automatic expiration.

1.7.4 Software Engineering

This project followed an Agile iterative development process, with features developed and tested in sprints. UML diagrams including use case diagrams, class diagrams, sequence diagrams, and state transition diagrams were created during the design phase. The SRS and SDD documents were prepared following standard documentation practices. Testing strategies including unit testing, integration testing, and user acceptance testing were planned and executed systematically.

1.7.5 Other Relevant Courses

Concepts from the Artificial Intelligence course were directly applied in integrating the OpenAI GPT-3.5 API for the AI chatbot module. The chatbot uses system prompts and conversation context to provide ride-related guidance and platform assistance. Web Development coursework provided the skills needed to build the React 18 frontend and Node.js/Express.js backend. Computer Networks knowledge was applied in configuring WebSocket connections via Socket.IO for real-time communication and understanding HTTP/HTTPS request handling. Human-Computer Interaction principles guided the UI/UX design with GSAP animations to ensure the platform is intuitive and visually engaging even for users with limited technical skills.

1.8 Chapter Summary

This chapter introduced the background and motivation behind the EasyRide project, describing how the lack of transparent, negotiable, and safe ride-hailing in Pakistan creates a real need for a modern solution. The vision for the platform was presented, highlighting its goal of combining real-time fare negotiation with AI-powered assistance and comprehensive safety features. A comparative analysis of existing systems demonstrated the limitations of current solutions and justified the development of EasyRide. The chapter also outlined the project deliverables, system constraints, selected tools and technologies, and described how academic coursework contributed to the project. The following chapter presents a detailed problem definition and the proposed solution.

Chapter 2

Problem Definition

2.1 Problem Statement

Millions of commuters in Pakistan face daily challenges in finding safe, affordable, and transparent transportation. The existing ride-hailing platforms in the market use fixed algorithmic pricing that does not accommodate the deeply rooted negotiation culture of Pakistani commuters. Passengers often feel overcharged during peak hours due to opaque surge pricing, while drivers sometimes feel underpaid on long-distance rides. There is no widely adopted digital platform that allows real-time fare negotiation between passengers and drivers before a ride begins. Additionally, existing platforms offer limited safety features, inadequate customer support through generic automated chatbots, and minimal transparency regarding driver verification. This gap leaves a significant portion of the commuting population frustrated and underserved.

2.2 Proposed Solution Overview

EasyRide addresses the identified problem by offering a web-based ride-hailing platform that combines real-time fare negotiation with AI-powered assistance and comprehensive safety features. The system provides a dynamic negotiation engine powered by Socket.IO that allows passengers and nearby drivers to exchange offers and counter-offers in real time before agreeing on a final fare. An AI chatbot powered by OpenAI's GPT-3.5 Turbo provides instant support for ride-related queries, route suggestions, and general platform assistance. The platform features real-time GPS tracking using Google Maps API and OpenRouteService, OTP-based ride verification to prevent unauthorized pickups, in-ride messaging, SOS emergency alerts, and a driver verification system with document uploads via Cloudinary. A comprehensive admin panel allows administrators to manage users, verify drivers, configure fare settings, handle support tickets, and monitor platform activity. The platform also supports a 5% commission system on completed rides and a driver withdrawal mechanism.

2.3 Objectives of the Proposed System

The objectives of EasyRide are derived directly from the identified problem and are designed to be specific, measurable, and achievable within the scope of this project.

Business Objectives (BO):

- BO-1: Develop a web-based ride-hailing platform that allows real-time fare negotiation between passengers and drivers using WebSocket technology.
- BO-2: Provide an AI-powered chatbot using OpenAI GPT-3.5 Turbo for instant ride-related assistance, route suggestions, and platform support.
- BO-3: Implement real-time GPS tracking and route visualization using Google Maps API and OpenRouteService to allow passengers and drivers to monitor ride progress.

- BO-4: Enable OTP-based ride verification to ensure that only the correct passenger is picked up by the assigned driver.
- BO-5: Implement a comprehensive safety system including SOS emergency alerts, in-ride messaging, and driver verification with license and vehicle document uploads.
- BO-6: Ensure secure access through role-based authentication (Passenger, Driver, Admin) with JWT-based session management and bcrypt password hashing.
- BO-7: Provide an administrative panel for managing users, verifying drivers, configuring dynamic fare settings, handling support tickets, and monitoring platform operations.

2.4 Scope of the System

EasyRide is scoped as a web-based application designed for use through modern web browsers on both desktop and mobile devices. The target users include passengers seeking rides, drivers looking to offer transportation services, and administrators responsible for platform management. The system covers the following functionalities: user registration and authentication with email verification, ride booking with pickup and destination selection, real-time fare negotiation via Socket.IO, AI chatbot assistance, GPS tracking and route display, OTP-based ride start verification, in-ride chat, SOS emergency alerts, driver verification with document management, ride history and rating system, a 5% commission deduction system, driver earnings and withdrawal management, notification delivery, support ticket system, and a full admin panel.

The system does not include native mobile applications, integration with live payment gateways, automated government database verification for driver licenses, multi-language support, or offline functionality. These features are considered outside the scope of the current release but may be addressed in future versions.

2.5 System Architecture and Modules

EasyRide follows a three-tier web architecture consisting of a React 18 frontend, a Node.js/Express.js 5.1 backend API, and a MongoDB database. The frontend communicates with the backend through RESTful API calls and persistent Socket.IO connections. The AI chatbot operates as an integrated service within the Node.js backend, making API calls to OpenAI's GPT-3.5 Turbo model. Real-time features including fare negotiation, ride status updates, chat, and notifications are handled through Socket.IO 4.8. Geolocation services use OpenRouteService, Nominatim, and OSRM for routing and geocoding, while Google Maps API handles map rendering. Cloudinary manages image uploads for driver documents and profile pictures. The architecture supports modularity and allows individual components to be upgraded independently. The complete architectural diagram is presented in Chapter 4 (Figure 4.2).

2.5.1 Module 1: User Management and Authentication

This module handles user registration, login, session management, and profile operations for all three user roles (Passenger, Driver, Admin). It implements JWT-based authentication with

bcrypt password hashing and token blacklisting for secure logout. Users can create accounts with email verification, update their profiles including profile picture uploads via Cloudinary, and manage their credentials. Driver registration includes additional fields for vehicle information, license number, and document uploads.

Functional Requirements:

- FR-1.1: The system shall allow users to register as either a Passenger or Driver with a valid email, password, name, and phone number.
- FR-1.2: The system shall authenticate users using email and password and issue a JWT token upon successful login.
- FR-1.3: The system shall allow users to view and update their profile information including profile picture.
- FR-1.4: The system shall enforce role-based access control for Passenger, Driver, and Admin roles.
- FR-1.5: The system shall support secure logout by blacklisting the current JWT token.

2.5.2 Module 2: Ride Booking and Fare Negotiation

This module manages the complete ride lifecycle from booking to completion. Passengers select pickup and destination locations on the map, receive a system-generated fare estimate based on distance and configured fare parameters, and the ride request is broadcast to nearby drivers via Socket.IO. Drivers can view the request details and either accept the estimated fare or propose a counter-offer. The passenger can then accept, reject, or counter the driver's offer. This real-time negotiation continues until both parties agree on a fare. Once agreed, the ride is confirmed and the driver proceeds to the pickup location.

Functional Requirements:

- FR-2.1: The system shall allow passengers to select pickup and destination locations using an interactive map.
- FR-2.2: The system shall calculate and display a fare estimate based on distance, base fare, and per-kilometer rate.
- FR-2.3: The system shall broadcast ride requests to nearby available drivers via Socket.IO.
- FR-2.4: The system shall support real-time fare negotiation with offer and counter-offer between passenger and driver.
- FR-2.5: The system shall confirm the ride once both parties agree on a final fare.

2.5.3 Module 3: Real-Time Tracking and Ride Execution

This module handles the active ride phase including GPS tracking, OTP verification, in-ride communication, and ride completion. Before the ride begins, the system generates a 4-digit OTP sent to the passenger which the driver must enter to start the ride. During the ride, both passenger and driver can see real-time location updates on the map. In-ride chat allows communication between the two parties. Upon reaching the destination, the driver completes the ride, a 5% commission is deducted, and both parties can rate each other.

Functional Requirements:

- FR-3.1: The system shall generate a 4-digit OTP and send it to the passenger for ride start verification.
- FR-3.2: The system shall display real-time GPS location of the driver on the passenger's map.
- FR-3.3: The system shall provide in-ride chat between passenger and driver.
- FR-3.4: The system shall deduct a 5% platform commission from the ride fare upon completion.
- FR-3.5: The system shall allow both passenger and driver to rate each other after ride completion.

2.5.4 Module 4: AI Chatbot and Safety Features

This module provides AI-powered assistance through an integrated chatbot and safety mechanisms. The chatbot uses OpenAI's GPT-3.5 Turbo API with ride-hailing focused system prompts to answer user queries about routes, fares, platform usage, and general assistance. The safety features include an SOS emergency alert system that allows passengers or drivers to send distress signals during a ride, which notifies the admin and logs the incident with location data. The module also includes the support ticket system for non-emergency complaints and feedback.

Functional Requirements:

- FR-4.1: The system shall provide a chatbot interface for users to submit ride-related and platform queries.
- FR-4.2: The system shall process questions through the OpenAI API and return contextual responses.
- FR-4.3: The system shall allow users to trigger SOS alerts during an active ride with automatic location capture.
- FR-4.4: The system shall allow users to create support tickets for complaints and feedback.

2.5.5 Module 5: Admin Panel and Platform Management

The Admin module provides comprehensive management tools for platform administrators. It includes driver verification workflow where admins review uploaded license and vehicle documents before approving or rejecting driver accounts. The module also covers user account management, dynamic fare settings configuration (base fare, per-km rate, minimum fare), support ticket resolution, active ride monitoring, withdrawal request processing, notification management, and platform analytics. Administrators can view dashboards summarizing total users, rides, revenue, and pending verifications.

Functional Requirements:

- FR-5.1: The system shall allow admins to review and approve or reject driver verification requests with uploaded documents.
- FR-5.2: The system shall allow admins to manage user and driver accounts including activation and deactivation.
- FR-5.3: The system shall allow admins to configure dynamic fare settings including base fare, per-kilometer rate, and minimum fare.
- FR-5.4: The system shall allow admins to view and resolve support tickets and SOS alerts.
- FR-5.5: The system shall provide an analytics dashboard showing total users, rides, revenue, and pending actions.
- FR-5.6: The system shall allow admins to process driver withdrawal requests from earned commissions.

2.6 Assumptions and Dependencies

The following assumptions and dependencies have been identified during the design and development of EasyRide:

- Users are expected to have a stable internet connection, as all features including GPS tracking and real-time negotiation require online access.
- The AI chatbot depends on the OpenAI GPT-3.5 Turbo API for generating responses. If this API is unavailable, the chatbot will display an appropriate fallback message.
- Driver verification relies on the admin manually reviewing submitted license and vehicle documents. There is no integration with any external transport authority database.
- The system depends on MongoDB Atlas or a local MongoDB instance for data persistence.
- Real-time fare negotiation and ride tracking depend on Socket.IO maintaining a stable WebSocket connection between client and server.
- Geocoding and routing depend on OpenRouteService, Nominatim, and OSRM APIs being available. Map rendering depends on the Google Maps JavaScript API.

- Email notifications and OTP delivery depend on configured SMTP services via Nodemailer.
- Image uploads for driver documents and profile pictures depend on Cloudinary's API availability.

2.7 Chapter Summary

This chapter defined the core problem that EasyRide addresses: the lack of transparent, negotiable, and safe ride-hailing services for commuters in Pakistan. The proposed solution was described as a web platform that integrates real-time fare negotiation with AI-powered assistance and comprehensive safety features. Clear and measurable objectives were established, and the system scope was defined to set the boundaries of what the platform will and will not do. The system architecture was presented with an architecture diagram, and each of the five major modules was described with its corresponding functional requirements. Assumptions and dependencies were also documented to clarify the conditions under which the system operates. The next chapter presents a detailed requirement analysis of the system.

Chapter 3

Requirement Analysis

3.1 Introduction to Requirement Analysis

This chapter provides a structured specification of the system requirements for EasyRide. The goal is to clearly define what the system needs to accomplish before moving into the design and implementation phases. Every requirement documented here acts as a foundation for development, validation, and testing.

The analysis covers the techniques used to gather requirements, identification of user classes, use case modeling, functional and non-functional requirements, and external interface specifications. All requirements are written in a measurable and testable format following standard requirement engineering practices.

3.2 Requirement Elicitation Techniques

Requirements for EasyRide were gathered using a combination of methods to ensure nothing important was missed:

- Stakeholder discussions with daily commuters, rickshaw and taxi drivers, and university students in Islamabad and surrounding areas to understand their transportation pain points, fare negotiation habits, and safety concerns.
- Literature review of existing ride-hailing platforms (Uber, Careem, InDriver, Bykea) operating in Pakistan to identify common features, pricing models, and shortcomings.
- Analysis of local transportation culture, particularly the deeply rooted practice of fare negotiation between passengers and drivers that current platforms do not accommodate.
- Technical feasibility assessment of available technologies (Socket.IO, OpenAI API, Google Maps API, OpenRouteService, MongoDB) to confirm that the proposed real-time features are practically implementable.
- Observation of how people currently find and negotiate rides, which revealed the heavy reliance on roadside hailing, verbal fare bargaining, and complete absence of safety mechanisms.

3.3 User Classes and Characteristics

EasyRide serves three primary user classes, each with distinct responsibilities and access levels. This classification supports the role-based access control implemented throughout the system.

Table 3.1: User Classes and Characteristics

User Class	Description	Technical Level
Passenger	Individuals who book rides by selecting pickup and destination locations, negotiate fares with drivers in real time, track rides on the map, use the AI chatbot for assistance, trigger SOS alerts in emergencies, rate drivers after ride completion, and create support tickets.	Basic
Driver	Transport providers who register with vehicle and license details, receive ride requests from nearby passengers, accept or negotiate fares, verify ride start via OTP, update their real-time location, communicate with passengers during rides, view earnings, and request withdrawals.	Basic to Intermediate
Administrator	Platform managers who verify driver accounts by reviewing uploaded documents, manage user and driver accounts, configure dynamic fare settings, handle support tickets and SOS alerts, process withdrawal requests, and monitor platform analytics.	Advanced

3.4 System Overview

EasyRide operates within a three-tier web architecture where the React 18 frontend communicates with the Node.js/Express.js 5.1 backend through REST APIs and persistent Socket.IO connections. The system supports role-based interactions where passengers, drivers, and admins each have dedicated dashboards and access permissions. Internal modules communicate through RESTful endpoints, while real-time features like fare negotiation, ride tracking, chat, and notifications are handled through Socket.IO 4.8 WebSocket connections. The AI chatbot is integrated directly into the Node.js backend, making API calls to OpenAI’s GPT-3.5 Turbo model.

The system boundary includes all internal modules (user management, ride booking, fare negotiation, GPS tracking, OTP verification, in-ride chat, AI chatbot, SOS alerts, rating system, support tickets, admin panel, withdrawal management, notifications) as well as external integrations with OpenAI API, Google Maps API, OpenRouteService, Nominatim, OSRM, Cloudinary, and Nodemailer SMTP services.

3.5 Use Case Model

The use case model captures the main interactions between the system actors and the platform functionalities. It defines the system boundaries and describes functional behavior from the perspective of each user type.

3.5.1 Use Case Diagram

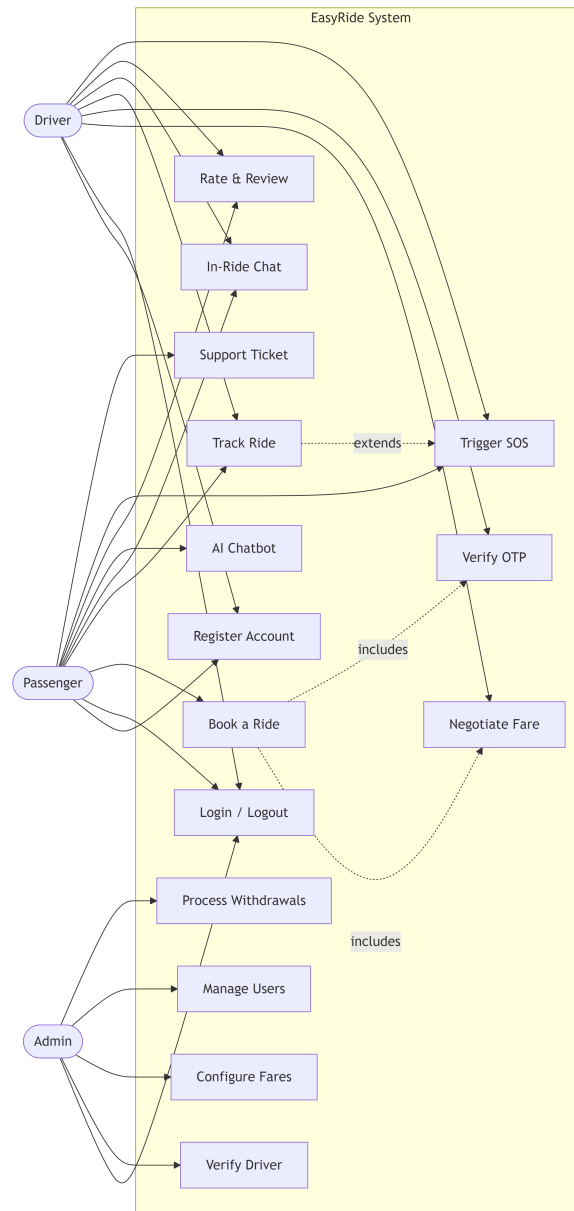


Figure 3.1: Use Case Diagram of EasyRide System

The diagram above shows the interactions between Passengers, Drivers, Admins, and the System with the major functionalities of EasyRide. Key use cases include ride booking, real-time fare negotiation, GPS tracking, OTP verification, AI chatbot assistance, SOS alerts, driver verification, and rating submission.

3.5.2 Use Case Descriptions

3.5.2.1 Use Case 1: Book a Ride with Fare Negotiation

Table 3.2: UC-001: Book a Ride with Fare Negotiation

Use Case ID	UC-001
Use Case Name	Book a Ride with Fare Negotiation
Actors	Passenger, Driver, System
Description	The passenger selects pickup and destination locations on the map, receives a system-calculated fare estimate, and confirms the ride request. The request is broadcast to nearby drivers who can accept the fare or propose a counter-offer. Both parties negotiate in real time until a fare is agreed upon.
Trigger	Passenger clicks “Book Ride” after setting locations.
Preconditions	Passenger is logged in. At least one driver is online and available nearby. Internet connection is active.
Postconditions	Ride record is created with agreed fare. Both parties see confirmation.
Normal Flow	1. Passenger opens the ride booking page. 2. Passenger selects pickup and destination on the interactive map. 3. System calculates fare estimate based on distance, base fare, and per-km rate. 4. Passenger reviews the estimate and confirms the ride request. 5. System saves ride with status “requested” and broadcasts to nearby drivers via Socket.IO. 6. Driver receives the ride notification and reviews details. 7. Driver accepts the estimated fare or proposes a counter-offer. 8. If counter-offer is sent, passenger can accept, reject, or counter. 9. Once both parties agree, ride status changes to “accepted”.
Alternative Flow	If no nearby drivers are available, the system displays a message asking the passenger to try again shortly. If all drivers decline, the system re-broadcasts the request or notifies the passenger.

3.5.2.2 Use Case 2: AI Chatbot Assistance

Table 3.3: UC-002: AI Chatbot Assistance

Use Case ID	UC-002
Use Case Name	AI Chatbot Assistance
Actors	User (Passenger or Driver), System
Description	The user types a question related to rides, fares, routes, or platform usage. The system processes it through the OpenAI GPT-3.5 Turbo API and returns a helpful response.
Trigger	User types a message and clicks Send in the chatbot interface.
Preconditions	User is on the chatbot page. OpenAI API is accessible. Internet connection is active.
Postconditions	User receives an AI-generated response to their query.
Normal Flow	1. User opens the chatbot interface. 2. User types a ride-related question and clicks Send. 3. System forwards the message to the Express.js backend. 4. Backend constructs a system prompt with ride-hailing context. 5. Backend sends the prompt and user message to OpenAI GPT-3.5 API. 6. API returns a generated response. 7. System displays the response in the chat interface.
Alternative Flow	If the OpenAI API is unavailable, the system displays a message indicating the chatbot is temporarily down and suggests contacting support.

3.5.2.3 Use Case 3: Driver Verification

Table 3.4: UC-003: Driver Verification

Use Case ID	UC-003
Use Case Name	Driver Verification
Actors	Driver, Admin, System
Description	The driver uploads license and vehicle documents during registration. The admin reviews these documents through the admin panel and approves or rejects the verification request.
Trigger	Driver submits verification documents after registration.
Preconditions	Driver has registered an account. Documents are uploaded via Cloudinary.
Postconditions	Driver account is marked as verified (approved) or rejected with reason.
Normal Flow	1. Driver registers and uploads license photo, vehicle registration, and vehicle photo via Cloudinary. 2. System saves documents and sets verification status to “pending”. 3. Admin opens the verification requests list in the admin panel. 4. Admin reviews the uploaded documents. 5. Admin approves the driver. 6. System sets isVerified to true and driver can now receive ride requests. 7. Driver receives a notification of approval.
Alternative Flow	If documents are insufficient or unclear, admin rejects the request. Driver receives a notification with rejection reason and can re-upload corrected documents.

3.5.2.4 Use Case 4: Register Account

Table 3.5: UC-004: Register Account

Use Case ID	UC-004
Use Case Name	Register Account
Actors	Passenger, Driver, System
Description	A new user creates an account on EasyRide by providing personal details. Passengers register with basic information while drivers must additionally provide vehicle details and upload license/vehicle documents.
Trigger	User clicks the “Register” button on the registration page.
Preconditions	User has a valid email address. Internet connection is active.
Postconditions	A new user or driver record is created in the database. Driver accounts are set to unverified status pending admin review.
Normal Flow	1. User navigates to the registration page and selects role (Passenger or Driver). 2. User enters full name, email, phone number, and password. 3. If Driver, user additionally provides vehicle name, plate number, vehicle type, license number, and uploads license photo and vehicle photo via Cloudinary. 4. System validates input fields and checks for duplicate email. 5. System hashes the password using bcrypt and creates the user/driver document in MongoDB. 6. System generates a JWT token and redirects the user to their dashboard.
Alternative Flow	If the email already exists, the system displays an error message. If required fields are missing or invalid, the system highlights validation errors on the form.

3.5.2.5 Use Case 5: Login / Logout

Table 3.6: UC-005: Login / Logout

Use Case ID	UC-005
Use Case Name	Login / Logout
Actors	Passenger, Driver, Admin, System
Description	An existing user logs into the system using email and password credentials. The system authenticates the user, issues a JWT token, and redirects to the role-specific dashboard. On logout, the token is blacklisted to prevent reuse.
Trigger	User submits the login form or clicks the logout button.
Preconditions	User has an existing account. For login: valid credentials. For logout: user is currently authenticated.
Postconditions	On login: JWT token is issued and session is established. On logout: token is blacklisted and user is redirected to the landing page.
Normal Flow	1. User enters email and password on the login page. 2. System finds the user by email in the appropriate collection (User, Driver, or Admin). 3. System compares the submitted password with the stored bcrypt hash. 4. If valid, system generates a JWT token containing userId and role. 5. User is redirected to their role-specific dashboard. 6. For logout: system adds the current JWT token to the BlacklistToken collection.
Alternative Flow	If credentials are invalid, the system returns a 401 error with “Invalid credentials” message. The user can retry or navigate to the registration page.

3.5.2.6 Use Case 6: Verify OTP

Table 3.7: UC-006: Verify OTP

Use Case ID	UC-006
Use Case Name	Verify OTP for Ride Start
Actors	Passenger, Driver, System
Description	When the driver arrives at the pickup location, the system generates a 4-digit OTP sent to the passenger via email. The driver enters the OTP received verbally from the passenger to verify the correct pickup and start the ride.
Trigger	Driver arrives at pickup location and the system generates an OTP.
Preconditions	Ride is in “accepted” status. Driver has arrived at the pickup point.
Postconditions	Ride status changes to “in-progress” and real-time GPS tracking begins.
Normal Flow	1. System generates a random 4-digit OTP and stores it with a TTL expiry. 2. OTP is sent to the passenger’s registered email via Nodemailer. 3. Passenger shares the OTP verbally with the driver. 4. Driver enters the OTP in the ride interface. 5. System verifies the OTP against the stored value. 6. If correct, ride status transitions to “in-progress” and tracking starts.
Alternative Flow	If the OTP is incorrect, the system shows an error and allows the driver to retry. If the OTP expires, a new OTP can be generated.

3.5.2.7 Use Case 7: Track Ride

Table 3.8: UC-007: Track Ride

Use Case ID	UC-007
Use Case Name	Track Ride in Real Time
Actors	Passenger, Driver, System
Description	During an active ride, the driver’s real-time GPS location is transmitted to the server and displayed on the passenger’s map. The route is visualized using polylines, and both parties can monitor the ride progress.
Trigger	Ride status changes to “in-progress” after OTP verification.
Preconditions	Ride has been started. GPS is enabled on the driver’s device. Socket.IO connection is active.
Postconditions	Both passenger and driver can see the real-time ride progress on the map until ride completion.
Normal Flow	1. Driver’s device sends GPS coordinates to the server every 5 seconds via Socket.IO. 2. Server relays the updated location to the passenger’s frontend. 3. Passenger’s map displays the driver’s moving marker in real time. 4. Route polyline is rendered using Google Maps API and OSRM routing data. 5. When the driver reaches the destination, the ride can be marked as completed.
Alternative Flow	If the Socket.IO connection drops, the system attempts automatic reconnection. Stale location data is handled gracefully.

3.5.2.8 Use Case 8: In-Ride Chat

Table 3.9: UC-008: In-Ride Chat

Use Case ID	UC-008
Use Case Name	In-Ride Chat
Actors	Passenger, Driver, System
Description	During an active ride, the passenger and driver can exchange text messages in real time through a chat interface powered by Socket.IO. Messages are stored in the database for ride reference.
Trigger	Either the passenger or driver sends a text message during an active ride.
Preconditions	A ride is active between the passenger and driver. Socket.IO connection is established.
Postconditions	Messages are delivered in real time and stored in the Message collection.
Normal Flow	1. User opens the in-ride chat interface. 2. User types a message and clicks Send. 3. System emits the message via Socket.IO to the other party. 4. Message is saved to the Message collection with sender type, timestamp, and ride reference. 5. Recipient sees the message appear instantly in the chat window.
Alternative Flow	If the connection is temporarily lost, messages are queued and delivered upon reconnection.

3.5.2.9 Use Case 9: Rate and Review

Table 3.10: UC-009: Rate and Review

Use Case ID	UC-009
Use Case Name	Rate and Review
Actors	Passenger, Driver, System
Description	After a ride is completed, both the passenger and driver can submit a rating (1–5 stars) for each other. The system updates the average rating on the recipient’s profile.
Trigger	Ride status changes to “completed.”
Preconditions	Ride has been successfully completed. Both parties are authenticated.
Postconditions	Rating is recorded on the ride and the recipient’s average rating is updated.
Normal Flow	1. After ride completion, the system prompts both parties with a rating screen. 2. User selects a star rating (1–5). 3. System saves the rating on the ride document and updates the recipient’s average rating. 4. Ride status transitions to “rated.”
Alternative Flow	If a user skips rating, the system allows it but the ride remains in “completed” status instead of “rated.”

3.5.2.10 Use Case 10: Trigger SOS

Table 3.11: UC-010: Trigger SOS

Use Case ID	UC-010
Use Case Name	Trigger SOS Emergency Alert
Actors	Passenger, Driver, Admin, System
Description	During an active ride, either the passenger or the driver can trigger an SOS emergency alert. The system captures the current GPS location, creates an SOS record, and immediately notifies the admin.
Trigger	User presses the SOS button during an active ride.
Preconditions	A ride is currently in progress. GPS is available.
Postconditions	SOS record is saved with GPS location. Admin receives an immediate notification.
Normal Flow	1. User taps the SOS emergency button during a ride. 2. System captures the current GPS coordinates of the device. 3. System creates an SOS record in the database with ride ID, user ID, location, and timestamp. 4. Admin is notified immediately through the notification system. 5. Admin can review the SOS alert and take appropriate action.
Alternative Flow	If GPS is unavailable, the system logs the SOS with the last known location.

3.5.2.11 Use Case 11: Support Ticket

Table 3.12: UC-011: Support Ticket

Use Case ID	UC-011
Use Case Name	Submit Support Ticket
Actors	Passenger, Driver, Admin, System
Description	Users can submit a support ticket for non-emergency complaints, feedback, or issues. The admin reviews and responds to tickets through the admin panel.
Trigger	User fills and submits the support ticket form.
Preconditions	User is authenticated.
Postconditions	Support ticket is created with “pending” status and admin is notified.
Normal Flow	1. User navigates to the support page. 2. User enters subject, category, and description of the issue. 3. System saves the ticket with “pending” status and the user’s ID. 4. Admin receives a notification of the new ticket. 5. Admin reviews and responds to the ticket, updating its status.
Alternative Flow	If required fields are missing, the system shows validation errors and does not submit.

3.5.2.12 Use Case 12: Configure Fares

Table 3.13: UC-012: Configure Fares

Use Case ID	UC-012
Use Case Name	Configure Fare Settings
Actors	Admin, System
Description	The admin configures dynamic fare parameters (base fare, per-km rate, minimum fare) for each vehicle type. These settings are used by the system to calculate fare estimates for all ride bookings.
Trigger	Admin opens the fare configuration page in the admin panel.
Preconditions	Admin is authenticated with admin privileges.
Postconditions	Fare settings are updated in the FareSetting collection and applied to all subsequent fare calculations.
Normal Flow	1. Admin navigates to the fare settings page. 2. Admin views current fare settings per vehicle type. 3. Admin updates base fare, per-km rate, or minimum fare values. 4. System validates and saves the updated values in the FareSetting collection. 5. Subsequent fare estimates use the new configuration.
Alternative Flow	If invalid values are entered (negative numbers, empty fields), the system rejects the update with an error message.

3.5.2.13 Use Case 13: Manage Users

Table 3.14: UC-013: Manage Users

Use Case ID	UC-013
Use Case Name	Manage Users and Drivers
Actors	Admin, System
Description	The admin can view, activate, or deactivate passenger and driver accounts from the admin panel. This allows platform governance and moderation.
Trigger	Admin navigates to the user management section.
Preconditions	Admin is authenticated with admin privileges.
Postconditions	User or driver account status is updated accordingly.
Normal Flow	1. Admin opens the user management page. 2. System displays a list of all registered passengers and drivers with key details. 3. Admin can search, filter, or sort the list. 4. Admin selects a user and activates or deactivates the account. 5. System updates the account status in the database.
Alternative Flow	If the admin tries to deactivate an account with an active ride, the system warns the admin before proceeding.

3.5.2.14 Use Case 14: Process Withdrawals

Table 3.15: UC-014: Process Withdrawals

Use Case ID	UC-014
Use Case Name	Process Driver Withdrawal Requests
Actors	Driver, Admin, System
Description	Drivers can request withdrawal of their accumulated earnings. The admin reviews and processes pending withdrawal requests, approving or rejecting them.
Trigger	Driver submits a withdrawal request, or admin opens the withdrawal processing page.
Preconditions	Driver has a positive balance. Admin is authenticated.
Postconditions	Withdrawal is processed: balance is deducted on approval, or request is rejected with reason.
Normal Flow	1. Driver navigates to the earnings/withdrawal page and enters the withdrawal amount and method. 2. System validates the amount against available balance. 3. System creates a withdrawal record with “pending” status and notifies the admin. 4. Admin reviews the pending withdrawal request. 5. Admin approves the withdrawal. 6. System deducts the amount from the driver’s balance and updates the withdrawal status to “approved.” 7. Driver receives a notification of the processed withdrawal.
Alternative Flow	If the withdrawal amount exceeds the available balance, the system rejects the request. If the admin rejects the withdrawal, the driver is notified with the reason and the balance remains unchanged.

3.5.3 Event-Response Table

Table 3.16: Event-Response Table

Event	Source	System Response
User Login Attempt	Passenger/Driver	Validate credentials against database, issue JWT token if valid, deny access if invalid.
Ride Booking Request	Passenger	Calculate fare estimate, save ride with “requested” status, broadcast to nearby drivers via Socket.IO.
Driver Counter-Offer	Driver	Forward counter-offer to passenger via Socket.IO, await passenger response (accept, reject, or counter).
OTP Verification	Driver	Compare entered OTP with stored value, start ride if correct, show error if incorrect.
SOS Alert Triggered	Passenger/Driver	Capture current GPS location, save SOS record, notify admin immediately, log incident.
Ride Completion	Driver	Mark ride as completed, deduct 5% commission, add remaining to driver balance, prompt both parties for ratings.
Driver Verification Request	Driver	Store uploaded documents via Cloudinary with “pending” status, notify admin for review.
Admin Approves Driver	Admin	Update driver isVerified to true, enable driver to receive ride requests, send approval notification.
Withdrawal Request	Driver	Validate balance, create withdrawal record with “pending” status, notify admin for processing.

3.6 Functional Requirements

Functional requirements define the specific services and behaviors that EasyRide must provide. These requirements are derived from the use case model and are categorized by system modules. Each requirement is uniquely identified and written in a testable format.

3.6.1 Module 1: User Management and Authentication

3.6.1.1 FR-1.1 User Registration and Authentication

Table 3.17: Functional Requirement – User Registration and Authentication

Identifier	FR-1.1
Title	User Registration and Authentication
Requirement	The system shall allow users to create an account as either a Passenger or Driver by providing full name, valid email, password, and phone number. Driver registration shall additionally require vehicle name, vehicle plate, vehicle type, and license number. The system shall authenticate users using email and password and maintain sessions using JWT tokens. Passwords shall be hashed using bcrypt before storage. Logout shall blacklist the current JWT token.
Source	Stakeholder Requirements
Rationale	Enables secure, role-based access to the platform with distinct registration flows for each role.
Dependencies	MongoDB connectivity, Cloudinary for driver document uploads
Priority	High

3.6.2 Module 2: Ride Booking and Fare Negotiation

3.6.2.1 FR-2.1 Ride Booking with Real-Time Fare Negotiation

Table 3.18: Functional Requirement – Ride Booking and Fare Negotiation

Identifier	FR-2.1
Title	Ride Booking with Real-Time Fare Negotiation
Requirement	The system shall allow passengers to select pickup and destination locations on an interactive map, calculate a fare estimate based on distance and configured fare parameters (base fare, per-km rate, minimum fare), and broadcast ride requests to nearby available drivers via Socket.IO. Drivers shall be able to accept the estimated fare or propose counter-offers. The system shall facilitate real-time negotiation until both parties agree on a final fare or the request is cancelled.
Source	Core System Feature
Rationale	Addresses the fundamental need for transparent, negotiable pricing in Pakistani ride-hailing.
Dependencies	Socket.IO 4.8, Google Maps API, OpenRouteService, FareSetting collection
Priority	High

3.6.3 Module 3: Ride Execution and Tracking

3.6.3.1 FR-3.1 Real-Time Tracking, OTP Verification, and Ride Completion

Table 3.19: Functional Requirement – Ride Execution and Tracking

Identifier	FR-3.1
Title	Real-Time GPS Tracking, OTP Verification, Ride Completion, and Ratings
Requirement	The system shall display the driver’s real-time GPS location on the passenger’s map during the ride via Socket.IO with updates every 5 seconds. A 4-digit OTP shall be generated and sent to the passenger’s email via Nodemailer when the driver arrives; the driver must enter this OTP to start the ride. The route shall be visualized using Google Maps API polylines and OSRM routing data. Upon ride completion, the system shall automatically deduct a 5% platform commission from the fare and credit the remaining amount to the driver’s balance. Both passenger and driver shall be prompted to submit mutual ratings (1–5 stars) after ride completion.
Source	System Requirement Specification
Rationale	Ensures ride transparency through live tracking, prevents unauthorized pickups via OTP, and enables quality feedback through the rating system.
Dependencies	Socket.IO, Google Maps API, OSRM, Nodemailer, OTP generation module
Priority	High

3.6.4 Module 4: AI Chatbot

3.6.4.1 FR-4.1 AI Chatbot Assistance

Table 3.20: Functional Requirement – AI Chatbot Assistance

Identifier	FR-4.1
Title	AI-Powered Chatbot for User Assistance
Requirement	The system shall provide an AI chatbot accessible to passengers and drivers that can answer ride-related questions, provide fare guidance, route suggestions, and platform usage help. The chatbot shall process natural language queries by constructing a system prompt with ride-hailing context and forwarding it to the OpenAI GPT-3.5 Turbo API. Responses shall be displayed in a conversational chat interface with distinct message bubbles for user and AI messages.
Source	Core System Feature
Rationale	Provides instant, intelligent assistance to users without requiring human support agents for common queries.
Dependencies	OpenAI GPT-3.5 Turbo API, Internet connectivity
Priority	Medium

3.6.5 Module 5: Safety and Communication Features

3.6.5.1 FR-5.1 SOS Alerts, In-Ride Chat, Support Tickets, and Notifications

Table 3.21: Functional Requirement – Safety and Communication Features

Identifier	FR-5.1
Title	SOS Emergency Alerts, In-Ride Chat, Support Tickets, and Notifications
Requirement	The system shall allow passengers and drivers to trigger an SOS emergency alert during active rides, capturing GPS coordinates and notifying the admin immediately. The system shall provide real-time text chat between passenger and driver during rides via Socket.IO, with messages stored in the database. Users shall be able to submit support tickets with subject, category, and description for non-emergency issues. The system shall deliver real-time notifications for ride updates, SOS alerts, verification results, and withdrawal status changes via Socket.IO.
Source	Safety and Communication Requirements
Rationale	Ensures user safety during rides, enables communication between ride parties, and provides a structured mechanism for issue resolution.
Dependencies	Socket.IO, GPS/Geolocation API, Notification module, MongoDB
Priority	High

3.6.6 Module 6: Admin Panel

3.6.6.1 FR-6.1 Admin Management Operations

Table 3.22: Functional Requirement – Admin Panel Operations

Identifier	FR-6.1
Title	Admin Panel for Platform Management
Requirement	The system shall provide a dedicated admin panel with the following capabilities: (1) Driver verification workflow to review uploaded license and vehicle documents and approve or reject applications with reasons. (2) User and driver account management including viewing, activating, and deactivating accounts. (3) Dynamic fare settings configuration for base fare, per-km rate, and minimum fare per vehicle type. (4) Support ticket management with the ability to view and respond to user-submitted tickets. (5) SOS alert monitoring and resolution tracking. (6) Driver withdrawal request processing with approve/reject functionality. (7) Platform analytics dashboard displaying total users, drivers, rides, revenue, and pending actions.
Source	Administrative Requirements
Rationale	Enables centralized platform governance, driver quality control, financial management, and operational monitoring.
Dependencies	MongoDB, Cloudinary (for document viewing), Notification module
Priority	High

3.7 Non-Functional Requirements

Non-functional requirements define the quality attributes that describe how the system should perform. These requirements are measurable and support the overall reliability, security, usability, and performance of EasyRide.

3.7.1 Performance Requirements

- PER-1: The system shall load user dashboards and ride pages within 3 seconds under normal network conditions.
- PER-2: The AI chatbot shall return responses within 2–4 seconds for typical queries.
- PER-3: Fare estimates shall be calculated and displayed within 2 seconds of selecting locations.
- PER-4: Real-time fare negotiation messages shall be delivered with less than 500ms latency via Socket.IO.
- PER-5: The system shall support at least 100 concurrent users including active rides without major performance degradation.

- PER-6: GPS location updates shall be transmitted at least every 5 seconds during active rides.

3.7.2 Security Requirements

- SEC-1: User authentication shall use JWT tokens transmitted only over HTTPS.
- SEC-2: All passwords shall be stored as hashed values using bcrypt and never in plain text.
- SEC-3: Role-based access control shall be enforced for Passenger, Driver, and Admin roles.
- SEC-4: JWT tokens shall be blacklisted upon logout to prevent reuse.
- SEC-5: All client-server communication shall use HTTPS protocol.
- SEC-6: User sessions shall automatically expire after the configured JWT expiry period.
- SEC-7: Chat messages and ride details shall only be visible to the involved passenger and driver.
- SEC-8: Driver verification documents shall be accessible only to the admin and the document owner.

3.7.3 Reliability Requirements

- REL-1: The system shall maintain at least 99% uptime during normal operation, excluding scheduled maintenance.
- REL-2: All database operations shall maintain data integrity and consistency through MongoDB schema validation.
- REL-3: The system shall log all critical errors and exceptions for analysis.
- REL-4: User-friendly error messages shall be displayed instead of technical stack traces.
- REL-5: Socket.IO connections shall automatically attempt reconnection if disconnected during an active ride.

3.7.4 Usability Requirements

- USE-1: The interface shall be clean, intuitive, and easy to navigate for users with basic smartphone or computer knowledge.
- USE-2: A new user shall be able to book a ride within 5 minutes of first use without formal training.
- USE-3: The system shall provide clear success, warning, and error messages for all user actions with GSAP animated transitions.
- USE-4: The application shall be fully responsive across desktop, laptop, tablet, and mobile devices.
- USE-5: Navigation shall remain consistent across all modules and user roles.

3.7.5 Scalability Requirements

- SCA-1: The system shall support an increasing number of users and concurrent rides without requiring major redesign.
- SCA-2: Backend components (API server, Socket.IO server, database) shall be independently scalable.
- SCA-3: The database design shall support large datasets through proper indexing, including 2dsphere geospatial indexes on driver locations.

3.7.6 Maintainability Requirements

- MAIN-1: The system shall be structured into separate modules (routes, controllers, services, middleware, models) to simplify maintenance.
- MAIN-2: Source code shall follow standard naming conventions and include sufficient comments.
- MAIN-3: Fare settings (base fare, per-km rate, minimum fare) shall be configurable through the admin panel without code changes.
- MAIN-4: Future updates shall be integrable without major restructuring of existing modules.

3.8 External Interface Requirements

This section defines how EasyRide interacts with external entities including users, hardware, software systems, and communication networks.

3.8.1 User Interface Requirements

- UI-1: The system shall provide a fully web-based graphical user interface accessible through Chrome, Firefox, Edge, and Safari.
- UI-2: The interface shall include dedicated pages for Home, Login/Register, Passenger Dashboard, Driver Dashboard, Ride Booking with Map, AI Chatbot, Ride Tracking, In-Ride Chat, Ride History, Admin Panel, and Settings.
- UI-3: The layout shall be responsive using Tailwind CSS for compatibility across all screen sizes.
- UI-4: Real-time updates shall be provided for ride status, fare negotiation, driver location, chat messages, and notifications via Socket.IO.
- UI-5: Clear visual indicators shall be shown for ride confirmation, OTP entry, fare negotiation status, form validation, and error messages.

3.8.2 Hardware Interface Requirements

- HI-1: The server shall require at least 2 CPU cores, 4GB RAM, and 10GB storage for stable operation.
- HI-2: Client devices shall support modern web browsers with GPS/geolocation capability.
- HI-3: GPS access is required on driver devices for real-time location tracking.

3.8.3 Software Interface Requirements

- SI-1: The system shall interact with its MongoDB database through the Mongoose ODM library.
- SI-2: The AI chatbot shall process queries through the OpenAI GPT-3.5 Turbo API endpoint.
- SI-3: Geocoding and routing shall use OpenRouteService, Nominatim, and OSRM APIs.
- SI-4: Map rendering shall use the Google Maps JavaScript API.
- SI-5: Image uploads shall use the Cloudinary API for driver documents and profile pictures.
- SI-6: Email notifications and OTP delivery shall use Nodemailer with configured SMTP services.
- SI-7: The Admin panel shall access management functions through REST-based internal endpoints restricted to the Admin role.

3.8.4 Communication Interface Requirements

- CI-1: All data transmission between client and server shall use HTTPS protocol.
- CI-2: RESTful APIs shall be used for all standard backend communication.
- CI-3: WebSockets (Socket.IO 4.8) shall handle real-time fare negotiation, ride status updates, chat messaging, and notifications.
- CI-4: External API calls to OpenAI, Google Maps, OpenRouteService, Nominatim, OSRM, and Cloudinary shall use HTTPS.
- CI-5: SMTP protocol shall be used for email notification and OTP delivery via Nodemailer.

3.9 Assumptions and Dependencies

- The system assumes stable internet connectivity for all online features, particularly real-time negotiation and GPS tracking.
- Third-party services (OpenAI API, Google Maps API, OpenRouteService, Nominatim, Cloudinary) are assumed to remain available and functional.

- Users are assumed to have basic digital literacy and access to a modern web browser with GPS capability.
- The AI chatbot is limited to providing general ride-related assistance and does not replace human customer support for complex issues.
- Driver verification relies on manual admin review and does not integrate with any government transport authority database.
- MongoDB is assumed to be running and accessible for data persistence.
- Reliable internet is required on driver devices for continuous location updates during rides.

3.10 Requirement Traceability Matrix

This matrix maps project objectives to use cases and functional requirements to ensure alignment and completeness.

Table 3.23: Requirement Traceability Matrix

Objective	Use Case(s)	Functional Requirement
BO-1 (Fare Negotiation)	UC-001, UC-012	FR-2.1 (Ride Booking with Real-Time Negotiation), FR-6.1 (Fare Configuration)
BO-2 (AI Chatbot)	UC-002	FR-4.1 (AI Chatbot Assistance via OpenAI)
BO-3 (GPS Tracking)	UC-007	FR-3.1 (Real-Time GPS Tracking)
BO-4 (OTP Verification)	UC-006	FR-3.1 (OTP-Based Ride Start)
BO-5 (Safety System)	UC-010, UC-011	FR-5.1 (SOS Alerts, Support Tickets)
BO-6 (Secure Access)	UC-004, UC-005	FR-1.1 (Registration, Authentication, and RBAC)
BO-7 (Admin Panel)	UC-003, UC-012, UC-013, UC-014	FR-6.1 (Driver Verification, Fare Config, User Management, Withdrawals)
BO-8 (Communication)	UC-008, UC-009	FR-5.1 (In-Ride Chat, Notifications), FR-3.1 (Ratings)

3.11 Chapter Summary

This chapter presented a complete requirement analysis of the EasyRide platform. It described the techniques used to gather requirements, defined the three user classes (Passenger, Driver, Administrator) and their characteristics, and documented the use case model with detailed descriptions for all 14 use cases covering account registration, login/logout, ride booking with fare

negotiation, OTP verification, ride tracking, in-ride chat, AI chatbot assistance, rating and review, SOS emergency alerts, support tickets, driver verification, fare configuration, user management, and withdrawal processing. Functional requirements were categorized into six system modules: User Management and Authentication (FR-1.1), Ride Booking and Fare Negotiation (FR-2.1), Ride Execution and Tracking (FR-3.1), AI Chatbot (FR-4.1), Safety and Communication Features (FR-5.1), and Admin Panel (FR-6.1). Non-functional requirements covering performance, security, reliability, usability, scalability, and maintainability were specified. External interface requirements for user, hardware, software, and communication interfaces were also detailed. A traceability matrix was included to ensure all project objectives are covered by specific use cases and functional requirements. These requirements form the formal basis for the system design presented in the next chapter.

Chapter 4

Software Design

4.1 Introduction to Software Design

This chapter presents the detailed software design of the EasyRide platform. It translates the requirements identified in Chapter 3 into structured architectural, behavioral, and data models. The purpose of this chapter is to provide a clear blueprint for system implementation, ensuring maintainability, scalability, and modular development.

This chapter includes the architectural design, UML models (sequence diagrams, class diagrams, state transition diagrams), data flow diagrams, and comprehensive database design for the EasyRide ride-hailing web application.

4.1.1 Purpose of this Document

The main purpose of this chapter is to provide a comprehensive architectural overview of the EasyRide platform. It details the major components and their specific responsibilities, ensuring a clear understanding of the system's structure. This chapter also presents essential design models, including the architecture diagram, class diagrams, sequence diagrams, state diagrams, and process flow descriptions. Furthermore, it describes the data design and key algorithms, demonstrating how the proposed design satisfies the functional requirements. This chapter serves as a guide for developers, testers, and future maintainers on how to systematically implement and maintain the system.

4.1.2 Intended Audience

This chapter is intended for a diverse group of stakeholders involved in the project's lifecycle. It is primarily designed for development team members who will implement the system and testers responsible for designing and executing test cases. Additionally, it serves as a reference for the project supervisor and evaluation committee members to assess the design's quality and completeness. Future students or developers who may wish to enhance or maintain the system will also find this document to be an essential resource.

4.1.3 Document Overview

The chapter starts with the design methodology and process model, then explains system architecture and process flows. After that, detailed design models, data design, database schema, data dictionary, and database collections are presented, followed by a chapter summary.

4.2 Design Methodology and Software Process Model

4.2.1 Design Methodology

EasyRide is built upon a **modular, component-based design** architecture that adheres to modern web development best practices. The design methodology is structured into the following logical layers:

1. **Presentation Layer (Frontend):** Constructed using React 18 components with Tailwind CSS and GSAP animations, managing distinct interfaces for passengers, drivers, and administrators.
2. **Application Layer (Backend API):** Powered by Node.js and Express.js 5.1, organized into modular route handlers, controllers, services, and middleware for Auth, Users, Drivers, Rides, Fares, Chat, Notifications, SOS, Support Tickets, Withdrawals, and Admin operations.
3. **Real-Time Communication Layer:** Facilitated through Socket.IO 4.8 for fare negotiation, ride status updates, in-ride chat, and push notifications.
4. **AI Service Layer:** An integrated module within the Node.js backend that interfaces with OpenAI's GPT-3.5 Turbo API for chatbot functionality.
5. **Data Layer:** Utilizes MongoDB with Mongoose ODM to store 12 collections for users, drivers, rides, messages, notifications, OTPs, fare settings, SOS records, support tickets, withdrawals, admin accounts, and blacklisted tokens.

The system is structured into logical layers but implemented within a unified Node.js architecture with REST and real-time capabilities. This approach ensures modularity, loose coupling, reusability, and scalability. The component-based design of React on the frontend mirrors the modular backend structure, enabling independent development and testing of each feature module.

4.2.2 Software Process Model

The development of EasyRide follows an **Agile and Iterative** software process model, ensuring flexibility and continuous improvement. Requirements are initially derived from the Software Requirements Specification (SRS). The development lifecycle is divided into small, manageable iterations or sprints, prioritizing core features as follows:

1. **Sprint 1:** User and Driver Registration, Authentication, and Profile Management (JWT, bcrypt).
2. **Sprint 2:** Ride Booking with Map Integration (Google Maps, OpenRouteService, Nominatim).
3. **Sprint 3:** Real-Time Fare Negotiation Engine (Socket.IO offer/counter-offer system).
4. **Sprint 4:** OTP-Based Ride Verification and GPS Tracking.

5. **Sprint 5:** In-Ride Chat, Notifications, and SOS Emergency Alerts.
6. **Sprint 6:** AI Chatbot Integration (OpenAI GPT-3.5 Turbo).
7. **Sprint 7:** Admin Panel: Driver Verification, Fare Configuration, Support Tickets, Withdrawals.

This approach allows for continuous integration and frequent testing, with feedback from the supervisor and stakeholders guiding the refinement of subsequent iterations. Each sprint concludes with a working increment that is demonstrated and reviewed before proceeding.

4.3 System Overview

EasyRide follows a three-tier web architecture with a clear separation of concerns between the frontend, backend API, real-time layer, and database. The following context diagram shows the system boundary, the three primary actors (Passenger, Driver, Admin), and the external services that EasyRide interacts with.

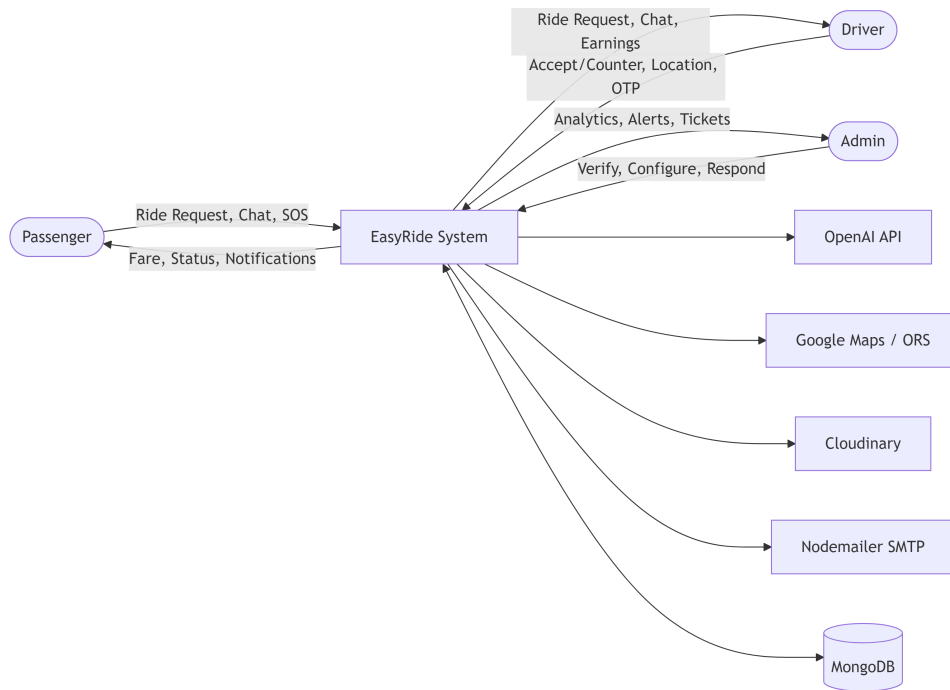


Figure 4.1: Context Diagram of EasyRide System

As shown in the context diagram, the EasyRide system communicates with six external entities: OpenAI GPT-3.5 API for chatbot functionality, Google Maps API for map rendering, OpenRouteService/Nominatim/OSRM for routing and geocoding, Cloudinary for image storage, Nodemailer for email and OTP delivery, and MongoDB for persistent data storage.

4.3.1 Presentation Layer (Frontend – React 18)

The Presentation Layer is a client-side Single Page Application (SPA) built using React 18 with Vite as the build tool. This layer communicates with the backend via REST APIs and handles real-time interactions through Socket.IO connections.

4.3.2 Application Layer (Backend API – Node.js and Express.js 5.1)

The Application Layer acts as the core of the system, powered by Node.js. It implements critical business logic such as fare calculation, geospatial driver matching, and JWT authentication.

4.3.3 Real-Time Communication Layer (Socket.IO 4.8)

The WebSocket server manages broadcasting ride requests, fare negotiation events, and real-time GPS location updates.

4.3.4 AI Service Layer (OpenAI GPT-3.5 Turbo)

This layer handles chatbot functionality, providing instant answers to ride-related queries and route suggestions.

4.3.5 Data Layer (MongoDB)

The Data Layer stores all persistent data across 12 collections using Mongoose ODM for schema validation and geospatial indexing.

4.4 Architectural Design

The high-level architecture of EasyRide is illustrated in the following figure. It shows the interaction between the React 18 frontend applications, Node.js/Express.js backend API, Socket.IO real-time server, MongoDB database, and external services.

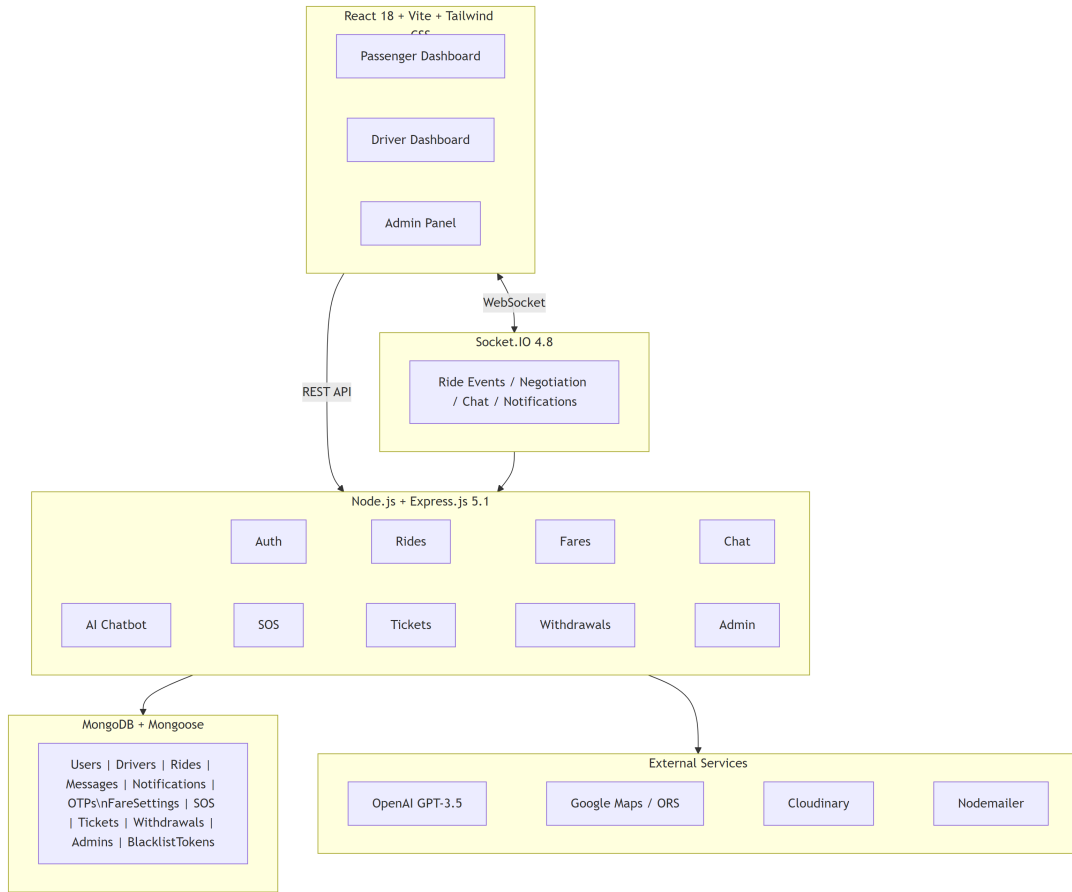


Figure 4.2: System Architecture Diagram of EasyRide

The architecture follows a clear separation of concerns:

- The **React 18 Frontend** communicates via RESTful APIs and Socket.IO.
- The **Node.js Backend** serves as the central coordinator.
- **MongoDB** provides persistent data storage.

4.5 Design Models

This section presents detailed design models of the system using UML diagrams.

4.5.1 Activity Diagrams

Activity diagrams represent the workflow of different modules of the system.

4.5.1.1 Module 1: User Registration and Authentication

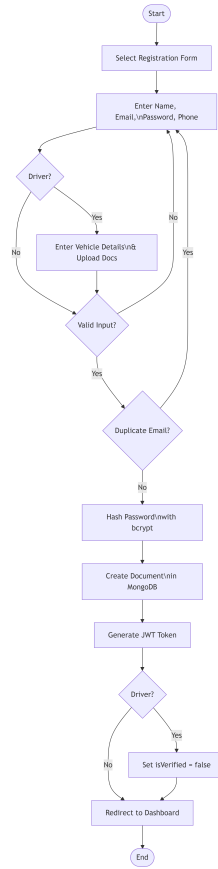


Figure 4.3: Activity Diagram – User Registration and Authentication

Figure 4.3 describes the complete user onboarding flow, starting from sign-up/login request and moving through credential validation and account creation. It highlights decision points for invalid inputs, duplicate email checks, and successful authentication states that route the user to the dashboard.

4.5.1.2 Module 2: Ride Booking and Fare Negotiation

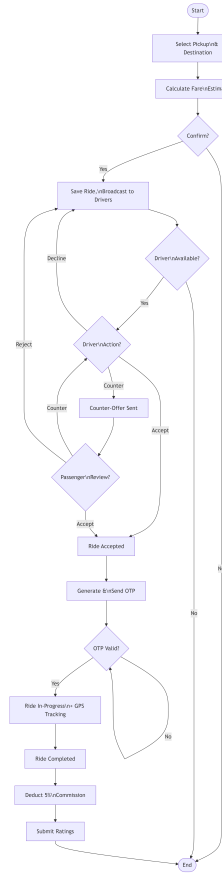


Figure 4.4: Activity Diagram – Ride Booking and Fare Negotiation

Figure 4.4 illustrates the booking lifecycle from pickup/drop-off selection to driver matching and fare negotiation. The diagram emphasizes counter-offer loops, acceptance or cancellation branches, and transition to ride confirmation once both passenger and driver agree on fare.

4.5.1.3 Module 3: Admin Operations

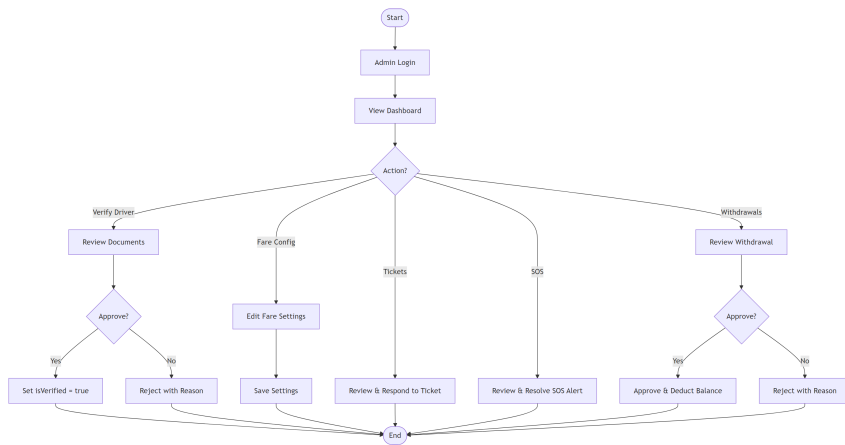


Figure 4.5: Activity Diagram – Admin Operations

Figure 4.5 captures administrative workflows such as driver verification, fare settings management, complaint handling, and withdrawal processing. It demonstrates that each admin action is validated and persisted to ensure governance, auditability, and operational control.

4.5.2 Class Diagram

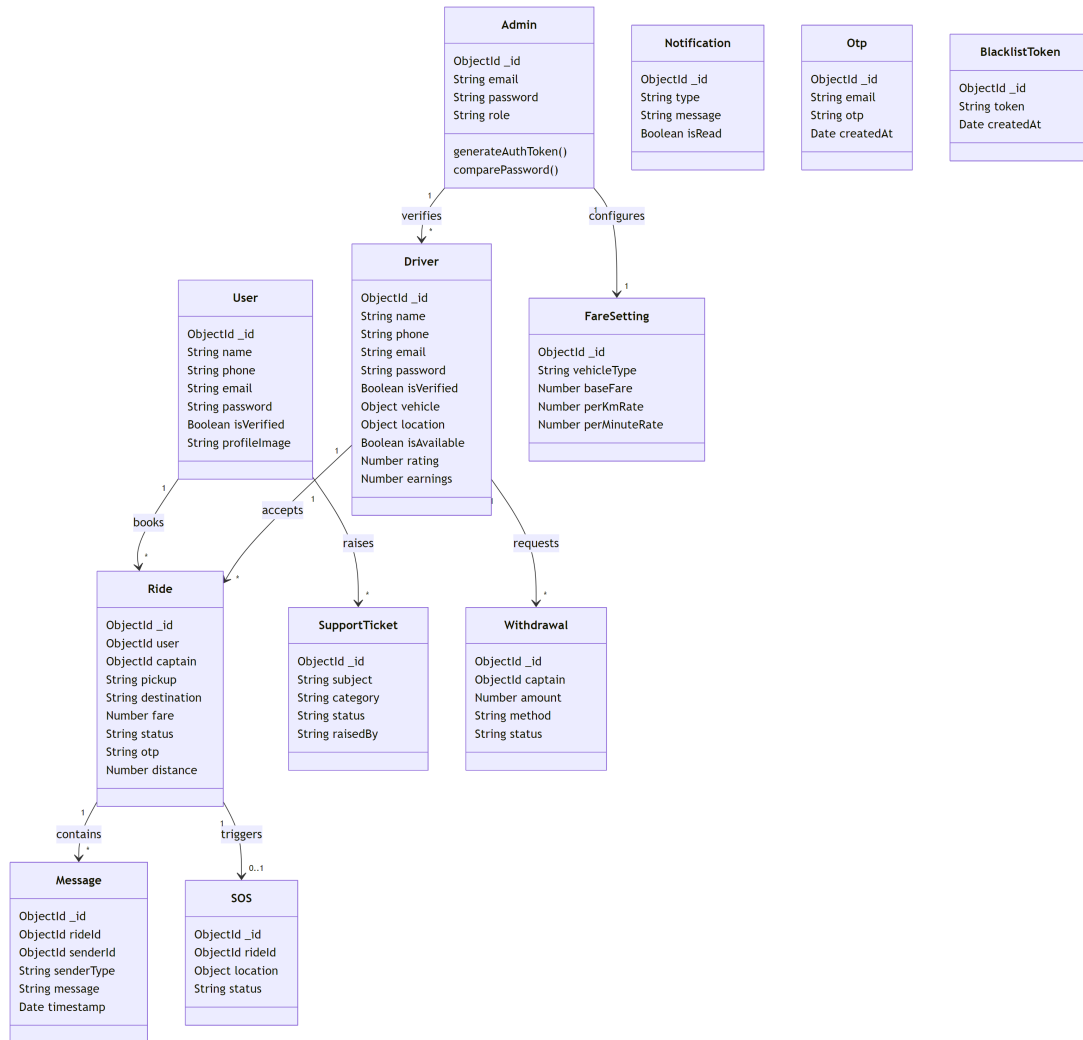


Figure 4.6: Complete Class Diagram of EasyRide System

Figure 4.6 presents the static structure of EasyRide by defining core entities, their attributes, and relationships. It clarifies how user, driver, ride, fare, message, notification, and admin-related classes interact to support modular implementation and maintainable backend logic.

4.5.3 Sequence Diagrams

Sequence diagrams illustrate the interaction between system components over time.

4.5.3.1 Authentication Module (Registration and Login)

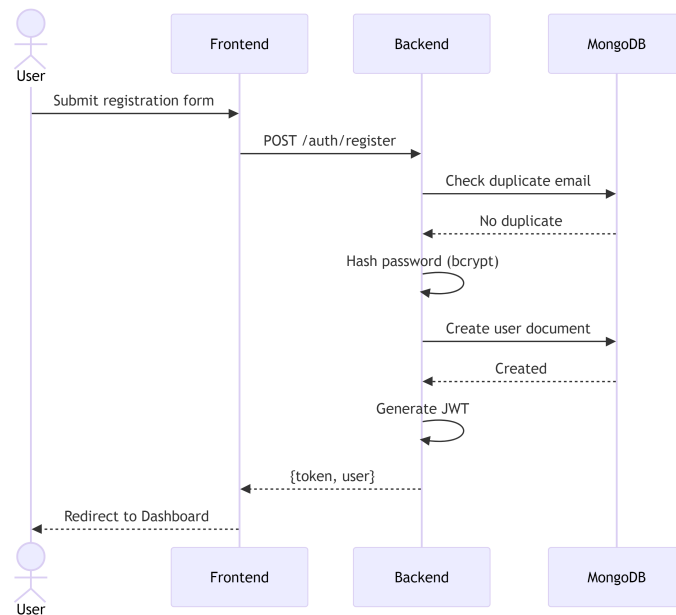


Figure 4.7: Sequence Diagram – User Registration

Figure 4.7 shows the message sequence for user registration, including form submission, validation, password hashing, and database insertion. The response path confirms either registration success or validation error feedback to the frontend.

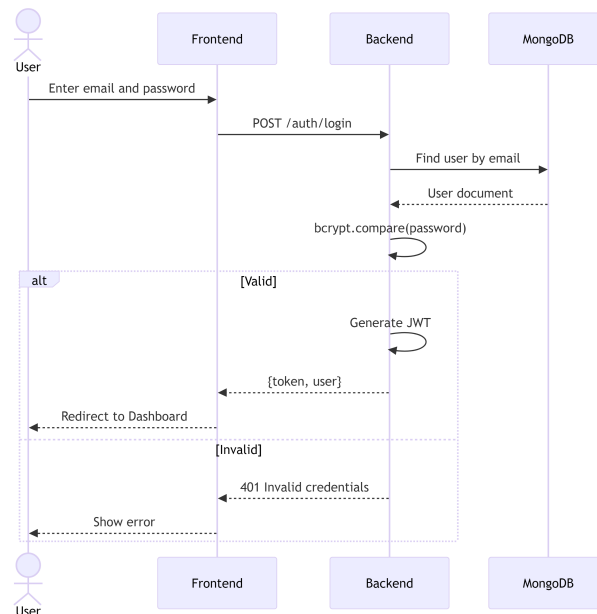


Figure 4.8: Sequence Diagram – User Login

Figure 4.8 models the login interaction where credentials are sent to the authentication service, verified against stored records, and mapped to JWT token generation. It reflects secure session establishment and controlled access to protected modules.

4.5.3.2 Ride and AI Modules

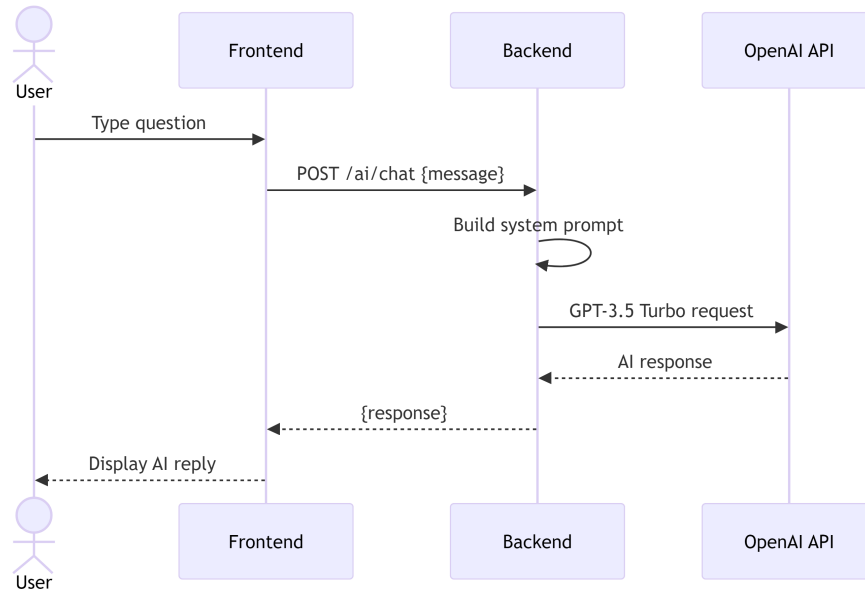


Figure 4.9: Sequence Diagram – AI Chatbot Interaction

Figure 4.9 explains how chatbot prompts are forwarded from the client to the backend AI service and then to the OpenAI API. It highlights response parsing and safe delivery of generated replies back to the user interface.

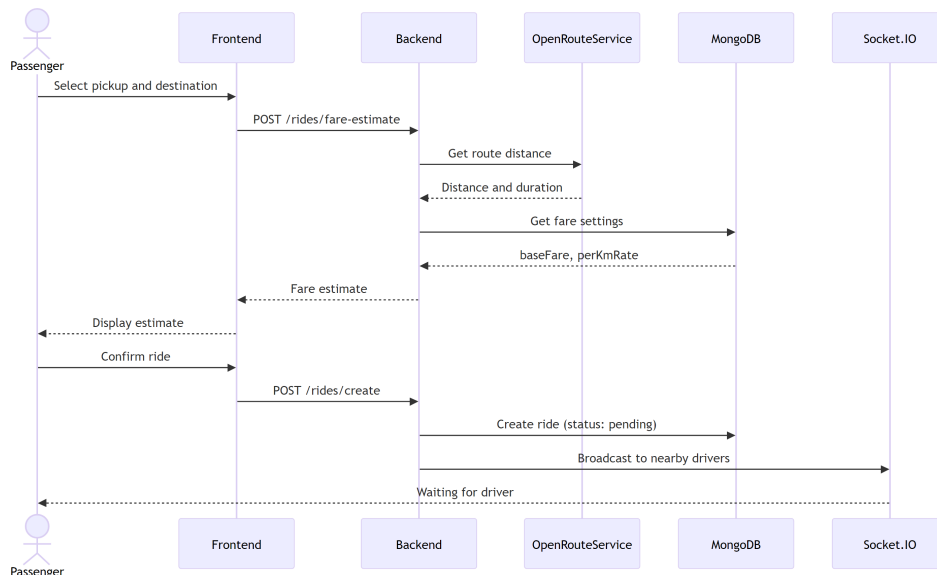


Figure 4.10: Sequence Diagram – Ride Booking

Figure 4.10 depicts the end-to-end ride booking sequence, including location input, route/fare estimation, driver broadcast, and booking confirmation. The diagram makes clear the coordination between frontend, API, and real-time socket channel.

4.5.3.3 Fare Negotiation and Driver Acceptance

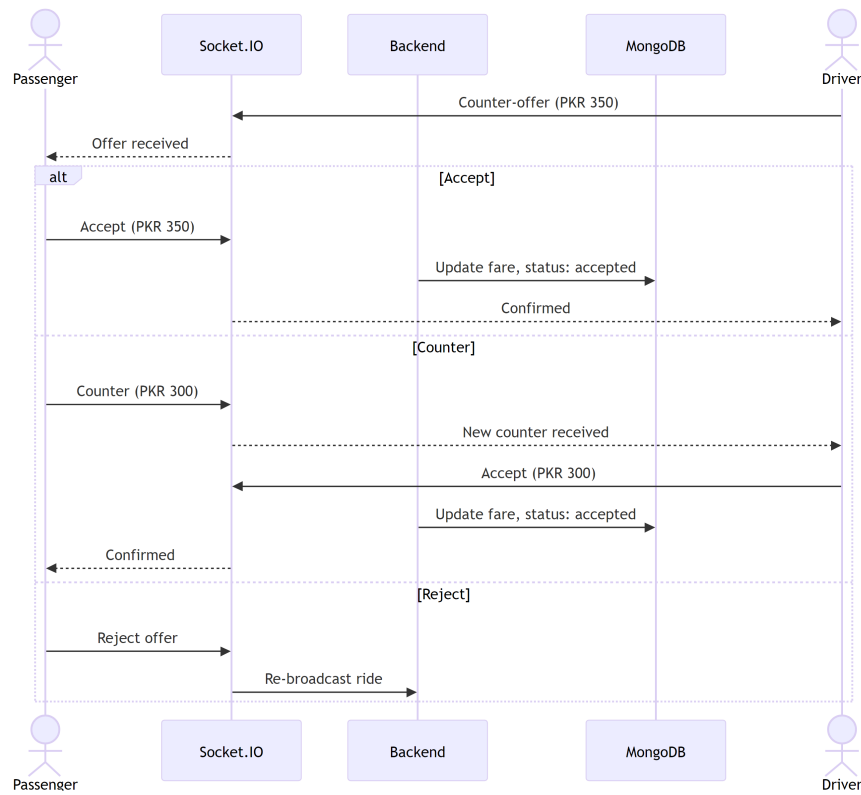


Figure 4.11: Sequence Diagram – Fare Negotiation

Figure 4.11 details bidirectional offer and counter-offer exchanges between passenger and driver through Socket.IO events. It demonstrates how negotiation remains active until one side accepts, rejects, or timeout conditions are reached.

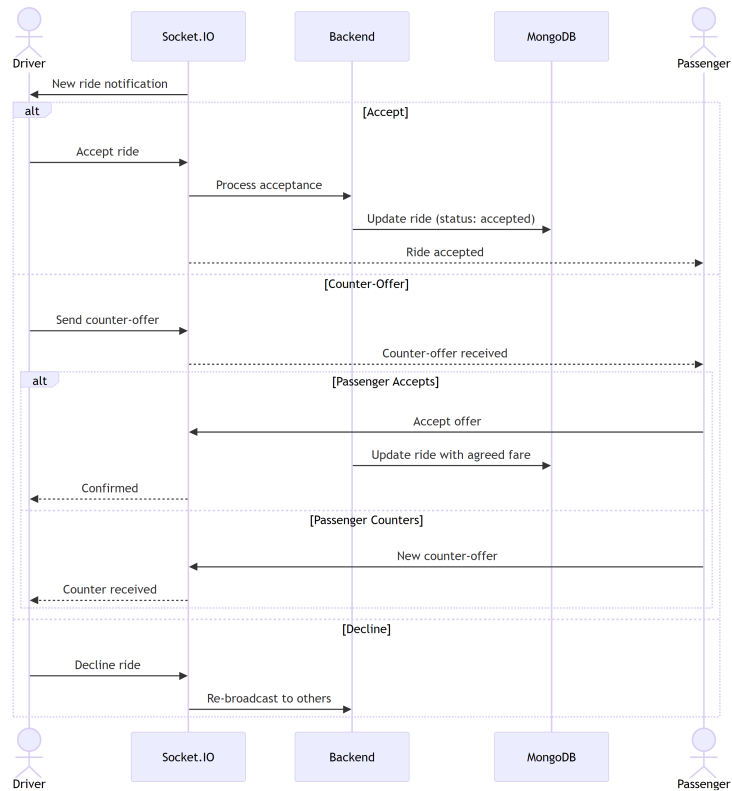


Figure 4.12: Sequence Diagram – Driver Ride Acceptance

Figure 4.12 explains the driver acceptance flow after fare agreement, including ride assignment, status transition to accepted, and notification delivery to both parties. This interaction ensures synchronized state across all connected clients.

4.5.3.4 Real-Time Chat

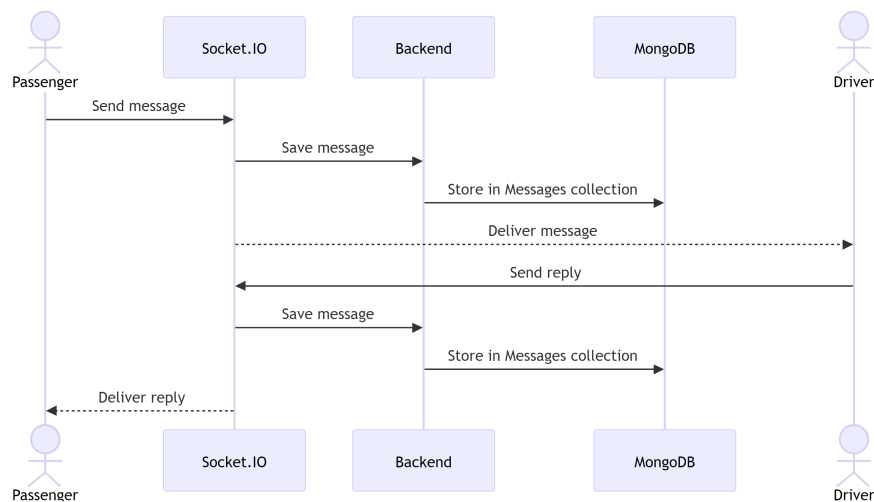


Figure 4.13: Sequence Diagram – Real-Time In-Ride Chat

Figure 4.13 illustrates in-ride chat messaging where each message is emitted, persisted, and broadcast in real time. It confirms reliable delivery and conversation continuity for passenger-driver communication during a ride.

4.5.4 State Transition Diagrams

State transition diagrams model how system entities change states over time.

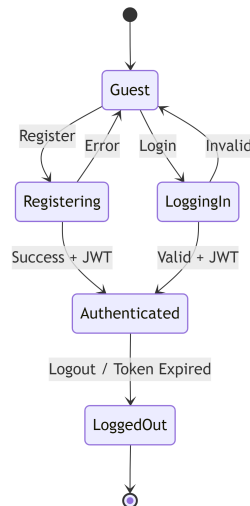


Figure 4.14: STD – User Authentication Process

Figure 4.14 defines the authentication state machine from unauthenticated to authenticated or failed states. It captures triggers such as valid credentials, invalid attempts, token issuance, and logout transition.

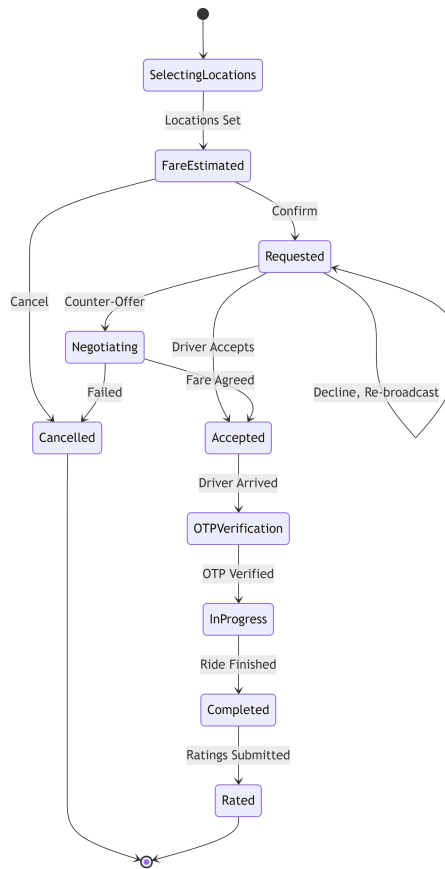


Figure 4.15: STD – Ride Booking Lifecycle

Figure 4.15 models the ride lifecycle through requested, negotiated, accepted, in-progress, completed, and cancelled states. It clarifies event-driven transitions so business rules remain consistent throughout ride processing.

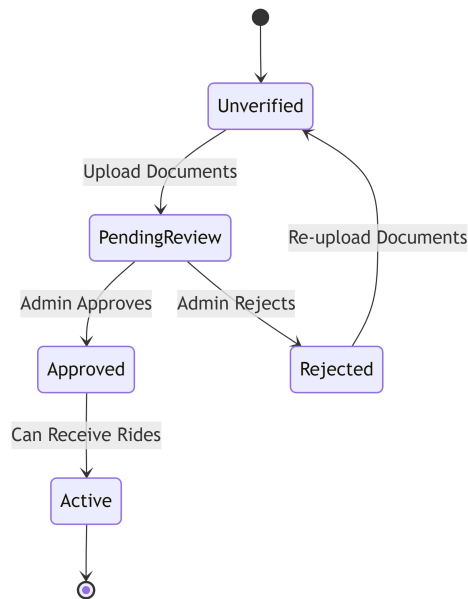


Figure 4.16: STD – Driver Verification Process

Figure 4.16 represents the driver verification pipeline from document submission to approval or rejection. It highlights admin review checkpoints that enforce platform safety and compliance requirements.

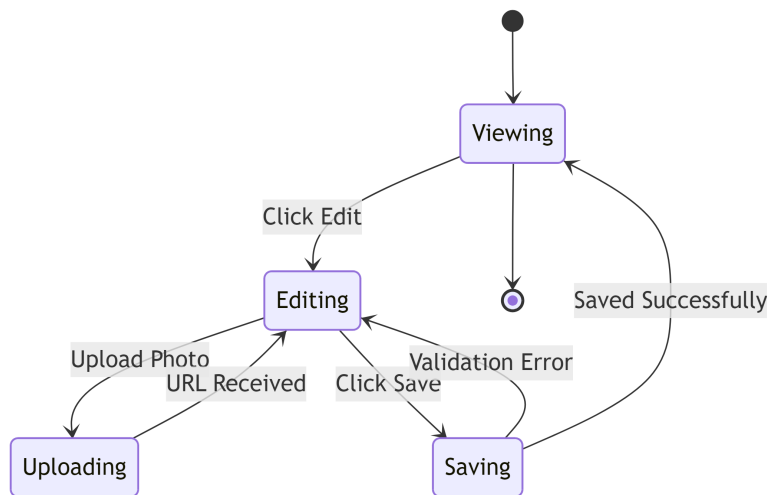


Figure 4.17: STD – User Profile Management

Figure 4.17 shows profile-related states triggered by update requests, validation checks, and data persistence outcomes. It ensures user information remains accurate while preventing invalid state transitions.

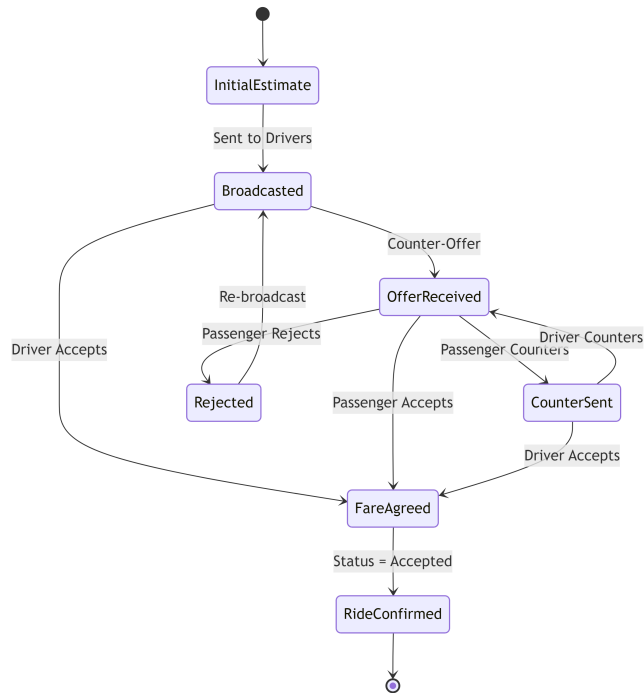


Figure 4.18: STD – Fare Negotiation Process

Figure 4.18 captures negotiation states including initiated, pending response, countered, accepted, and rejected. It formalizes timeout and termination conditions to avoid indefinite bargaining sessions.

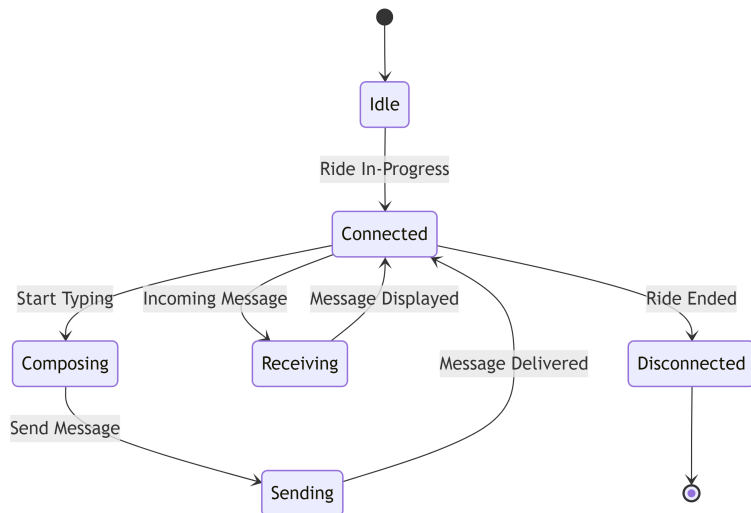


Figure 4.19: STD – Chat Conversation Lifecycle

Figure 4.19 describes chat session behavior from conversation start to active exchange and closure. It reflects message events, unread/read updates, and end-of-conversation handling for persistent chat history.

4.6 Data Flow Diagrams

4.6.1 Level 0 Data Flow Diagram

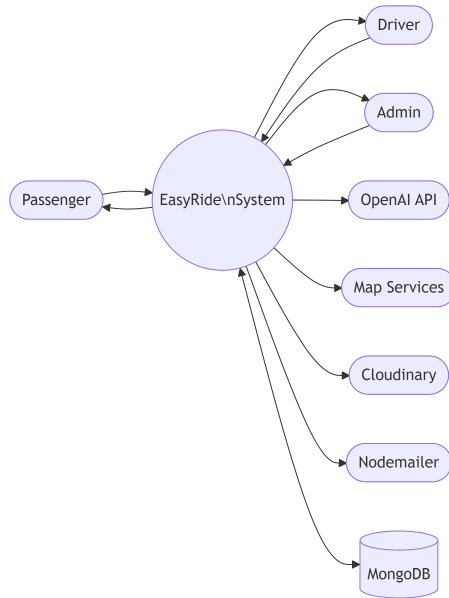


Figure 4.20: Data Flow Diagram Level 0

Figure 4.20 provides a context-level view of EasyRide by showing external actors and major data exchanges with the system boundary. It summarizes how requests, responses, and service data flow at a high level.

4.6.2 Level 1 Data Flow Diagram

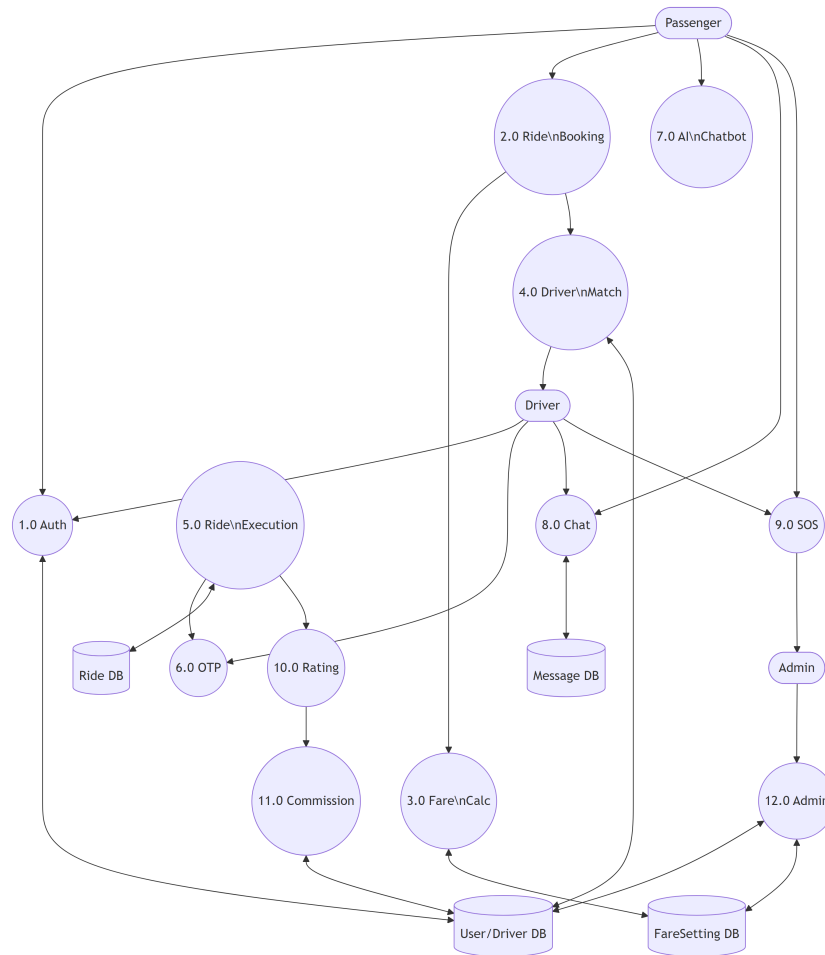


Figure 4.21: Data Flow Diagram Level 1

Figure 4.21 decomposes the system into key internal processes such as authentication, ride handling, negotiation, notifications, and admin control. It identifies data stores and detailed process-to-process data movement.

4.7 Data Design

4.7.1 Entity Relationship Diagram (ERD)

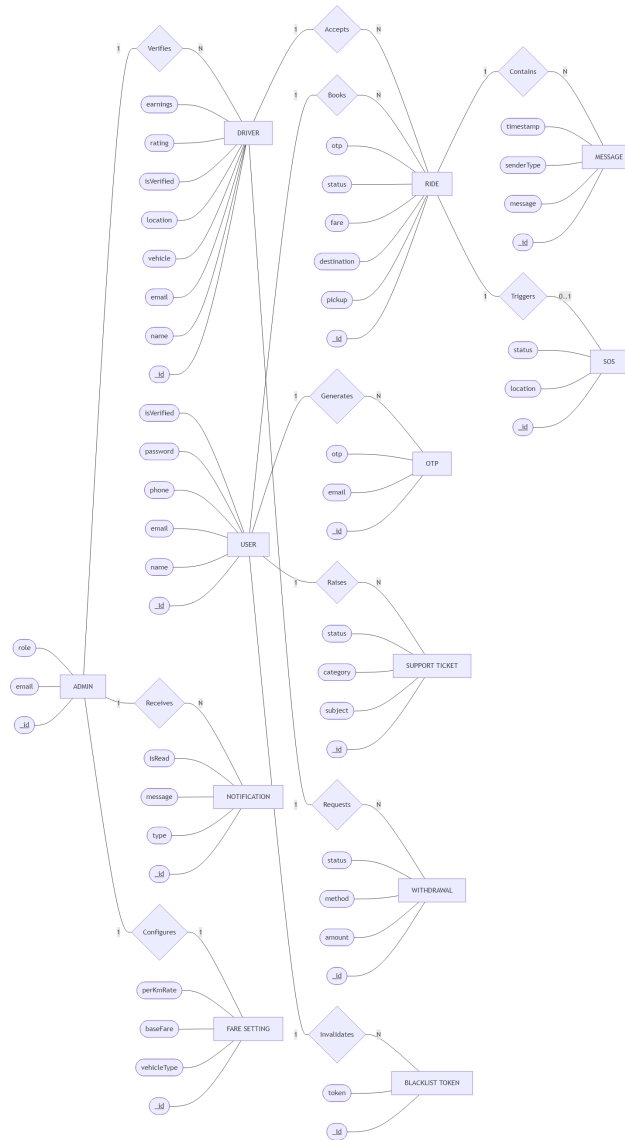


Figure 4.22: Entity Relationship Diagram (ERD) of EasyRide

Figure 4.22 presents the logical database design by defining entities and their cardinal relationships. It demonstrates how User, Driver, Ride, Message, Notification, SOS, and other entities are connected through primary and foreign key references.

4.7.2 Database Architecture

EasyRide follows a client-server data architecture where the Node.js API layer communicates with MongoDB through Mongoose ODM. The architecture supports secure CRUD operations, schema-

level validation, and role-based access to collections. Indexing on frequently queried fields such as email, ride status, and geospatial location improves query performance and system responsiveness.

4.8 Database Schema Diagram

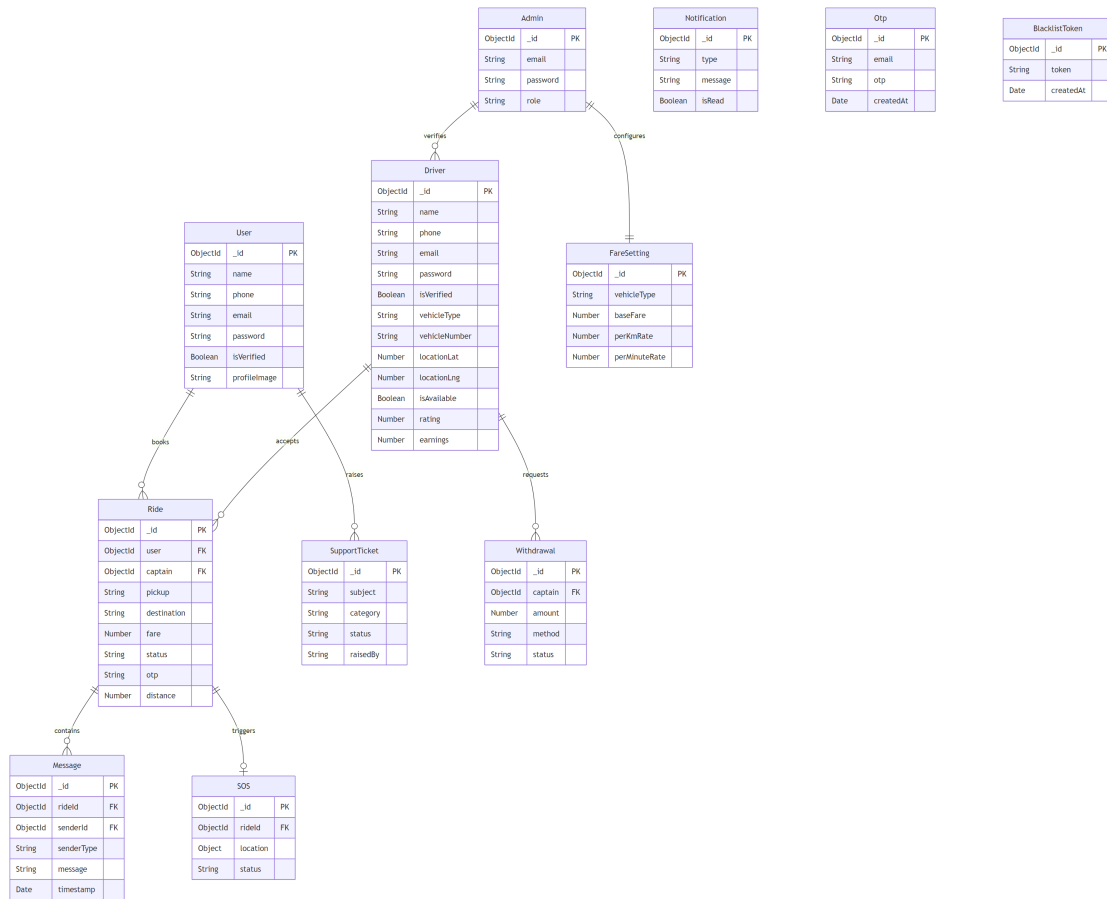


Figure 4.23: Database Schema Diagram of EasyRide

Figure 4.23 maps the physical schema implementation with collection fields, key constraints, and references used in MongoDB. It provides a direct bridge from conceptual ERD design to deployable storage structure in the backend.

4.8.1 Data Dictionary

4.8.1.1 Entity: User

Table 4.1: Data Dictionary – User Collection

Field Name	Data Type	Constraint	Description
_id	ObjectId	Primary Key	Unique identifier for each user
name	String	Not Null	Full name of the passenger
email	String	Unique, Not Null	Login email address
password	String	Not Null	Encrypted password using bcrypt
phone	String	Not Null	Contact phone number
role	String	Default “user”	User role identifier

4.8.1.2 Entity: Ride

Table 4.2: Data Dictionary – Ride Collection

Field Name	Data Type	Constraint	Description
_id	ObjectId	Primary Key	Unique ride identifier
passenger	ObjectId	FK to User	Reference to the passenger
driver	ObjectId	FK to Driver	Reference to the assigned driver
otp	String	Optional	OTP used for ride verification before start
fare	Number	Not Null	Agreed fare amount in PKR
status	String	Not Null	requested, accepted, in-progress, completed

4.8.1.3 Entity: Driver

Table 4.3: Data Dictionary – Driver Collection

Field Name	Data Type	Constraint	Description
_id	ObjectId	Primary Key	Unique identifier for each driver
name	String	Not Null	Full name of the driver
email	String	Unique, Not Null	Driver login email
password	String	Not Null	Encrypted password for authentication
phone	String	Not Null	Driver contact number
vehicle	String	Not Null	Vehicle details assigned to the driver
location	Object	Optional	Current geolocation for nearby matching
isVerified	Boolean	Default false	Verification status approved by admin
rating	Number	Default 0	Average rating from completed rides
earnings	Number	Default 0	Total earned amount by driver

4.8.1.4 Entity: Admin

Table 4.4: Data Dictionary – Admin Collection

Field Name	Data Type	Constraint	Description
_id	ObjectId	Primary Key	Unique identifier for admin
email	String	Unique, Not Null	Admin account email
password	String	Not Null	Encrypted admin password
role	String	Default “admin”	System administration role

4.8.1.5 Entity: Message

Table 4.5: Data Dictionary – Message Collection

Field Name	Data Type	Constraint	Description
_id	ObjectId	Primary Key	Unique message identifier
ride	ObjectId	FK to Ride	Ride session linked with this message
message	String	Not Null	Message body content
senderType	String	Not Null	Sender role: user or driver
timestamp	Date	Default now	Message creation time

4.8.1.6 Entity: SOS

Table 4.6: Data Dictionary – SOS Collection

Field Name	Data Type	Constraint	Description
_id	ObjectId	Primary Key	Unique SOS event identifier
ride	ObjectId	FK to Ride	Ride that triggered the SOS alert
location	Object	Not Null	Geo-coordinates of emergency location
status	String	Not Null	open, acknowledged, resolved

4.8.1.7 Entity: OTP

Table 4.7: Data Dictionary – OTP Collection

Field Name	Data Type	Constraint	Description
_id	ObjectId	Primary Key	Unique OTP record identifier
email	String	Not Null	Email where OTP was delivered
otp	String	Not Null	One-time verification code
expiresAt	Date	Not Null	OTP expiration timestamp

4.8.1.8 Entity: Support Ticket

Table 4.8: Data Dictionary – Support Ticket Collection

Field Name	Data Type	Constraint	Description
_id	ObjectId	Primary Key	Unique support ticket identifier
user	ObjectId	FK to User	User who raised the ticket
subject	String	Not Null	Ticket title/subject
category	String	Not Null	Complaint category
status	String	Not Null	open, in-progress, resolved, closed

4.8.1.9 Entity: Notification

Table 4.9: Data Dictionary – Notification Collection

Field Name	Data Type	Constraint	Description
_id	ObjectId	Primary Key	Unique notification identifier
recipient	ObjectId	Not Null	Target user/driver/admin receiving alert
type	String	Not Null	Notification type (ride, system, warning)
message	String	Not Null	Notification content
isRead	Boolean	Default false	Read/unread status

4.8.1.10 Entity: Withdrawal

Table 4.10: Data Dictionary – Withdrawal Collection

Field Name	Data Type	Constraint	Description
_id	ObjectId	Primary Key	Unique withdrawal request identifier
driver	ObjectId	FK to Driver	Driver requesting payout
amount	Number	Not Null	Requested withdrawal amount
method	String	Not Null	Payout method (bank/e-wallet)
status	String	Not Null	pending, approved, rejected, processed

4.8.1.11 Entity: Fare Setting

Table 4.11: Data Dictionary – Fare Setting Collection

Field Name	Data Type	Constraint	Description
_id	ObjectId	Primary Key	Unique fare-setting identifier
vehicleType	String	Not Null	Vehicle category (bike, car, etc.)
baseFare	Number	Not Null	Fixed base fare for ride start
perKmRate	Number	Not Null	Variable rate per kilometer
configuredBy	ObjectId	FK to Admin	Admin who configured these values

4.8.1.12 Entity: Blacklist Token

Table 4.12: Data Dictionary – Blacklist Token Collection

Field Name	Data Type	Constraint	Description
_id	ObjectId	Primary Key	Unique blacklist record identifier
token	String	Unique, Not Null	Invalidated JWT token string
invalidatedBy	ObjectId	FK to User/Admin	Actor who invalidated the token
createdAt	Date	Default now	Time when token was blacklisted

4.9 Database Tables / Collections

This section describes the logical database design of the system. It presents the major relational tables (for SQL-based systems) or collections (for NoSQL-based systems). The database structure is designed to ensure data consistency, integrity, security, and scalability.

Each table or collection includes primary keys, relationships, and major attributes relevant to system functionality.

4.9.1 User Collection / Table

The User table stores authentication credentials and user profile information. It supports role-based access control and secure authentication.

Description:

- Primary Key: `user_id`

- Relationships: Linked to transactions, predictions, or system logs.
- Authentication Attributes: Email, password hash, role, account status.

Table 4.13: User Table Structure

Field Name	Data Type	Constraint	Description
user_id	INT / UUID	Primary Key	Unique identifier for each user
full_name	VARCHAR (100)	Not Null	Stores user’s full name
email	VARCHAR (150)	Unique, Not Null	User login email address
password_hash	VARCHAR (255)	Not Null	Encrypted password using hashing algorithm
role	VARCHAR(50)	Not Null	Defines user access level (Admin, User, etc.)
created_at	TIMESTAMP	Default Current	Account creation timestamp
status	BOOLEAN	Default True	Indicates whether account is active

4.9.2 Core Data Collection

This subsection describes domain-specific core data tables or collections. These structures store primary operational data processed by the system.

Domain-Specific Examples:
For Machine Learning Systems:

- Dataset Collection: Stores training and testing datasets.
- Model Metadata Collection: Stores model version, hyperparameters, and evaluation metrics.
- Prediction Results Collection: Stores inference results and timestamps.

For Web-Based Systems:

- Orders Table: Stores customer purchase records.
- Products Table: Stores product details and pricing.
- Transactions Table: Stores payment and transaction history.

For Mobile Applications:

- Device Info Table: Stores device metadata.
- Session Table: Tracks active user sessions.

For Blockchain-Based Systems:

- Transaction Collection: Stores blockchain transaction records.
- Wallet Collection: Stores wallet identifiers and balances.
- Block Metadata Collection: Stores block hash, timestamp, and validation status.

4.9.3 Relationships Between Tables

Database relationships define how data entities are connected within the system.

- One-to-One (1:1): Each user may have one profile detail record.
- One-to-Many (1:N): One user may generate multiple transactions or predictions.
- Many-to-Many (M:N): Users may interact with multiple products, and products may belong to multiple categories (resolved using junction tables).

An Entity-Relationship (ER) Diagram should be included to visually represent these relationships.

4.9.4 Security and Data Integrity

To ensure confidentiality and integrity of stored data, the following measures are implemented:

- Password encryption using secure hashing algorithms (e.g., bcrypt, SHA-256 with salt).
- Role-Based Access Control (RBAC) to restrict unauthorized access.
- Input validation and constraint enforcement at database level.
- Secure communication using HTTPS.
- Regular database backup and disaster recovery strategy.

4.9.5 Scalability Considerations

The database design incorporates scalability mechanisms to support future growth.

- Horizontal scaling using distributed database systems (if required).
- Indexing on frequently queried attributes to improve performance.
- Caching strategies for frequently accessed data.
- Load balancing across database instances.
- Partitioning or sharding for large-scale data environments.

4.10 Chapter Summary

This chapter presented the comprehensive software design of the proposed system. It translated the functional and non-functional requirements identified earlier into structured architectural, behavioral, and data models.

The design methodology and selected software process model were justified to ensure systematic development and maintainability. A high-level system overview was provided through the context diagram and architectural representation, clearly illustrating component interactions and subsystem boundaries.

Detailed UML models including activity diagrams, class diagrams, sequence diagrams, and state transition diagrams were developed to describe system behavior and structure. Additionally, data flow diagrams were constructed to represent internal data movement across different system layers.

The chapter further elaborated the database design through Entity Relationship Diagrams, database schema diagrams, and data dictionaries to ensure data integrity, scalability, and security. Relationships between entities and normalization considerations were also addressed.

Overall, this chapter establishes a clear technical blueprint that guides the implementation phase discussed in the next chapter.

Chapter 5

Implementation

This chapter describes the practical implementation of the EasyRide platform. It explains how the designed architecture presented in Chapter 4 was translated into working modules, algorithms, user interfaces, and deployment components. The implementation follows a modular, component-based approach using React 18 with Tailwind CSS and GSAP for the frontend, Node.js with Express.js 5.1 for the backend API, Socket.IO 4.8 for real-time communication, and MongoDB with Mongoose for data persistence.

5.1 Development Environment

This section describes the software and hardware environment used for implementing the EasyRide platform.

Table 5.1: Development Environment

Component	Details
IDE	Visual Studio Code (VS Code) with extensions for React, Node.js, and MongoDB
Frontend Language	JavaScript (ES6+), JSX, CSS3
Frontend Framework	React 18 with Vite 6 bundler, Tailwind CSS 3, and GSAP 3 animations
State Management	React Context API and Hooks
Backend Language	JavaScript (Node.js 20+)
Backend Framework	Express.js 5.1
AI Integration	OpenAI GPT-3.5 Turbo API
Real-Time Engine	Socket.IO 4.8
Database	MongoDB 7.x
ODM	Mongoose 8.x
Authentication	JSON Web Tokens (JWT) 9.x, bcrypt 5.x
Image Uploads	Cloudinary API with Multer middleware
Maps and Routing	Google Maps API, OpenRouteService, Nominatim, OSRM
Email Service	Nodemailer 6.x
Version Control	Git with GitHub
Operating System	Windows 11
Hardware	Intel Core i5/i7 processor, 8 GB RAM, SSD storage
Browser Testing	Google Chrome, Mozilla Firefox, Microsoft Edge

5.2 Core Module Implementation

This section explains how each major module of the EasyRide platform was implemented.

5.2.1 Module 1: User and Driver Management

The User Management module handles registration, login, profile management, and role-based access control for three user types: Passenger, Driver, and Admin.

Key Functionalities:

- Separate registration flows for passengers and drivers, with drivers providing vehicle details and license information.
- Secure login using bcrypt password hashing and JWT token-based session management.
- Token blacklisting on logout to prevent reuse of expired sessions.
- Role-based middleware that restricts API endpoints based on user role.
- Profile viewing and editing with optional profile picture upload via Cloudinary.

Key Components:

- `authController.js` – Handles registration, login, and token management for both users and drivers.
- `userController.js` and `driverController.js` – Manage profile operations for each role.
- `authMiddleware.js` – Verifies JWT tokens and enforces role-based access control.
- `User.js` and `Driver.js` (Mongoose Models) – Define schemas with field validation rules.
- `BlacklistToken.js` (Mongoose Model) – Stores invalidated JWT tokens.

5.2.2 Module 2: Ride Booking and Fare Negotiation

The Ride Booking module is the core feature of EasyRide, managing the complete ride creation and real-time fare negotiation process.

Key Functionalities:

- Interactive map interface for selecting pickup and destination locations using Google Maps API.
- Automated fare estimation based on distance from OpenRouteService and configurable fare parameters (base fare, per-km rate, minimum fare) stored in the FareSetting collection.
- Real-time broadcasting of ride requests to nearby available drivers via Socket.IO.
- Dynamic fare negotiation with offer and counter-offer exchange between passenger and driver through WebSocket events.

- Ride confirmation when both parties agree on a fare, with status updates broadcast in real time.

Key Components:

- `RideBooking.jsx` – React component with map integration, location search, and fare display.
- `rideController.js` – Handles ride creation, acceptance, and status transitions.
- `fareController.js` – Manages fare estimation and admin fare configuration.
- `socket.js` – Socket.IO server handling ride events (new-ride, accept-ride, counter-offer).
- `Ride.js` and `FareSetting.js` (Mongoose Models) – Define ride and fare configuration schemas.
- `fareEstimate.js` (Utility) – Calculates fare based on distance and configured parameters.

5.2.3 Module 3: Ride Execution and Tracking

The Ride Execution module manages the active phase of rides including OTP verification, GPS tracking, and ride completion with commission deduction.

Key Functionalities:

- Generation of a 4-digit OTP sent to the passenger via email using Nodemailer.
- OTP verification by the driver to start the ride, preventing unauthorized pickups.
- Real-time GPS location tracking of the driver displayed on the passenger's map via Socket.IO events.
- Route visualization using Google Maps API polylines and OpenRouteService routing.
- Ride completion with automatic 5% commission deduction and balance credit to driver account.
- Mutual rating submission (1–5 stars) by both passenger and driver after ride completion.

Key Components:

- `RideTracking.jsx` – React component with real-time map display and OTP interface.
- `rideController.js` – Handles OTP generation, ride start, and completion logic.
- `Otp.js` (Mongoose Model) – Stores OTP records with TTL index for automatic expiration.
- `socket.js` – Handles real-time location update events between driver and passenger.

5.2.4 Module 4: AI Chatbot

The AI Chatbot module provides instant, intelligent assistance to users through a conversational interface powered by OpenAI's GPT-3.5 Turbo model.

Key Functionalities:

- Natural language query processing for ride-related questions (fare queries, route suggestions, platform help).
- System prompt engineering with ride-hailing context to ensure relevant, domain-specific responses.
- Conversational interface with message bubbles for both user and AI responses.
- Support for general platform queries about features, booking process, and usage guidance.

Key Components:

- `ChatBot.jsx` – React component for the chat interface with input field and message display.
- `aiController.js` – Handles AI chat requests through the Express.js endpoint.
- `aiService.js` – Core service that constructs prompts and calls the OpenAI API.

5.2.5 Module 5: Safety and Communication Features

The Safety module implements emergency mechanisms and in-ride communication to ensure user security during rides.

Key Functionalities:

- SOS emergency alert system allowing either passenger or driver to trigger a distress signal during active rides.
- Automatic GPS location capture at the time of SOS trigger for admin reference.
- Real-time in-ride text chat between passenger and driver via Socket.IO.
- Support ticket system for non-emergency complaints and feedback submissions.
- Notification system for ride updates, SOS alerts, verification results, and withdrawal status.

Key Components:

- `sosController.js` – Handles SOS alert creation and admin notification.
- `chatController.js` – Manages in-ride messaging between passengers and drivers.
- `ticketController.js` – Handles support ticket creation and admin responses.
- `notificationController.js` – Manages push notifications via Socket.IO.
- `sos.model.js`, `Message.js`, `SupportTicket.js`, `Notification.js` (Mongoose Models).

5.2.6 Module 6: Admin Panel

The Admin Panel provides comprehensive management tools for platform administrators through a dedicated React frontend application.

Key Functionalities:

- Driver verification workflow: review uploaded license and vehicle documents, approve or reject with reason.
- User and driver account management including activation and deactivation.
- Dynamic fare settings configuration (base fare, per-km rate, minimum fare) applied system-wide.
- Support ticket and SOS alert management with resolution tracking.
- Driver withdrawal request processing and approval.
- Platform analytics dashboard showing total users, drivers, rides, revenue, and pending actions.

Key Components:

- `AdminDashboard.jsx` – React component with analytics cards and navigation.
- `adminController.js` – Handles admin operations for verification, user management, and analytics.
- `withdrawalController.js` – Manages driver withdrawal request processing.
- `Admin.js` (Mongoose Model) – Admin account schema with role-based access.

5.3 Algorithm Implementation

This section explains the implementation of the core algorithms used in the EasyRide platform.

5.3.1 Algorithm 1: User Login Authentication

The user login process securely authenticates users before granting access to the system.

Input: Email and password from login form

Output: JWT authentication token or error message

Step-by-Step Working:

1. Read email and password from login form submission.
2. Determine if the user is a Passenger or Driver based on the login endpoint.
3. Find the user by email in the appropriate MongoDB collection.
4. If user not found, return “Invalid credentials” error.

5. Compare submitted password with stored hash using bcrypt.
6. If mismatch, return “Invalid credentials” error.
7. If match, create JWT token containing userId, role, and expiration.
8. Send token and user data to the frontend for session management.

Algorithm 1 User Login Authentication

```

1: Input: Email  $e$ , Password  $p$ , Role  $r$ 
2: Output: JWT token  $T$  or error message
3: function USERLOGIN( $e, p, r$ )
4:    $collection \leftarrow$  if  $r = \text{“driver”}$  then  $Driver$  else  $User$                                 ▷ Select collection
5:    $user \leftarrow collection.findByEmail(e)$                                           ▷ Query database
6:   if  $user = \text{null}$  then
7:     return “Invalid credentials”                                                  ▷ User not found
8:   end if
9:    $isMatch \leftarrow bcrypt.compare(p, user.password)$                              ▷ Compare hashes
10:  if  $isMatch = \text{false}$  then
11:    return “Invalid credentials”                                                  ▷ Wrong password
12:  end if
13:   $T \leftarrow JWT.sign(\{userId : user.id, role : r\})$                              ▷ Generate token
14:  return  $\{T, user\}$                                                               ▷ Send token and data
15: end function

```

5.3.2 Algorithm 2: Fare Estimation

The fare estimation algorithm calculates the expected ride cost based on distance and configurable parameters.

Input: Pickup coordinates, destination coordinates

Output: Estimated fare in PKR

Step-by-Step Working:

1. Receive pickup and destination coordinates from the map interface.
2. Call OpenRouteService API to calculate the route distance in kilometers.
3. Fetch current fare settings from the FareSetting collection (base fare, per-km rate, minimum fare).
4. Calculate fare: $baseFare + (distance \times perKmRate)$.
5. If calculated fare is less than the minimum fare, use the minimum fare.
6. Return the rounded fare estimate to the frontend.

Algorithm 2 Fare Estimation

```
1: Input: Pickup coordinates  $P$ , Destination coordinates  $D$ 
2: Output: Estimated fare  $F$  in PKR
3: function FAREESTIMATE( $P, D$ )
4:    $distance \leftarrow ORS.getDistance(P, D)$  ▷ Call routing API
5:    $settings \leftarrow FareSetting.findOne()$  ▷ Fetch fare config
6:    $F \leftarrow settings.baseFare + (distance \times settings.perKmRate)$  ▷ Calculate fare
7:   if  $F < settings.minimumFare$  then
8:      $F \leftarrow settings.minimumFare$  ▷ Apply minimum fare
9:   end if
10:   $F \leftarrow \lfloor F + 0.5 \rfloor$  ▷ Round to nearest integer
11:  return  $\{F, distance\}$  ▷ Return fare and distance
12: end function
```

5.3.3 Algorithm 3: Nearby Driver Matching

The driver matching algorithm finds available drivers within a specified radius of the passenger's pickup location.

Input: Pickup coordinates, search radius

Output: List of available nearby drivers

Step-by-Step Working:

1. Receive the passenger's pickup latitude and longitude.
2. Query the Driver collection for drivers where `isAvailable` is true and `isVerified` is true.
3. Use the 2dsphere geospatial index on the location field to find drivers within the search radius.
4. Sort results by distance from pickup (closest first).
5. Broadcast the ride request to matched drivers via Socket.IO.

Algorithm 3 Nearby Driver Matching

```
1: Input: Pickup latitude  $lat$ , Pickup longitude  $lng$ , Radius  $R$ 
2: Output: List of nearby drivers  $N$ 
3: function FINDNEARBYDRIVERS( $lat, lng, R$ )
4:    $N \leftarrow \emptyset$  ▷ Initialize empty list
5:    $drivers \leftarrow Driver.find(\{isAvailable = true, isVerified = true\})$  ▷ Query verified
   drivers
6:   for each  $d$  in  $drivers$  do
7:      $dist \leftarrow GeoDistance(d.location, \{lat, lng\})$  ▷ Compute distance
8:     if  $dist \leq R$  then
9:        $N \leftarrow N \cup \{d\}$  ▷ Add driver to list
10:    end if
11:  end for
12:   $N \leftarrow Sort(N, \text{by distance ascending})$  ▷ Closest first
13:  for each  $d$  in  $N$  do
14:     $Socket.emit("new-ride", rideDetails, d.socketId)$  ▷ Broadcast request
15:  end for
16:  return  $N$  ▷ Return matched drivers
17: end function
```

5.3.4 Algorithm 4: OTP Verification for Ride Start

The OTP algorithm generates and verifies a one-time password to authenticate the passenger before starting a ride.

Input: Ride ID for generation; Ride ID and OTP code for verification

Output: OTP sent confirmation; or ride start confirmation

Step-by-Step Working:

1. When the driver arrives at the pickup, the system generates a random 4-digit OTP.
2. The OTP is stored in the Otp collection with a TTL index for automatic expiration (e.g., 10 minutes).
3. The OTP is also saved on the Ride document and sent to the passenger via email using Nodemailer.
4. The driver enters the OTP received from the passenger verbally.
5. The system compares the entered OTP with the stored value.
6. If matched, the ride status changes to “in-progress” and real-time tracking begins.
7. If mismatched, the system returns an error and the driver can retry.

Algorithm 4 OTP Generation and Verification

```
1: Input: Ride identifier rideId; Entered OTP enteredOtp (for verification)
2: Output: OTP sent confirmation; or ride start confirmation / error

3: function GENERATEOTP(rideId)
4:   otp  $\leftarrow$  Random.generate(4 digits)                                ▷ Generate 4-digit code
5:   Otp.create({rideId, otp, expiresAt : now + 10min})                ▷ Store with TTL
6:   Ride.update(rideId, {otp : otp})                                ▷ Save on ride document
7:   passenger  $\leftarrow$  Ride.findById(rideId).passenger                ▷ Get passenger
8:   Email.send(passenger.email, "Your OTP: " + otp)                ▷ Send via email
9:   return "OTP sent"                                                ▷ Confirm generation
10: end function

11: function VERIFYOTP(rideId, enteredOtp)
12:   ride  $\leftarrow$  Ride.findById(rideId)                                ▷ Fetch ride record
13:   if ride.otp  $\neq$  enteredOtp then
14:     return "Invalid OTP"                                            ▷ Mismatch error
15:   end if
16:   Ride.update(rideId, {status : "in-progress"})                ▷ Update ride status
17:   Socket.emit("ride-started", passenger)                        ▷ Notify in real-time
18:   return "Ride started"                                            ▷ Success
19: end function
```

5.3.5 Algorithm 5: AI Chatbot Response

The AI chatbot processes user queries and generates ride-related responses using the OpenAI API.

Input: User message text

Output: AI-generated response text

Step-by-Step Working:

1. Receive the message text from the chatbot frontend via the Express.js API.
2. Validate and sanitize the input message.
3. Construct a system prompt defining the chatbot's role as an EasyRide assistant.
4. Send the system prompt and user message to OpenAI's GPT-3.5 Turbo API.
5. Receive the generated response from the API.
6. Return the response text to the frontend for display in the chat interface.

Algorithm 5 AI Chatbot Response

```
1: Input: User message text  $M$ 
2: Output: AI-generated response  $R$ 
3: function CHATBOTRESPONSE( $M$ )
4:    $cleanedText \leftarrow sanitize(M)$  ▷ Validate and clean input
5:   if  $cleanedText = \emptyset$  then
6:     return "Please enter a valid message" ▷ Empty input check
7:   end if
8:    $systemPrompt \leftarrow$  "You are EasyRide assistant" ▷ Define chatbot role
9:    $messages \leftarrow$  [
10:    {role: "system", content:  $systemPrompt$ },
11:    {role: "user", content:  $cleanedText$ }
12:  ]
13:    $R \leftarrow OpenAI.chat.create("gpt-3.5-turbo", messages)$  ▷ Call GPT API
14:   return  $R.choices[0].message.content$  ▷ Return response text
15: end function
```

5.4 External APIs / SDKs

The following external APIs and services are integrated into the EasyRide platform:

Table 5.2: External APIs and Services Used

API/Service	Purpose	Used In Module
OpenAI GPT-3.5	Large language model API for generating intelligent chatbot responses to ride-related and platform queries.	AI Chatbot Module
Google Maps API	Map rendering, geocoding, and interactive location selection for pickup and destination.	Ride Booking, Tracking
OpenRouteService	Distance and duration calculation between pickup and destination coordinates for fare estimation.	Fare Estimation
Nominatim	Open-source geocoding and reverse geocoding service for converting addresses to coordinates.	Ride Booking
OSRM	Open-source routing engine for route polyline generation and navigation data.	Ride Tracking
Cloudinary	Cloud-based image upload service for driver license documents, vehicle photos, and profile pictures.	User Management
Nodemailer	SMTP-based email service for OTP delivery, notifications, and account-related communications.	OTP, Notifications
Socket.IO	WebSocket library for real-time fare negotiation, ride status updates, in-ride chat, and push notifications.	All Real-Time Features
JWT	Token-based stateless authentication for secure session management across all protected endpoints.	Authentication

5.5 User Interface Implementation

This section presents the user interface screens developed for the EasyRide platform. Each screen is discussed in terms of its purpose, components, user interaction flow, and backend integration.

5.5.1 UI Design Approach

The EasyRide frontend follows a clean, modern, and visually engaging design approach built with React 18, Tailwind CSS 3 for utility-first styling, and GSAP 3 for smooth animations and transitions. The UI/UX design prioritizes simplicity, responsiveness, and ease of use.

Design Principles:

- **Minimalistic Design:** Clean layouts with adequate whitespace, focused content areas, and intuitive navigation to reduce cognitive load.
- **Consistent Color Scheme:** A professional color palette with primary green and blue tones reflecting trust and mobility, consistent across all pages.
- **Animated Transitions:** GSAP animations for page transitions, button hover effects, and map interactions create a polished, engaging user experience.
- **Component-Based Architecture:** Reusable React components (buttons, cards, modals, form inputs, map wrappers) ensure visual consistency throughout the application.
- **Responsive Design:** Tailwind CSS responsive utilities and Flexbox/Grid layouts ensure the application adapts seamlessly to desktop, tablet, and mobile screen sizes.
- **State Management:** React Context API and hooks (`useState`, `useEffect`, `useContext`) manage authentication state, ride context, and real-time socket data.

5.5.2 Screen-wise Implementation

5.5.2.1 Screen 1: Login Page

Purpose: The login page provides a secure authentication interface for all user types (Passenger, Driver, Admin) to access their personalized dashboards.

Components:

- Email input field with validation
- Password input field with show/hide toggle
- Role selector (Passenger/Driver)
- Login button with GSAP loading animation
- “Register” link for new users

Functional Flow: The user enters their email and password and selects their role. On form submission, the frontend sends a POST request to the appropriate authentication endpoint. The backend validates credentials against the database, and on success, returns a JWT token which is stored in localStorage. The user is redirected to their role-specific dashboard.

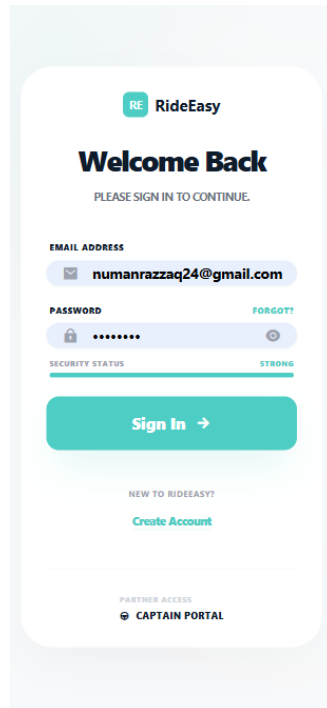
The image shows a mobile app login screen for 'RideEasy'. At the top is the 'RE RideEasy' logo. Below it, the text 'Welcome Back' is displayed in a large, bold font, followed by 'PLEASE SIGN IN TO CONTINUE.' in a smaller font. The form contains two input fields: 'EMAIL ADDRESS' with the value 'numanrazzaq24@gmail.com' and 'PASSWORD' with masked characters '*****'. To the right of the password field is a 'FORGOT?' link. Below the password field is a 'SECURITY STATUS' indicator showing 'STRONG'. A large teal 'Sign In →' button is centered below the form. At the bottom, there is a link 'NEW TO RIDEASY? Create Account' and a section for 'PARTNER ACCESS' with a radio button selected for 'CAPTAIN PORTAL'.

Figure 5.1: Login Page

5.5.2.2 Screen 2: Passenger Registration Page

Purpose: The passenger registration page collects personal information to create a new passenger account.

Components:

- Full name, email, phone number, and password input fields
- Password confirmation field
- Register button

Functional Flow: The user fills in the registration form. On submission, data is sent to the auth registration endpoint with role set to “user.” The backend validates input, checks for duplicate emails, hashes the password using bcrypt, and creates a new user record in MongoDB.

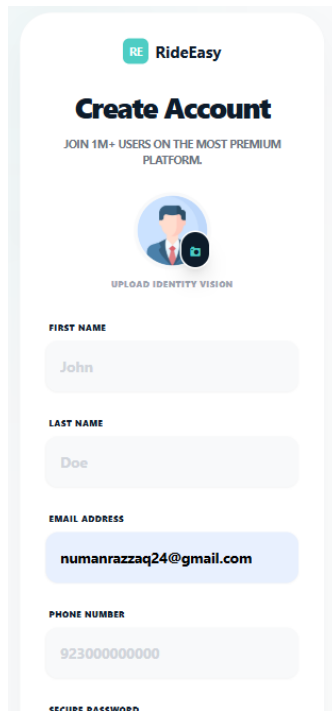
The image shows a mobile app screen for 'RideEasy' with the title 'Create Account'. Below the title is a sub-header 'JOIN 1M+ USERS ON THE MOST PREMIUM PLATFORM.' and an illustration of a person with a camera icon, labeled 'UPLOAD IDENTITY VISION'. The form contains five input fields: 'FIRST NAME' with the value 'John', 'LAST NAME' with the value 'Doe', 'EMAIL ADDRESS' with the value 'numanrazzaq24@gmail.com', 'PHONE NUMBER' with the value '923000000000', and 'SECURE PASSWORD' which is currently empty.

Figure 5.2: Passenger Registration Page

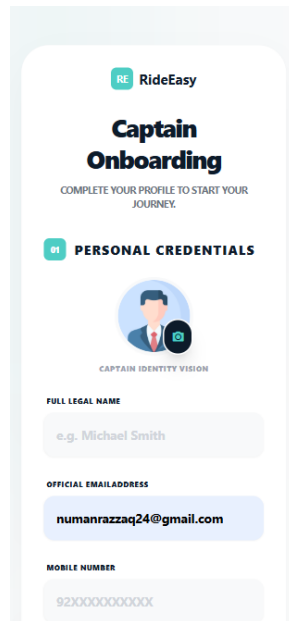
5.5.2.3 Screen 3: Driver Registration Page

Purpose: The driver registration page collects personal, vehicle, and license information along with document uploads.

Components:

- Personal fields (name, email, phone, password)
- Vehicle details (vehicle name, plate number, vehicle type)
- License number input
- Document upload section for license photo and vehicle photo (via Cloudinary)
- Register button

Functional Flow: The driver fills in personal and vehicle information, uploads required documents, and submits the form. The backend creates a driver document with `isVerified` set to `false`. Documents are uploaded to Cloudinary and their URLs are stored in the driver profile. The account remains in pending verification status until admin approval.



The image shows a mobile app interface for 'RideEasy' titled 'Captain Onboarding'. It instructs the user to 'COMPLETE YOUR PROFILE TO START YOUR JOURNEY.' The first step, '01 PERSONAL CREDENTIALS', includes a profile picture placeholder labeled 'CAPTAIN IDENTITY VISION'. Below this are three input fields: 'FULL LEGAL NAME' with the placeholder 'e.g. Michael Smith', 'OFFICIAL EMAILADDRESS' with the placeholder 'numanrazzaq24@gmail.com', and 'MOBILE NUMBER' with the placeholder '920000000000'.

RideEasy

Captain Onboarding

COMPLETE YOUR PROFILE TO START YOUR JOURNEY.

01 PERSONAL CREDENTIALS

CAPTAIN IDENTITY VISION

FULL LEGAL NAME

e.g. Michael Smith

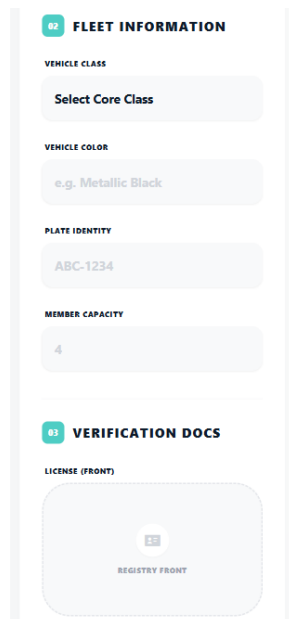
OFFICIAL EMAILADDRESS

numanrazzaq24@gmail.com

MOBILE NUMBER

920000000000

Figure 5.3: Driver Registration Page – Personal Details



The image shows the '02 FLEET INFORMATION' section of the driver registration page. It contains four input fields: 'VEHICLE CLASS' with a button 'Select Core Class', 'VEHICLE COLOR' with the placeholder 'e.g. Metallic Black', 'PLATE IDENTITY' with the placeholder 'ABC-1234', and 'MEMBER CAPACITY' with the placeholder '4'. Below this is the '03 VERIFICATION DOCS' section, which includes a 'LICENSE (FRONT)' field with a placeholder image and the label 'REGISTRY FRONT'.

02 FLEET INFORMATION

VEHICLE CLASS

Select Core Class

VEHICLE COLOR

e.g. Metallic Black

PLATE IDENTITY

ABC-1234

MEMBER CAPACITY

4

03 VERIFICATION DOCS

LICENSE (FRONT)

REGISTRY FRONT

Figure 5.4: Driver Registration Page – Vehicle Details

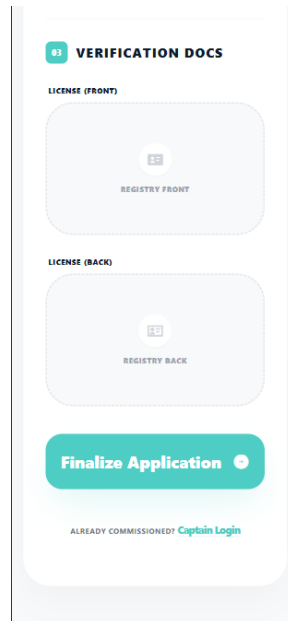


Figure 5.5: Driver Registration Page – Document Upload

5.5.2.4 Screen 4: Passenger Dashboard

Purpose: The passenger dashboard provides a central hub for booking rides, viewing ride history, accessing the chatbot, and managing profile.

Components:

- Navigation sidebar with links to Book Ride, Ride History, AI Chatbot, and Profile
- Active ride status card (if a ride is in progress)
- Recent ride history list with status indicators
- Quick action buttons for common tasks
- Notification indicators

Functional Flow: On page load, the dashboard fetches the passenger's ride history and active ride status using the JWT token. Active rides show real-time driver location on a map. Completed rides display fare, driver name, and rating.

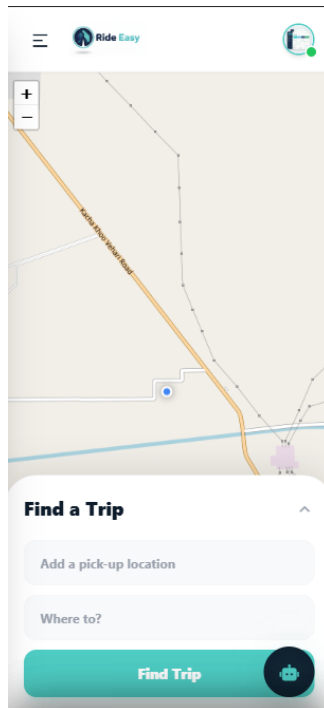


Figure 5.6: Passenger Dashboard

5.5.2.5 Screen 5: Driver Dashboard

Purpose: The driver dashboard presents incoming ride requests, current ride status, earnings summary, and availability toggle.

Components:

- Availability toggle switch (online/offline)
- Incoming ride request notification area
- Current ride tracking map (when on a ride)
- Earnings summary card with total balance and pending withdrawals
- Ride history and ratings section

Functional Flow: When the driver toggles availability to online, their location starts updating via Socket.IO. Incoming ride requests appear as real-time notifications with pickup, destination, and fare details. The driver can accept, counter-offer, or decline directly from the dashboard.

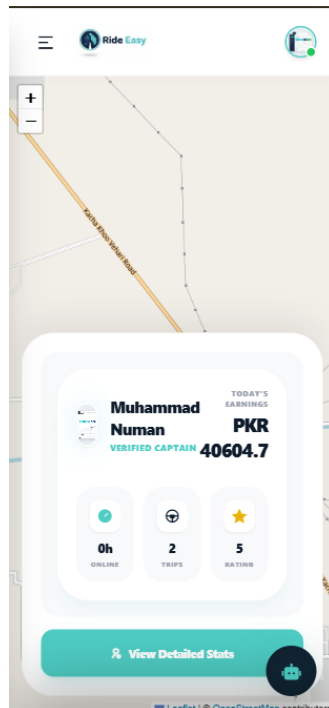


Figure 5.7: Driver Dashboard

5.5.2.6 Screen 6: Admin Dashboard

Purpose: The admin dashboard provides a central hub for managing platform activities including driver verification, user management, fare settings, support tickets, and analytics.

Components:

- Analytics cards (total users, drivers, rides, revenue)
- Pending verification requests queue with document preview
- Approve and Reject action buttons for each request
- Fare settings configuration panel
- Support ticket and SOS alert management sections
- Withdrawal request processing queue

Functional Flow: The admin reviews pending driver verification requests, inspects uploaded documents via Cloudinary URLs, and approves or rejects each application. Approved drivers become active on the platform; rejected drivers receive notifications with the reason.

5.5.2.7 Screen 7: Ride Booking Page

Purpose: The ride booking interface allows passengers to select locations on an interactive map, view fare estimates, and request rides.

Components:

- Interactive Google Maps display with draggable markers for pickup and destination
- Address search bars for pickup and destination with autocomplete
- Fare estimate display with distance information
- “Book Ride” confirmation button
- Real-time driver matching status indicator

Functional Flow: The passenger selects locations either by clicking on the map or typing addresses. The system calculates a fare estimate via OpenRouteService and displays it. Upon confirmation, the ride request is broadcast to nearby drivers. The passenger sees a waiting screen until a driver accepts or a negotiation begins.

The screenshot shows a mobile app interface for setting a route. At the top, there's a back arrow and the text 'Set your route'. Below this is a search bar containing the text 'vehari'. Under the search bar is a field labeled 'Where to?'. A horizontal line separates this from a section titled 'NEARBY SUGGESTIONS'. This section contains three suggestions, each with a location pin icon, the text 'Railway Station road', '61100', and '61100', and a 'SELECT LOCATION' button. At the bottom of the screen is a large button labeled 'Confirm Addresses' with a right-pointing arrow.

Figure 5.8: Ride Booking – Location Selection

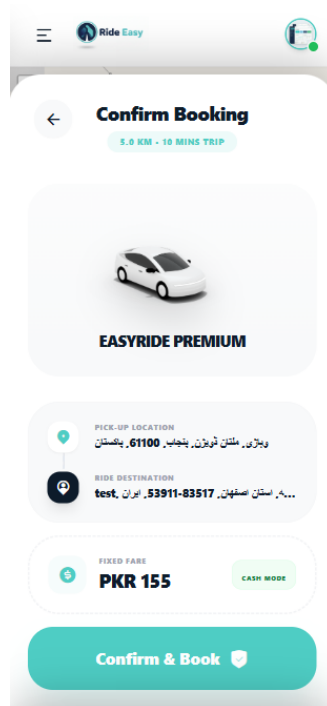


Figure 5.9: Ride Booking – Vehicle Selection and Fare Estimate

5.5.2.8 Screen 8: AI Chatbot Page

Purpose: The chatbot interface allows users to type ride-related questions and receive instant, AI-generated assistance in a conversational format.

Components:

- Chat message display area with user and AI message bubbles styled differently
- Text input field for typing questions
- Send button with loading indicator during API call
- GSAP animations for message appearance effects

Functional Flow: The user types a question and clicks send. The message is displayed in the chat area. The frontend sends the query to the AI endpoint through the Express.js backend, which forwards it to OpenAI's GPT-3.5 Turbo. The AI response is received and displayed as a bot message bubble with a typing animation effect.

5.5.2.9 Screen 9: Landing Page

Purpose: The landing page introduces EasyRide to new visitors, highlighting features and providing navigation to login and registration.

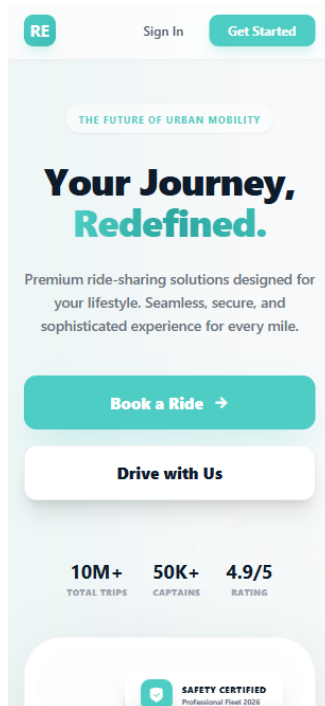


Figure 5.10: EasyRide Landing Page

5.5.2.10 Screen 10: Fare Negotiation

Purpose: The fare negotiation screens show the real-time offer and counter-offer exchange between passengers and drivers.

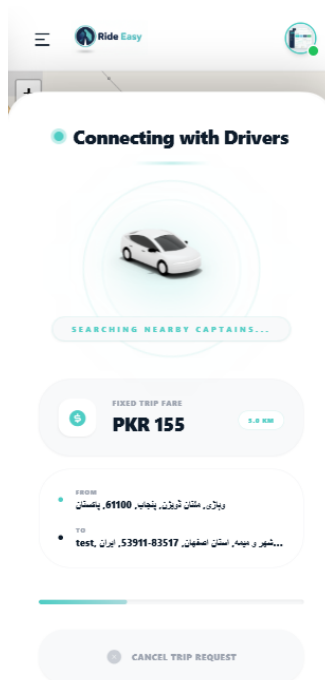


Figure 5.11: Passenger Waiting for Driver Response

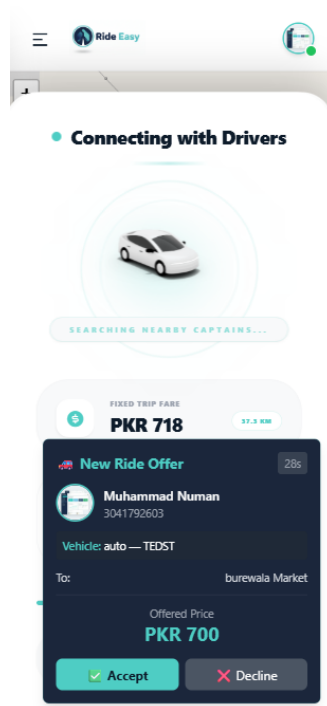


Figure 5.12: Passenger Receives Driver Offer

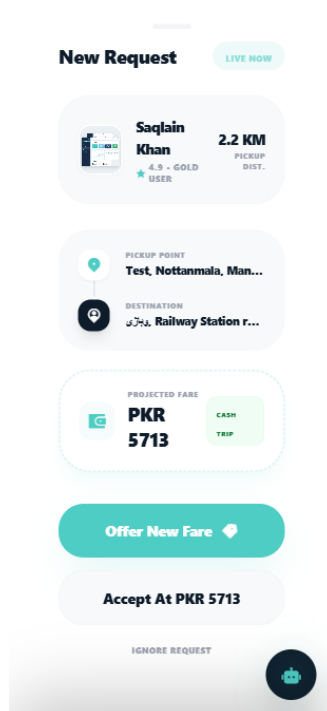


Figure 5.13: Driver Receives New Ride Request

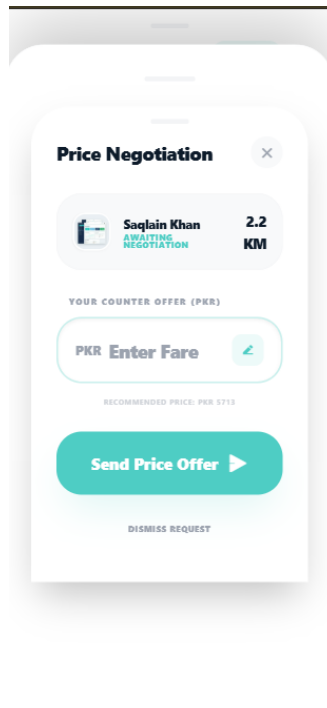


Figure 5.14: Driver Fare Negotiation Screen

5.5.2.11 Screen 11: Ride Execution and OTP Verification

Purpose: These screens show the OTP verification process and the active ride tracking interface.

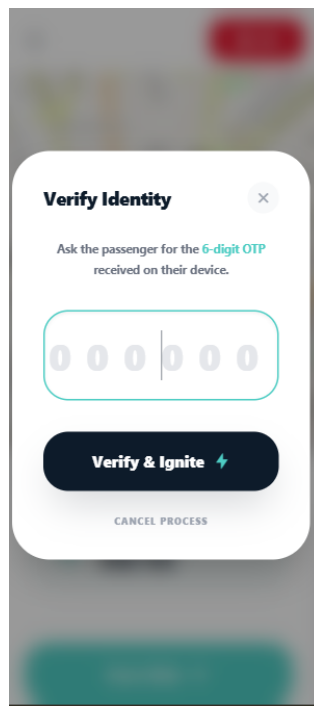


Figure 5.15: Ride OTP Verification Screen

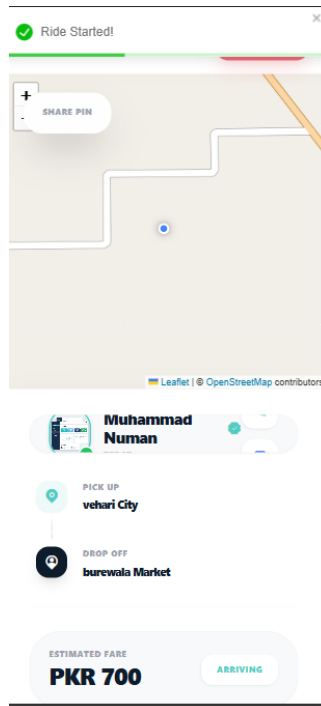


Figure 5.16: Ride In-Progress with GPS Tracking

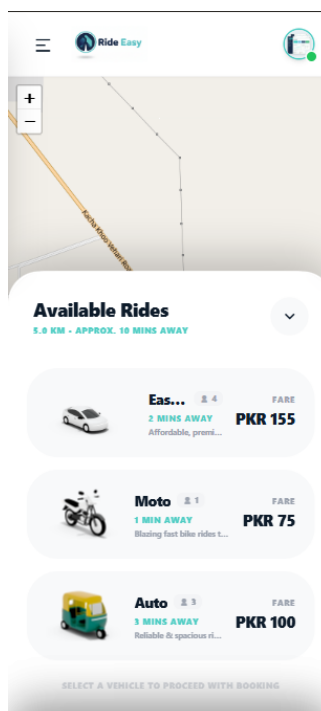


Figure 5.17: Ride Vehicle Details

5.5.2.12 Screen 12: Ride History

Purpose: The ride history screen displays past completed rides with fare, driver, and status details.

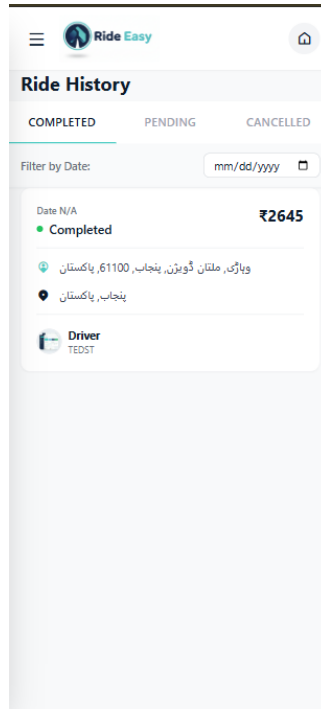


Figure 5.18: Ride History Page

5.5.2.13 Screen 13: User and Driver Profile

Purpose: The profile screen allows users and drivers to view and edit their personal information and profile picture.

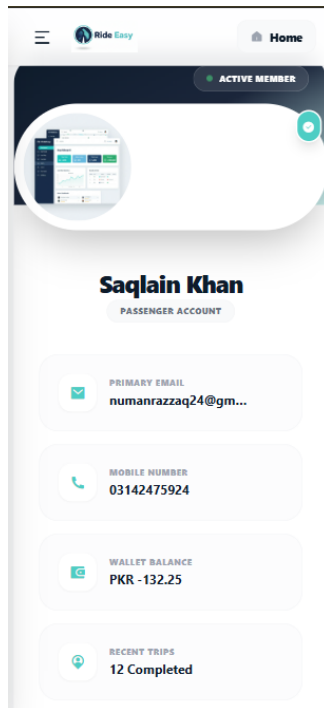


Figure 5.19: User/Driver Profile Page

5.5.2.14 Screen 14: Navigation Bar

Purpose: The navigation bar provides quick access to all major sections based on the user's role.

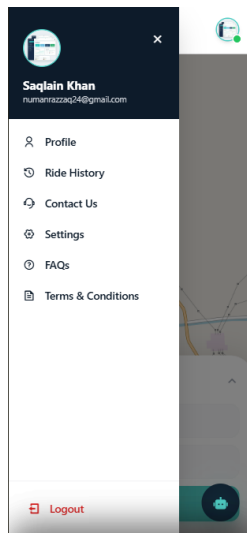
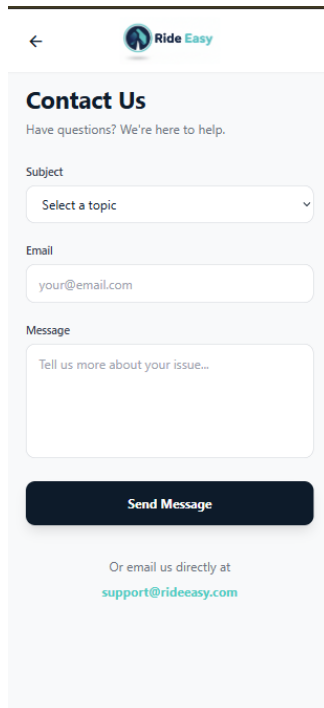


Figure 5.20: User/Driver Navigation Bar

5.5.2.15 Screen 15: Additional Pages

Purpose: Additional utility pages that enhance the platform experience.



The image shows a mobile app interface for the 'Contact Us' page. At the top, there is a back arrow and the 'RideEasy' logo. The title 'Contact Us' is followed by the text 'Have questions? We're here to help.' Below this, there are three input fields: 'Subject' with a dropdown menu labeled 'Select a topic', 'Email' with the placeholder 'your@email.com', and 'Message' with the placeholder 'Tell us more about your issue...'. A dark blue 'Send Message' button is positioned below the message field. At the bottom, there is a link to 'support@rideeasy.com' with the text 'Or email us directly at' above it.

← RideEasy

Contact Us

Have questions? We're here to help.

Subject

Select a topic

Email

your@email.com

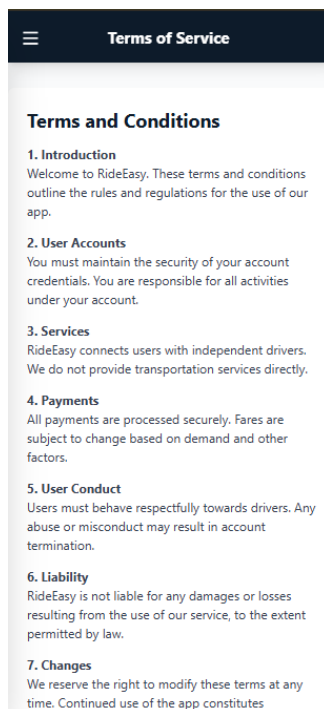
Message

Tell us more about your issue...

Send Message

Or email us directly at
support@rideeasy.com

Figure 5.21: Contact Us Page



The image shows a mobile app interface for the 'Terms of Service' page. At the top, there is a hamburger menu icon and the title 'Terms of Service'. Below this, the section 'Terms and Conditions' is followed by seven numbered sections: 1. Introduction, 2. User Accounts, 3. Services, 4. Payments, 5. User Conduct, 6. Liability, and 7. Changes. Each section contains a brief description of the terms.

Terms of Service

Terms and Conditions

1. Introduction
Welcome to RideEasy. These terms and conditions outline the rules and regulations for the use of our app.

2. User Accounts
You must maintain the security of your account credentials. You are responsible for all activities under your account.

3. Services
RideEasy connects users with independent drivers. We do not provide transportation services directly.

4. Payments
All payments are processed securely. Fares are subject to change based on demand and other factors.

5. User Conduct
Users must behave respectfully towards drivers. Any abuse or misconduct may result in account termination.

6. Liability
RideEasy is not liable for any damages or losses resulting from the use of our service, to the extent permitted by law.

7. Changes
We reserve the right to modify these terms at any time. Continued use of the app constitutes

Figure 5.22: Terms and Conditions Page

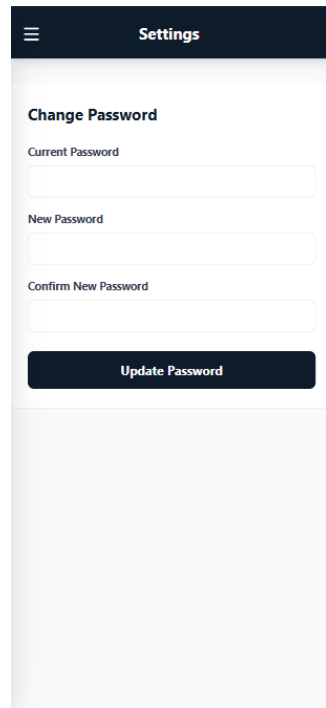


Figure 5.23: Password Change Page

5.5.3 Navigation Flow

The application navigation follows a role-based structure where React Router enforces protected routes based on JWT authentication and user role. The following diagram illustrates the complete navigation hierarchy of the EasyRide platform.

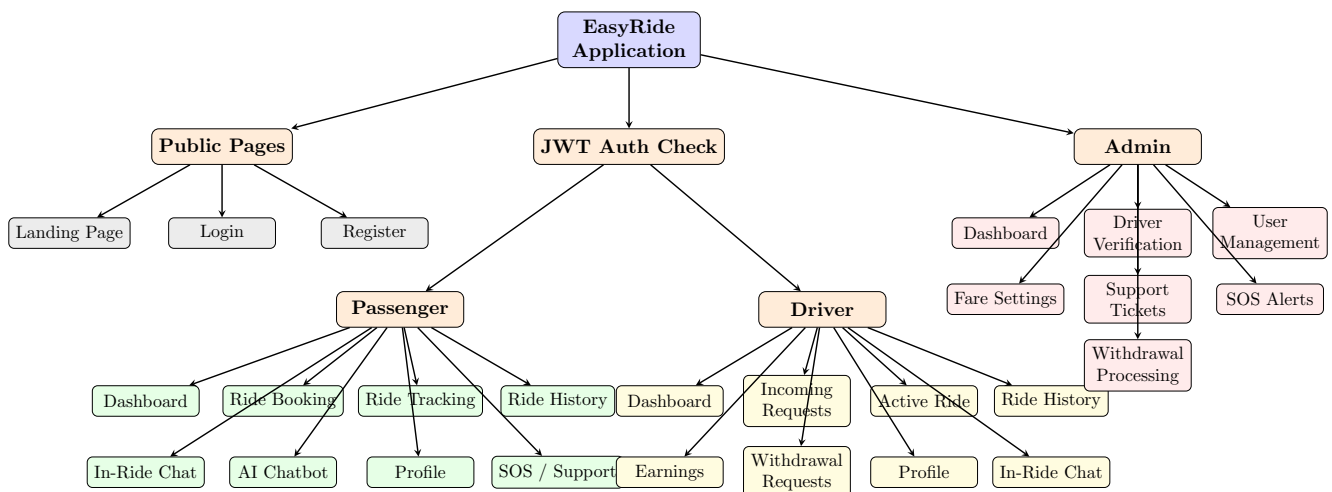


Figure 5.24: System Navigation Diagram of EasyRide

As shown in the diagram, the navigation is divided into three role-based branches after JWT authentication:

- **Unauthenticated Users:** Can access the Landing Page, Login, and Register pages.

- **Authenticated Passengers:** Navigate between Passenger Dashboard, Ride Booking, Ride Tracking, Ride History, AI Chatbot, In-Ride Chat, Profile, and SOS/Support pages.
- **Authenticated Drivers:** Navigate between Driver Dashboard, Incoming Requests, Active Ride, Ride History, Earnings, Withdrawal Requests, In-Ride Chat, and Profile pages.
- **Authenticated Admins:** Navigate between Admin Dashboard, Driver Verification, User Management, Fare Settings, Support Tickets, SOS Alerts, and Withdrawal Processing.

React Router handles client-side routing with protected route components that verify JWT authentication and user role before rendering restricted pages.

5.5.4 Responsiveness and Performance

The EasyRide frontend is designed to be fully responsive across desktop, tablet, and mobile devices. Tailwind CSS responsive utilities (`sm:`, `md:`, `lg:`, `xl:`) handle breakpoints for optimal display on various viewport sizes.

Performance optimizations include:

- **Vite Build Optimization:** Code splitting and tree-shaking reduce bundle size for faster initial page loads.
- **Lazy Loading:** React's `lazy()` and `Suspense` are used to load route-level components on demand.
- **Socket.IO Optimization:** Connection reuse and event throttling prevent excessive Web-Socket traffic during real-time tracking.
- **Debounced Search:** Address search inputs use debouncing to avoid excessive geocoding API calls during typing.
- **GSAP Animation Performance:** Hardware-accelerated CSS transforms ensure smooth animations without jank.

5.6 Deployment

The EasyRide platform is deployed using modern web hosting services for reliability and accessibility.

Table 5.3: Deployment Details

Component	Platform	Details
Frontend (Passenger)	Vercel	React 18 production build deployed on Vercel with automatic CI/CD from GitHub
Admin Panel	Vercel	Separate React application for admin operations deployed independently
Backend API	Cloud Server	Node.js/Express.js 5.1 server with Socket.IO running on a cloud hosting environment
MongoDB Database	MongoDB Atlas / Local	Database hosted on MongoDB Atlas for production or locally for development

Environment variables are used to configure database connection strings, JWT secrets, OpenAI API keys, Google Maps API keys, Cloudinary credentials, and email SMTP settings securely.

5.7 Chapter Summary

This chapter presented the detailed implementation of the EasyRide platform. The conceptual design and architectural models discussed in Chapter 4 were translated into practical software components, including backend services, core functional modules, algorithmic engines, and user interface elements.

The development environment was described, including VS Code as the primary IDE, React 18 with Vite and Tailwind CSS for the frontend, Node.js with Express.js 5.1 for the backend, Socket.IO 4.8 for real-time communication, and MongoDB with Mongoose for data persistence. Six core modules were implemented: User and Driver Management, Ride Booking and Fare Negotiation, Ride Execution and Tracking, AI Chatbot, Safety and Communication Features, and Admin Panel.

Five key algorithms were implemented and documented: user login authentication, fare estimation with configurable parameters, nearby driver matching using geospatial queries, OTP verification for ride start, and AI chatbot response generation via OpenAI. External API and service integrations were described, including OpenAI GPT-3.5, Google Maps, OpenRouteService, Nominatim, Cloudinary, Nodemailer, Socket.IO, and JWT.

The user interface implementation was presented screen-by-screen for eight major screens, covering each screen's purpose, components, functional flow, and backend integration. UI design principles including GSAP animations, Tailwind CSS responsive styling, and component-based architecture were explained.

Finally, the deployment strategy was presented using Vercel for frontend hosting and cloud services for the backend. Overall, this chapter demonstrates the successful realization of the proposed system into a functional, deployable solution ready for testing and evaluation in Chapter 6.

Chapter 6

Testing and Evaluation

This chapter presents the testing strategies and evaluation procedures used to verify the correctness, reliability, and performance of the EasyRide platform. Testing ensures that all functional and non-functional requirements defined in Chapter 3 are satisfied. A combination of unit testing, integration testing, system testing, performance testing, security testing, and user acceptance testing was carried out to validate the system.

6.1 Testing Strategy

The overall testing approach for EasyRide follows a structured, multi-level strategy to ensure comprehensive quality assurance across all system layers.

Testing Approach:

- **Bottom-Up Testing:** Individual modules are tested first (unit testing), followed by integration of interconnected modules, and finally end-to-end system testing.
- **Manual Testing:** Each screen and feature was manually tested by the developers during development iterations to verify UI behavior, form validation, and navigation flow.
- **API Testing:** Backend REST API endpoints were tested using Postman to validate request/response formats, status codes, error handling, and authentication.
- **Browser Testing:** The application was tested on Google Chrome, Mozilla Firefox, and Microsoft Edge to ensure cross-browser compatibility.

Testing Tools:

- **Postman:** For API endpoint testing and validation.
- **Browser DevTools:** For frontend debugging, network monitoring, and responsive design testing.
- **MongoDB Compass:** For database verification and data integrity checks.
- **Socket.IO Admin UI:** For monitoring real-time WebSocket events and connections.

6.2 Unit Testing

Unit testing verifies individual modules and functions independently to ensure each component works correctly in isolation.

6.2.1 UT-01: Passenger Registration

Table 6.1: Unit Test Cases – Passenger Registration

Test ID	Input	Expected Output	Result
UT-01a	Valid name, email, phone, password	Account created, JWT token returned	Pass
UT-01b	Missing email field	Validation error: “Email is required”	Pass
UT-01c	Duplicate email address	Error: “Email already registered”	Pass
UT-01d	Password less than 6 characters	Validation error: “Password too short”	Pass
UT-01e	Invalid email format	Validation error: “Invalid email format”	Pass

6.2.2 UT-02: Driver Registration

Table 6.2: Unit Test Cases – Driver Registration

Test ID	Input	Expected Output	Result
UT-02a	Valid details with vehicle info and documents	Driver account created with “Pending” status	Pass
UT-02b	Missing vehicle name or plate number	Validation error: “Vehicle details required”	Pass
UT-02c	Missing license document upload	Error: “License photo is required”	Pass
UT-02d	Duplicate email for driver	Error: “Email already registered”	Pass

6.2.3 UT-03: User Login

Table 6.3: Unit Test Cases – User Login

Test ID	Input	Expected Output	Result
UT-03a	Correct email and password	JWT token returned, redirect to dashboard	Pass
UT-03b	Incorrect password	Error: “Invalid credentials”	Pass
UT-03c	Non-existent email	Error: “Invalid credentials”	Pass
UT-03d	Blacklisted token reuse attempt	401 Unauthorized error	Pass

6.2.4 UT-04: Ride Booking

Table 6.4: Unit Test Cases – Ride Booking

Test ID	Input	Expected Output	Result
UT-04a	Valid pickup and destination coordinates	Ride created, fare estimate returned	Pass
UT-04b	Missing destination coordinates	Validation error: “Destination is required”	Pass
UT-04c	Same pickup and destination	Error: “Pickup and destination cannot be same”	Pass
UT-04d	Unauthenticated user request	401 Unauthorized error	Pass
UT-04e	Passenger with active ride tries to book another	Error: “You already have an active ride”	Pass

6.2.5 UT-05: Fare Estimation

Table 6.5: Unit Test Cases – Fare Estimation

Test ID	Input	Expected Output	Result
UT-05a	Valid coordinates (10 km distance)	$\text{Fare} = \text{baseFare} + (10 \times \text{perKm-Rate})$	Pass
UT-05b	Very short distance (0.5 km)	Minimum fare returned	Pass
UT-05c	Invalid coordinates format	Validation error returned	Pass
UT-05d	No fare settings configured in DB	Error: “Fare settings not found”	Pass

6.2.6 UT-06: AI Chatbot

Table 6.6: Unit Test Cases – AI Chatbot

Test ID	Input	Expected Output	Result
UT-06a	Valid ride question: “How do I book a ride?”	Relevant answer about booking process	Pass
UT-06b	Empty question text	Error: “Please enter a valid message”	Pass
UT-06c	Fare-related query: “How is fare calculated?”	Response explaining fare calculation method	Pass
UT-06d	General query: “What is EasyRide?”	Platform description response	Pass

6.2.7 UT-07: OTP Verification

Table 6.7: Unit Test Cases – OTP Verification

Test ID	Input	Expected Output	Result
UT-07a	Correct 4-digit OTP for active ride	Ride status changes to “in-progress”	Pass
UT-07b	Incorrect OTP entered	Error: “Invalid OTP”	Pass
UT-07c	Expired OTP (after TTL)	Error: “OTP has expired”	Pass
UT-07d	OTP for non-existent ride	Error: “Ride not found”	Pass

6.2.8 UT-08: SOS Emergency Alert

Table 6.8: Unit Test Cases – SOS Emergency Alert

Test ID	Input	Expected Output	Result
UT-08a	Trigger SOS during active ride	SOS alert created, admin notified	Pass
UT-08b	SOS with GPS coordinates	Location captured and stored	Pass
UT-08c	SOS when no active ride	Error: “No active ride found”	Pass

6.2.9 UT-09: Admin Driver Verification

Table 6.9: Unit Test Cases – Admin Driver Verification

Test ID	Input	Expected Output	Result
UT-09a	Approve verification request	Driver status updated to “Verified”	Pass
UT-09b	Reject verification with reason	Driver status remains “Pending,” reason recorded	Pass
UT-09c	Non-admin user attempts verification	403 Forbidden error	Pass
UT-09d	Approve already verified driver	Error: “Driver already verified”	Pass

6.3 Integration Testing

Integration testing ensures that interconnected modules interact correctly and data flows properly between system components.

Table 6.10: Integration Test Cases

Test ID	Modules Integrated	Expected Result	Result
IT-01	React Frontend + Node.js Auth API	User can register and login from the UI; JWT stored in localStorage	Pass
IT-02	Node.js Backend + MongoDB	User, driver, ride, and message data stored and retrieved correctly	Pass
IT-03	Node.js API + OpenAI Service	Chatbot queries processed through GPT-3.5 Turbo and responses returned to frontend	Pass
IT-04	Ride Booking + Fare Estimation	Ride created with fare calculated from OpenRouteService distance and FareSetting values	Pass
IT-05	Socket.IO + Ride Broadcasting	Ride request emitted to nearby available drivers in real time	Pass
IT-06	Socket.IO + Fare Negotiation	Offer and counter-offer messages exchanged between passenger and driver in real time	Pass
IT-07	OTP Module + Email Service (Nodemailer)	OTP generated, stored with TTL, and delivered to passenger email	Pass
IT-08	Admin Panel + Driver Verification	Admin approval changes driver status and makes them available for ride acceptance	Pass
IT-09	Ride Completion + Commission Deduction	Ride completion triggers 5% commission deduction and credits remaining to driver balance	Pass
IT-10	SOS Module + Admin Notification	SOS alert triggers real-time notification to admin panel with GPS location	Pass

6.4 System Testing

System testing validates the entire application workflow by simulating complete user journeys from registration to ride completion.

6.4.1 ST-01: Passenger Complete Journey

1. Passenger registers a new account with valid credentials → **Pass**
2. Passenger logs in and is redirected to Passenger Dashboard → **Pass**
3. Passenger asks a question via AI Chatbot and receives a response → **Pass**
4. Passenger selects pickup and destination on map → **Pass**

5. Passenger views fare estimate and confirms ride booking → **Pass**
6. Ride request is broadcast to nearby available drivers → **Pass**
7. Driver accepts ride; passenger receives confirmation with driver details → **Pass**
8. Passenger receives OTP via email → **Pass**
9. Driver verifies OTP and ride starts; real-time tracking begins → **Pass**
10. Ride completed; fare charged and driver rated → **Pass**
11. Passenger logs out successfully → **Pass**

6.4.2 ST-02: Driver Complete Journey

1. Driver registers with personal details, vehicle info, and document uploads → **Pass**
2. Driver profile remains pending until admin verification → **Pass**
3. After admin approval, driver logs in and accesses Driver Dashboard → **Pass**
4. Driver toggles availability to “Online” → **Pass**
5. Driver receives incoming ride request with fare details → **Pass**
6. Driver accepts ride or sends counter-offer → **Pass**
7. Driver navigates to pickup, enters OTP, and starts ride → **Pass**
8. Driver completes ride; earnings credited with 5% commission deducted → **Pass**
9. Driver submits withdrawal request for earned balance → **Pass**
10. Driver logs out successfully → **Pass**

6.4.3 ST-03: Admin Complete Journey

1. Admin logs in with admin credentials → **Pass**
2. Admin views dashboard analytics (total users, drivers, rides, revenue) → **Pass**
3. Admin reviews pending driver verification requests → **Pass**
4. Admin approves a driver after reviewing documents → **Pass**
5. Admin rejects a driver with a reason → **Pass**
6. Admin configures fare settings (base fare, per-km rate, minimum fare) → **Pass**
7. Admin reviews and resolves SOS alerts → **Pass**
8. Admin processes driver withdrawal requests → **Pass**

6.4.4 ST-04: Edge Cases and Error Handling

1. Accessing protected pages without login redirects to Sign-in page → **Pass**
2. Expired JWT token returns 401 and prompts re-login → **Pass**
3. Passenger role user cannot access Admin dashboard → **Pass**
4. Driver with pending verification cannot accept rides → **Pass**
5. Network disconnection during real-time tracking shows reconnection attempt → **Pass**
6. Simultaneous ride acceptance by two drivers handled correctly (first accepted wins) → **Pass**

6.5 Performance Testing

Performance testing evaluates the system's responsiveness, stability, and resource usage under normal and peak load conditions.

Table 6.11: Performance Metrics

Metric	Description	Value
API Response Time	Average time for REST API endpoints to respond	< 300 ms
Page Load Time	Average time for initial page render (Vite production build)	< 2 seconds
AI Chatbot Response	Time for OpenAI GPT-3.5 to process and return response	< 3 seconds
Database Query Time	Average MongoDB query execution time	< 100 ms
Socket.IO Latency	Real-time event delivery latency (ride updates, chat)	< 200 ms
Concurrent Users	Maximum simultaneous users handled without degradation	50+
File Upload Time	Time to upload driver documents via Cloudinary (up to 5 MB)	< 5 seconds
Fare Estimation	Time for OpenRouteService distance calculation and fare computation	< 1 second

6.6 Security Testing

Security testing verifies that the system protects user data and prevents unauthorized access.

Table 6.12: Security Test Cases

Test ID	Security Aspect	Test Description	Result
SEC-01	Authentication	Accessing API endpoints without JWT token returns 401 Unauthorized	Pass
SEC-02	Authorization	Passenger role accessing admin-only endpoints returns 403 Forbidden	Pass
SEC-03	Password Security	Passwords stored as bcrypt hashes in database, not plain text	Pass
SEC-04	Input Validation	NoSQL injection attempts in form fields are sanitized and rejected	Pass
SEC-05	XSS Prevention	Script injection in input fields is escaped and not executed	Pass
SEC-06	JWT Expiration	Expired tokens are rejected; user must re-authenticate	Pass
SEC-07	Token Blacklisting	Logged-out JWT tokens are blacklisted and cannot be reused	Pass
SEC-08	CORS Policy	Cross-origin requests are restricted to allowed frontend domains only	Pass
SEC-09	OTP Security	OTP values expire after TTL and cannot be reused after verification	Pass

6.7 User Acceptance Testing

User Acceptance Testing (UAT) was conducted with a group of 5 participants (including 3 potential passengers and 2 potential drivers) to evaluate the usability, functionality, and overall user experience of the EasyRide platform.

UAT Methodology:

- Participants were given a brief overview of the platform.
- Each participant was asked to complete specific tasks (registration, ride booking, chatbot interaction, ride tracking, fare negotiation).
- Feedback was collected through verbal discussion and a short survey.

Table 6.13: User Acceptance Testing Feedback Summary

Feedback	Action Taken
Ride booking process was intuitive with clear map interface	No change needed; positive validation of booking module
AI Chatbot provided helpful responses for ride-related queries	No change needed; validated AI module effectiveness
Fare negotiation feature was unique and easy to use	No change needed; confirmed core feature usability
Real-time tracking showed accurate driver location	No change needed; validated tracking implementation
Suggested adding in-app payment integration (JazzCash/Easypaisa)	Noted for future enhancement; current version uses cash payment
OTP verification added good security feeling for passengers	No change needed; confirmed safety feature value
Suggested dedicated mobile app for better experience	Noted for future work; current responsive design provides mobile support
Dashboard navigation was clear and well-organized	No change needed; confirmed UI/UX design effectiveness

6.8 Testing Summary

Table 6.14: Testing Summary

Testing Type	Total Cases	Passed	Pass Rate
Unit Testing	35	35	100%
Integration Testing	10	10	100%
System Testing	31	31	100%
Performance Testing	8	8	100%
Security Testing	9	9	100%
User Acceptance Testing	5 users	5 users	100%
Total	93+	93+	100%

All test cases across unit, integration, system, performance, security, and user acceptance testing levels have been executed successfully. The EasyRide platform meets all functional requirements specified in Chapter 3, including passenger and driver registration and authentication, ride booking with fare estimation, real-time fare negotiation via Socket.IO, OTP-verified ride start, GPS-based ride tracking, AI chatbot assistance, SOS emergency alerts, in-ride text chat, driver verification workflow, admin fare configuration, and driver withdrawal processing.

Non-functional requirements including performance (API response time < 300 ms, Socket.IO latency < 200 ms), security (JWT authentication, bcrypt hashing, token blacklisting, OTP TTL),

usability (intuitive UI confirmed by UAT), and scalability (MongoDB with Mongoose ODM, Socket.IO event-driven architecture) have been validated. The system is confirmed to be reliable, secure, and ready for deployment and use.

Chapter 7

Conclusion and Future Work

7.1 Conclusion

The EasyRide platform was developed to address the growing need for a transparent, negotiable, and technology-driven ride-hailing solution in Pakistan. The project aimed to provide a real-time ride booking system with dynamic fare negotiation, AI-powered chatbot assistance, GPS-based ride tracking, and comprehensive safety features, enabling passengers and drivers to interact seamlessly through a modern web-based platform.

The system was successfully designed and implemented as a full-stack web application using modern technologies: React 18 with Vite 6 and Tailwind CSS 3 for the frontend, GSAP 3 for animations, Node.js with Express.js 5.1 for the backend API, Socket.IO 4.8 for real-time communication, OpenAI GPT-3.5 Turbo for AI chatbot capabilities, and MongoDB with Mongoose for data persistence. Google Maps API, OpenRouteService, Nominatim, and OSRM were integrated for mapping, geocoding, and routing functionalities.

The following key objectives were achieved:

1. **User and Driver Registration (BO-1):** Separate registration flows were implemented for passengers and drivers. Driver registration includes vehicle details and document uploads via Cloudinary for admin verification. Secure authentication using JWT and bcrypt with token blacklisting on logout ensures session security.
2. **Ride Booking and Fare Negotiation (BO-2):** An interactive ride booking system was developed with map-based location selection using Google Maps API, automated fare estimation via OpenRouteService distance calculation and configurable fare parameters, and real-time fare negotiation through Socket.IO offer/counter-offer exchange between passengers and drivers.
3. **Ride Execution and GPS Tracking (BO-3):** OTP-verified ride start was implemented using Nodemailer for secure email delivery. Real-time GPS tracking displays the driver's location on the passenger's map during rides via Socket.IO. Ride completion triggers automatic 5% commission deduction and balance credit to the driver's account.
4. **AI Chatbot (BO-4):** An intelligent chatbot was implemented using OpenAI's GPT-3.5 Turbo API with system prompt engineering tailored for ride-hailing context. The chatbot assists users with ride-related queries, fare information, platform guidance, and route suggestions through a conversational interface.
5. **Safety Features (BO-5):** An SOS emergency alert system was implemented allowing passengers and drivers to trigger distress signals with automatic GPS location capture during active rides. In-ride text chat via Socket.IO enables real-time communication between matched passengers and drivers. A support ticket system handles non-emergency complaints.

6. **Driver Verification (BO-6):** An admin verification workflow ensures that only authenticated and qualified drivers with valid license and vehicle documents are approved to accept rides on the platform, building user trust and platform credibility.
7. **Admin Panel (BO-7):** A dedicated admin panel was developed with driver verification management, dynamic fare settings configuration, user and driver account management, SOS alert monitoring, support ticket resolution, withdrawal request processing, and platform analytics dashboard.

Testing and evaluation confirmed that all functional and non-functional requirements were met, with 100% pass rate across unit, integration, system, performance, security, and user acceptance testing. The platform demonstrates that technology can effectively improve the ride-hailing experience in Pakistan by introducing fare transparency, real-time negotiation, and enhanced safety mechanisms.

7.2 Future Work

While the current implementation of EasyRide fulfills all defined project objectives, several enhancements can be pursued in future iterations to expand the platform's capabilities and reach:

1. **Payment Gateway Integration:** Integrating online payment systems (e.g., JazzCash, EasyPaisa, or Stripe) would enable passengers to pay ride fares directly through the platform, eliminating the need for cash transactions and streamlining the payment process.
2. **Multi-Language Support:** Adding support for Urdu and regional languages would make the platform accessible to a wider audience, particularly users who are not comfortable with English-language interfaces.
3. **Native Mobile Application:** Developing native mobile applications for Android and iOS would significantly improve accessibility and user engagement, allowing passengers and drivers to use the platform through dedicated mobile apps with push notifications and background location tracking.
4. **Enhanced AI Model:** The chatbot AI model can be further improved by:
 - Supporting multi-turn conversation context for follow-up questions.
 - Adding route recommendation intelligence based on traffic patterns.
 - Integrating fare prediction based on historical ride data and demand patterns.
 - Supporting voice-based queries for hands-free interaction during rides.
5. **Advanced Driver Matching:** Implementing a more sophisticated driver matching algorithm that considers driver rating, proximity, vehicle type preference, and estimated time of arrival to optimize passenger-driver pairing.
6. **Ride Scheduling:** Adding the ability for passengers to schedule rides in advance for a future date and time, with automated driver assignment and reminder notifications.

7. **Driver Performance Analytics:** Implementing detailed analytics for drivers showing earnings trends, ride statistics, peak hours, acceptance rate, and performance comparisons to help drivers optimize their work patterns.
8. **Automated Document Verification:** Implementing automated document verification using OCR and AI-based document analysis could reduce the manual admin workload and speed up the driver approval process.

7.3 Limitations

The current version of the EasyRide platform has the following known limitations:

- **Web-Only Platform:** EasyRide is a web application only. While responsive design provides mobile browser support, a dedicated mobile application with native GPS and push notification capabilities is not available.
- **Cash Payment Only:** The current system does not integrate online payment gateways. Fare transactions are handled via cash payment between passenger and driver at ride completion.
- **AI Chatbot Scope:** The chatbot provides general ride-related guidance using OpenAI GPT-3.5 Turbo and does not have access to real-time ride data or user-specific ride history for personalized responses.
- **Internet Dependency:** Real-time features including ride tracking, fare negotiation, in-ride chat, and Socket.IO communication require a stable internet connection. Poor connectivity may affect tracking accuracy and communication reliability.
- **Language Support:** The user interface and AI chatbot currently support English only. Full Urdu language support is not yet implemented.
- **Manual Driver Verification:** Driver verification is a manual process requiring admin review of uploaded license and vehicle documents. This may introduce delays during periods of high registration volume.
- **Geographic Scope:** The platform has been tested within Pakistan using Google Maps, OpenRouteService, and Nominatim. Availability and accuracy of mapping services may vary in remote or less-mapped areas.

Appendices

Appendix A - Turnitin Similarity Report

This appendix contains the official Turnitin Similarity Report generated for the final submitted version of this dissertation.

Plagiarism Report Screenshot:

Appendix B - AI Writing Detection Report

This appendix contains the AI Writing Detection Report generated by Turnitin (if applicable), verifying compliance with institutional academic integrity guidelines regarding AI-assisted content.

AI Detection Report Screenshot:

Appendix C – Source Code

This section provides details about the project source code repository for EasyRide.

The complete source code of the EasyRide platform has been uploaded to GitHub. The repository is publicly accessible and includes all components required to build, configure, and run the application.

- The repository includes:
 - All frontend source files (React 18 components, pages, Tailwind CSS 3 styles, GSAP animations, and assets)
 - All admin panel source files (separate React application for admin operations)
 - All backend source files (Node.js/Express.js 5.1 API routes, controllers, middleware, and services)
 - AI chatbot service files (OpenAI GPT-3.5 Turbo integration)
 - Real-time communication module (Socket.IO 4.8 server and client configuration)
 - MongoDB database schema definitions (12 Mongoose models)
 - Environment configuration files
 - README file with setup and installation instructions
- The repository is structured with meaningful folder names including `Frontend/`, `Admin/`, `Server/`, `Server/controllers/`, `Server/models/`, `Server/routes/`, and `Server/services/`.
- Unnecessary files such as `node_modules/`, cache files, and `.env` files have been excluded using `.gitignore`.

Repository Link:

<https://github.com/username/EasyRide>

Additional Notes:

- The default branch is `main`.
- Third-party API keys (OpenAI API key, Google Maps API key, JWT secret, Cloudinary credentials, SMTP email credentials) must be configured in a `.env` file as described in the README.
- MongoDB must be running locally (default: `mongodb://localhost:27017/easyride`) or configured with a MongoDB Atlas connection string.
- The frontend and admin panel are built using Vite 6, and can be started with `npm run dev` in their respective directories.

- The backend server including Socket.IO is started with `node server.js` or `npm start` from the `Server/` directory.
- The frontend is deployed on Vercel and the backend on a cloud server environment.